

35. フル MLE 同定本体

solar_ws0129-main

2026-07-29

Contents

1 対象	4
2 このファイルを読む理由	4
3 呼び出し関係	4
3.1 どこから呼ばれるか	4
3.2 次に読むと理解が進むファイル	4
3.3 同一リポジトリ内の主要依存	4
4 ソース冒頭の把握	4
4.1 代表コード断片	4
5 主要構造の読み取り	5
5.1 主要クラス・関数	5
5.2 ファイルを上から読んだときの定義順	7
5.3 import 群の詳細	8
6 このファイルを読む前に必要な基礎知識	10
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	10
6.2 名前、代入、型、参照	11
6.3 関数、引数、戻り値、スコープ	11
6.4 module、package、import、console_scripts	11
6.5 例外、try/finally、with、資源解放	12
6.6 list、dict、deque、NumPy 配列、pandas DataFrame	12
6.7 CLI、PowerShell、Bash、環境変数、終了コード	12
6.8 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明	12
6.9 天候、route、補間、時刻、単位	13
6.10 freshness、filter、guard、fallback、fail-safe	13
6.11 CSV/YAML の data contract と検証	13
6.12 単体試験、SILS、replay、観測可能性	13
7 関数・クラスを上から順に解説	14
7.1 L123 関数 neutralize_identification_scalars	14

7.2	L135 関数 <code>resolve_relative</code>	15
7.3	L144 関数 <code>stage</code>	16
7.4	L148 関数 <code>load_manifest</code>	16
7.5	L168 関数 <code>relpath_from</code>	18
7.6	L177 関数 <code>tex_path_fragment</code>	19
7.7	L199 関数 <code>tex_text_fragment</code>	20
7.8	L216 関数 <code>load_yaml_if_exists</code>	21
7.9	L223 関数 <code>declared_control_stop_km</code>	21
7.10	L245 関数 <code>_terminal_anchor_from_payload</code>	23
7.11	L292 関数 <code>_append_reason_column</code>	25
7.12	L300 関数 <code>resolve_manifest_context</code>	26
7.13	L306 関数 <code>resolve_manifest_context.opt_path</code>	29
7.14	L413 関数 <code>hampel_mask</code>	30
7.15	L425 関数 <code>apply_sensor_quality_annotations</code>	31
7.16	L520 関数 <code>polarization_current_trace</code>	34
7.17	L544 関数 <code>fit_battery_polarization</code>	36
7.18	L682 関数 <code>apply_battery_polarization</code>	39
7.19	L706 関数 <code>resolve_fit_plan</code>	40
7.20	L750 関数 <code>resolve_identification_output_layout</code>	42
7.21	L777 関数 <code>identification_profile_output_path</code>	43
7.22	L789 関数 <code>load_ocv_df</code>	44
7.23	L797 関数 <code>build_source_map_assets</code>	45
7.24	L801 関数 <code>build_source_map_assets.rel</code>	46
7.25	L821 関数 <code>apply_actual_event_annotations</code>	47
7.26	L863 関数 <code>truncate_at_retire_event</code>	49
7.27	L892 関数 <code>normalize_generic_log</code>	51
7.28	L994 関数 <code>build_terminal_anchor</code>	54
7.29	L1029 関数 <code>terminal_metrics</code>	56
7.30	L1078 関数 <code>replay_day_metrics</code>	58
7.31	L1099 関数 <code>identification_selection_score</code>	59
7.32	L1135 関数 <code>condition_vehicle_fit_on_measured_pv</code>	61
7.33	L1155 関数 <code>calibrate_solar_measurement_to_pack</code>	62
7.34	L1271 関数 <code>_shift_acceleration_within_segments</code>	65
7.35	L1283 関数 <code>_acceleration_trace_from_speed</code>	66
7.36	L1321 関数 <code>_bilinear_interp_array</code>	67
7.37	L1341 関数 <code>_map_efficiency_array</code>	69
7.38	L1364 関数 <code>_motion_predictions_for_acceleration</code>	70
7.39	L1431 関数 <code>fit_acceleration_timestamp_alignment</code>	73
7.40	L1631 関数 <code>_grade_from_smoothed_elevation</code>	77
7.41	L1653 関数 <code>fit_grade_observation_alignment</code>	78
7.42	L1750 関数 <code>fit_grade_observation_alignment.rmse</code>	82

7.43	L1886 関数 <code>pv_leave_one_day_out_validation</code>	83
7.44	L1950 関数 <code>pv_leave_one_day_out_validation.rmse</code>	86
7.45	L1964 関数 <code>add_end_to_end_metrics</code>	86
7.46	L1973 関数 <code>add_battery_conditional_metrics</code>	87
7.47	L1981 関数 <code>write_replay_csv</code>	88
7.48	L1998 関数 <code>apply_fit_to_cfg</code>	89
7.49	L2113 関数 <code>update_profile</code>	93
7.50	L2157 関数 <code>update_profile_artifact_references</code>	94
7.51	L2180 関数 <code>write_terminal_consistency_from_anchor</code>	96
7.52	L2318 関数 <code>sync_canonical_fullsim_profile</code>	98
7.53	L2353 関数 <code>write_generic_summary</code>	100
7.54	L2422 関数 <code>write_generic_report</code>	102
7.55	L3075 関数 <code>main</code>	110
7.56	CLI 引数	114
8	処理の流れ	115
9	このファイルの位置づけ	115

1 対象

項目	内容
ファイル	scripts/run_vehicle_identification.py
ソース SHA-256	beea09d61241299016d808d93b3ce656b2bccebb2d44dade39cebff6710f289a
種別	Python
区分	identification
役割	実車ログ、weather、grounded maps、battery/PV/vehicle モデルを用いて MLE 同定 を実行する大型スクリプト。
起動文脈	fit action の本丸。

2 このファイルを読む理由

実車ログ、weather、grounded maps、battery/PV/vehicle モデルを用いて MLE 同定を実行する大型スクリプト。

- 出力 tag を持つ immutable run を作る。
- adopt までは canonical profile を上書きしない流れに対応する。

3 呼び出し関係

3.1 どこから呼ばれるか

- scripts/solar_control.sh

3.2 次に読むと理解が進むファイル

- scripts/generate_fit_fullsim_report.py
- scripts/tune_upper_planner_weights.py

3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
}

def neutralize_identification_scalars(model):
    """Fit each calibration factor once on top of the declared physical maps."""
    model.p.panel_gain = 1.0
    model.p.drive_eff_scale = 1.0
```

```

model.p.regen_eff_scale = 1.0
model.p.regen_utilization = 1.0
model.p.rint_scale = 1.0
model.p.r_polarization_ohm = 0.0
model.aux_power_override_w = None
return model

```

```

def resolve_relative(base_dir: Path, raw: str) -> Path:
    path = Path(str(raw or "")).strip()
    if not path:
        return base_dir
    if path.is_absolute():
        return path
    return (base_dir / path).resolve()

```

5 主要構造の読み取り

主要関数は `neutralize_identification_scalars`, `resolve_relative`, `stage`, `load_manifest`, `relpath_from`, `tex_path_fragment`, `tex_text_fragment`, `load_yaml_if_exists`。CLI 引数宣言は 9 件。

5.1 主要クラス・関数

行	種別	名前	補足
123	function	<code>neutralize_identification_scalars</code>	args: model
135	function	<code>resolve_relative</code>	args: base_dir, raw
144	function	<code>stage</code>	args: message
148	function	<code>load_manifest</code>	args: package_dir, manifest_arg
168	function	<code>relpath_from</code>	args: base_dir, target
177	function	<code>tex_path_fragment</code>	args: value
199	function	<code>tex_text_fragment</code>	args: value
216	function	<code>load_yaml_if_exists</code>	args: path
223	function	<code>declared_control_stop_km</code>	args: profile_cfg, profile_path
245	function	<code>_terminal_anchor_from_payload</code>	args: payload
292	function	<code>_append_reason_column</code>	args: frame, mask, reason
300	function	<code>resolve_manifest_context</code>	args: package_dir, manifest
306	function	<code>opt_path</code>	args: raw_value
413	function	<code>hampel_mask</code>	args: series
425	function	<code>apply_sensor_quality_annotations</code>	args: work
520	function	<code>polarization_current_trace</code>	args: replay
544	function	<code>fit_battery_polarization</code>	args: replay
682	function	<code>apply_battery_polarization</code>	args: replay, dynamic_fit

706	function	resolve_fit_plan	args: options
750	function	resolve_identification_output_layout	args: package_dir, profile_cfg
777	function	identification_profile_output_path	args: canonical_profile, run_output_dir
789	function	load_ocv_df	args: profile_cfg, profile_yaml
797	function	build_source_map_assets	args: profile_cfg, profile_yaml
801	function	rel	args: key, fallback
821	function	apply_actual_event_annotations	args: work, payload
863	function	truncate_at_retire_event	args: work, payload
892	function	normalize_generic_log	args: log_csv
994	function	build_terminal_anchor	args: log_df, ocv_df, base_model, anchor_km
1029	function	terminal_metrics	args: replay_df, ocv_df, base_model, batt, anchor_km
1078	function	replay_day_metrics	args: replay_df
1099	function	identification_selection_score	args: metrics
1135	function	condition_vehicle_fit_on_measured_pv	args: frame
1155	function	calibrate_solar_measurement_to_pack	args: frame
1271	function	_shift_acceleration_within_segments	args: frame, sample_shift, acceleration
1283	function	_acceleration_trace_from_speed	args: frame
1321	function	_bilinear_interp_array	args: x_grid, y_grid, values, x, y
1341	function	_map_efficiency_array	args: base_model, maps, speed_ms, torque_nm
1364	function	_motion_predictions_for_acceleration	args: frame, acceleration, base_model, mot
1431	function	fit_acceleration_timestamp_alignment	args: frame, base_model, mot, options
1541	function	rmse	args: mask
1631	function	_grade_from_smoothed_elevation	args: distance_km, elevation_m, smoothing_window_km
1653	function	fit_grade_observation_alignment	args: frame, route_profile_csv, output_csv, base_model, mot, options
1750	function	rmse	args: predicted, mask
1886	function	pv_leave_one_day_out_validation	args: frame, model
1950	function	rmse	args: values
1964	function	add_end_to_end_metrics	args: primary, end_to_end
1973	function	add_battery_conditional_metrics	args: primary, conditional
1981	function	write_replay_csv	args: frame, output_path

1998	function	apply_fit_to_cfg	args: cfg
2113	function	update_profile	args: profile_yaml, cfg, map_assets, pv, batt, mot, observed_log_csv, battery_dynamic_fit, solar_measurement_calibration, route_profile_asset, package_dir
2157	function	update_profile_artifact_references	args: profile_yaml, package_dir
2180	function	write_terminal_consistency_from_anchor	args: output_path, terminal_anchor
2318	function	sync_canonical_fullsim_profile	args: package_dir
2353	function	write_generic_summary	args: package_dir, manifest_path, profile_yaml, map_assets, pv, batt, mot, metrics, terminal_anchor, stage_anchors, map_shape_fit, post_refine, day_metrics, battery_dynamic_fit, fit_plan, manifest_context, output_dir, replay_csv, battery_conditioned_replay_csv, end_to_end_replay_csv
2422	function	write_generic_report	args: package_dir, profile_yaml, manifest_path, summary_yaml, observed_log_csv, pv, batt, mot, metrics, post_refine, map_assets
3075	function	main	

5.2 ファイルを上から読んだときの定義順

行	内容
20	SCRIPT_DIR に Path(__file__).resolve().parent の結果を代入する。
21	ROOT に SCRIPT_DIR.parents[0] の結果を代入する。
22	(SCRIPT_DIR, ROOT) を順に走査し、各要素を candidate に入れて処理する。
67	FIT_QUALITY_PRESETS に {'quick': {'battery_restart_count': 2, 'battery_maxiter': 50, 'motion_restart_count': 3, 'motion_maxiter': 70, 'joint_restart_count': 2, 'joint_random_start_count': 8, 'joint_local_topk': 2, 'joint_maxiter': 24, 'fit_stride': 3, 'allow_map_shape_fit': False, 'post_refine_enabled': False}, 'standard': {'battery_restart_count': 4, 'battery_maxiter': 80, 'motion_restart_count': 5, 'motion_maxiter': 100, 'joint_restart_count': 4, 'joint_random_start_count': 6, 'joint_local_topk': 3, 'joint_maxiter': 40, 'fit_stride': 2, 'allow_map_shape_fit': True, 'post_refine_enabled': False}, 'full': {'battery_restart_count': 6, 'battery_maxiter': 140, 'motion_restart_count': 8, 'motion_maxiter': 180, 'joint_restart_count': 6, 'joint_random_start_count': 18, 'joint_local_topk': 6, 'joint_maxiter': 80, 'fit_stride': 2, 'allow_map_shape_fit': True, 'post_refine_enabled': False}, 'ultra': {'battery_restart_count': 8, 'battery_maxiter': 220, 'motion_restart_count': 10, 'motion_maxiter': 260, 'joint_restart_count': 8, 'joint_random_start_count': 28, 'joint_local_topk': 8, 'joint_maxiter': 120, 'fit_stride': 1, 'allow_map_shape_fit': True, 'post_refine_enabled': True}} を代入する。
123	関数 neutralize_identification_scalars を定義する。
135	関数 resolve_relative を定義する。
144	関数 stage を定義する。
148	関数 load_manifest を定義する。
168	関数 relpath_from を定義する。
177	関数 tex_path_fragment を定義する。

199	関数 <code>tex_text_fragment</code> を定義する。
216	関数 <code>load_yaml_if_exists</code> を定義する。
223	関数 <code>declared_control_stop_km</code> を定義する。
245	関数 <code>_terminal_anchor_from_payload</code> を定義する。
292	関数 <code>_append_reason_column</code> を定義する。
300	関数 <code>resolve_manifest_context</code> を定義する。
413	関数 <code>hampel_mask</code> を定義する。
425	関数 <code>apply_sensor_quality_annotations</code> を定義する。
520	関数 <code>polarization_current_trace</code> を定義する。
544	関数 <code>fit_battery_polarization</code> を定義する。
682	関数 <code>apply_battery_polarization</code> を定義する。
706	関数 <code>resolve_fit_plan</code> を定義する。
750	関数 <code>resolve_identification_output_layout</code> を定義する。
777	関数 <code>identification_profile_output_path</code> を定義する。
789	関数 <code>load_ocv_df</code> を定義する。
797	関数 <code>build_source_map_assets</code> を定義する。
821	関数 <code>apply_actual_event_annotations</code> を定義する。
863	関数 <code>truncate_at_retire_event</code> を定義する。
892	関数 <code>normalize_generic_log</code> を定義する。
994	関数 <code>build_terminal_anchor</code> を定義する。
1029	関数 <code>terminal_metrics</code> を定義する。
1078	関数 <code>replay_day_metrics</code> を定義する。
1099	関数 <code>identification_selection_score</code> を定義する。
1135	関数 <code>condition_vehicle_fit_on_measured_pv</code> を定義する。
1155	関数 <code>calibrate_solar_measurement_to_pack</code> を定義する。
1271	関数 <code>_shift_acceleration_within_segments</code> を定義する。
1283	関数 <code>_acceleration_trace_from_speed</code> を定義する。
1321	関数 <code>_bilinear_interp_array</code> を定義する。
1341	関数 <code>_map_efficiency_array</code> を定義する。
1364	関数 <code>_motion_predictions_for_acceleration</code> を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
2	<code>from__future_ _importannotations</code>	型ヒントの遅延評価を有効にし、自己参照型や <code>forward reference</code> を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
4	<code>importargparse</code>	CLI 引数を宣言し、実行時パラメータを外から受け取るため。このファイル内での主な使用位置は L3076。
5	<code>importjson</code>	<code>manifest</code> 、 <code>checkpoint</code> 、 <code>UDP payload</code> をやり取りするため。このファイル内での主な使用位置は L3181, L3841。
6	<code>importmath</code>	Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L539, L1595, L1596, L1834, L3619。

7	<code>importos</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L23, L172, L174, L2013, L2014, L2380, L2381, L2382, ...。
8	<code>importre</code>	<code>re</code> モジュールを利用するため。このファイル内での主な使用位置は L764。
9	<code>importsys</code>	<code>sys</code> モジュールを利用するため。このファイル内での主な使用位置は L24, L25。
10	<code>importtextwrap</code>	<code>textwrap</code> モジュールを利用するため。このファイル内での主な使用位置は L3070。
11	<code>frompathlibimportPath</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L20, L135, L136, L148, L151, L158, L168, L216, ...。
12	<code>fromtypingimportAny,Dict,Tuple</code>	<code>typing</code> から <code>Any</code> , <code>Dict</code> , <code>Tuple</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L67, L148, L245, L251, L331, L366, L544, L600, ...。
14	<code>importnumpyasnp</code>	数値配列、 <code>clip</code> 、補間、統計計算を行うため。このファイル内での主な使用位置は L296, L422, L458, L466, L492, L502, L512, L525, ...。
15	<code>importpandasaspd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L292, L413, L414, L426, L430, L464, L465, L466, ...。
16	<code>importyaml</code>	<code>profile</code> 、 <code>stop</code> 、 <code>schedule</code> 、 <code>summary</code> YAML を読み書きするため。このファイル内での主な使用位置は L164, L220, L2154, L2164, L2174, L2311, L2333, L2349, ...。
17	<code>fromscipy.optimizeimportleast_squares</code>	<code>scipy.optimize</code> から <code>least_squares</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L1206, L1213, L1232。
18	<code>fromscipy.signalimportsavgol_filter</code>	<code>scipy.signal</code> から <code>savgol_filter</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L1304, L1643。

27	frombuild_bwsc2025_fitted_ packageimportBATTERY_ PACK_MAX_CHARGE_V, BATTERY_NOMINAL_VOLTAGE_ V,INVALID_PACK_VOLTAGE_ MIN_V,ROOT,TIMEZONE_ LOCAL,BatteryFitResult, MotionFitResult, PostRefineResult, PvFitResult,attach_archive_ pv_model,build_grounded_ map_assets,build_stage_ anchors,build_model_from_ map_assets,build_model_ from_profile_cfg,compile_ tex,dcir_observations, ensure_dir,fit_battery_ parameters,fit_map_shapes, fit_motion_parameters,fit_ pv_parameters,fit_regen_ utilization,fit_stop_ tilt_fraction,infer_soc_ from_loaded_state,joint_ refine_parameters,joint_ replay,load_profile_yaml, metrics_from_replay,motion_ power_prediction,post_ refine_replay_scalars, replay_segment_start_mask, resample_for_fit,soc_fit_ upper_bound,write_current_ maps_and_coefficients, write_scaled_maps	build_bwsc2025_fitted_package から BAT- TERRY_PACK_MAX_CHARGE_V, BAT- TERRY_NOMINAL_VOLTAGE_V, IN- VALID_PACK_VOLTAGE_MIN_V, ROOT, TIMEZONE_LOCAL, BatteryFitResult, Mo- tionFitResult, PostRefineResult, PvFitResult, at- tach_archive_pv_model, build_grounded_map_assets, build_stage_anchors, build_model_from_map_assets, build_model_from_profile_cfg, compile_tex, dcir_observations, ensure_dir, fit_battery_parameters, fit_map_shapes, fit_motion_parameters, fit_pv_parameters, fit_regen_utilization, fit_stop_tilt_fraction, infer_soc_from_loaded_state, joint_refine_parameters, joint_replay, load_profile_yaml, metrics_from_replay, motion_power_prediction, post_refine_replay_scalars, replay_segment_start_mask, resample_for_fit, soc_fit_upper_bound, write_current_maps_and_coefficients, write_scaled_maps を読み込み、このファイルの 処理を組み立てるため。このファイル内での主な 使用位置は L21, L22, L157, L381, L444, L531, L567, L842,
64	fromaudit_identification_ residualsimportrun_ auditasrun_residual_audit	audit_identification_residuals から run_audit as run_residual_audit を読み込み、このファイルの 処理を組み立てるため。このファイル内での主な使 用位置は L3671。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や ‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 ‘(N,)’、候補集団なら ‘(population, N)’ となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

6.7 CLI、PowerShell、Bash、環境変数、終了コード

CLI は端末からプログラム名と引数を渡す操作界面である。‘argparse’は文字列として届く引数を名前、型、既定値、必須性に従って解析する。

PowerShell と Bash は別の shell であり、変数記法、改行継続、引用、パス表記が異なる。このプロジェクトでは Windows 側の SolarSim.ps1 が WSL 側の solar_control.sh へ処理を渡す。

環境変数は親プロセスから子プロセスへ受け渡される名前付き文字列である。ROS_DOMAIN_ID、RMW_IMPLEMENTATION、Python の数値スレッド数などはコード外から動作を変えるため、実行記録へ残す必要がある。

終了コード 0 は一般に成功、0 以外は失敗を示す。shell ルータは子プロセスの終了コードを握り潰さず上位へ返すことで、自動運用が失敗を検知できる。

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)

6.8 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明

数値最適化器は、利用者が与えた目的関数を複数の候補点で評価し、より小さい値を持つ候補を探す。solver が物理を理解するのではなく、物理と運用価値は cost 関数へ書かれる。

L-BFGS-B は変数ごとの上下限を扱える局所最適化法である。初期値の近くの谷へ収束し得るため、非凸問題では複数 seed や大域探索と組み合わせる。success が False でも有限な候補が返る場合があるため、採用条件をコード側で決める。

SHGO は定めた sampling と局所最適化を組み合わせる大域最適化法である。有限 Cartesian grid の全列挙は、その grid 上の最良を証明できるが、連続領域全体の最良を自動的に証明しない。資料ではこの証明範囲を区別する。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)
- SciPy 公式: [scipy.optimize.shgo](#)

6.9 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻 t と距離 s が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 timezone、naive datetime を混ぜると数時間のずれが発生する。入力で timezone を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

6.10 freshness、filter、guard、fallback、fail-safe

分散システムでは最後に受け取った値が現在も有効とは限らない。受信時刻と timeout から freshness を判定し、stale 値を計画状態へ無条件に同期しない。

filter は noise と一時的な飛び値を抑えるが、遅れを生む。slew limit は指令変化率を制限する。安全 guard は solver の cost 罰則とは別に、現在出力へ強制制約を適用する最後の防波堤である。

fallback は失敗時の代替動作を事前に決める設計である。前回計画保持、物理に基づく決定論的入力、停止、低速制限などから、故障 mode ごとに選ぶ。fallback 発生は status と log へ残し、正常解と区別する。

6.11 CSV/YAML の data contract と検証

data contract は列名、型、単位、timezone、欠損可否、並び順、重複、許容範囲、先頭行、encoding を事前に決めた仕様である。単に CSV として読めることは、モデル入力として正しいことを意味しない。

同定用実測、map、route、forecast、stop、schedule はそれぞれ grain が異なる。生成時に schema validation、物理範囲、時間単調性、route 範囲、coverage を検査し、検査結果を artifact として残す。

学習用データと独立検証データを分離し、RMSE だけでなく bias、時系列残差、energy 積算誤差、終端 SoC、温度・電圧制約、外挿領域を評価する。

6.12 単体試験、SILS、replay、観測可能性

単体試験は関数やモデル項の局所契約、SILS は複数 Node と時間進行を含む閉ループ、historical replay は実測入力への再現性、本番 preflight は実機通信と停止動作を確認する。目的が異なるため一つで代用しない。

再現可能な診断には、入力、出力、内部状態、solver 成否、候補数、計算時間、fallback、source revision、profile hash を同じ run ID で記録する。

非同期不具合は最終値だけでは追えない。topic 周期、message age、callback 所要時間、queue 遅延、Node 生存、publisher 数を同時に観測する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 公式: Executors](#)

7 関数・クラスを上から順に解説

7.1 L123 関数 `neutralize_identification_scalars`

- 行範囲: L123–L132
- 所属: module 直下
- 定義: `neutralize_identification_scalars(model)`
- docstring: Fit each calibration factor once on top of the declared physical maps.
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: `model`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `model.p.panel_gain` に 1.0 の結果を代入する。
2. `model.p.drive_eff_scale` に 1.0 の結果を代入する。
3. `model.p.regen_eff_scale` に 1.0 の結果を代入する。
4. `model.p.regen_utilization` に 1.0 の結果を代入する。
5. `model.p.rint_scale` に 1.0 の結果を代入する。
6. `model.p.r_polarization_ohm` に 0.0 の結果を代入する。
7. `model.aux_power_override_w` に `None` の結果を代入する。
8. `model` を返す。

代表コード断片

```
def neutralize_identification_scalars(model):
    """Fit each calibration factor once on top of the declared physical maps."""
    model.p.panel_gain = 1.0
    model.p.drive_eff_scale = 1.0
    model.p.regen_eff_scale = 1.0
    model.p.regen_utilization = 1.0
    model.p.rint_scale = 1.0
    model.p.r_polarization_ohm = 0.0
    model.aux_power_override_w = None
    return model
```

7.2 L135 関数 `resolve_relative`

- 行範囲: L135–L141
- 所属: module 直下
- 定義: `resolve_relative(base_dir:Path, raw:str)->Path`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `is_absolute`, `resolve`, `str`, `strip`
- 戻り値の要点: `(base_dir / path).resolve() / base_dir / path`
- 主なローカル代入名: `path`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `path` に `Path(str(raw or "").strip())` の結果を代入する。
2. 条件 `not path` を判定し、真なら内部処理を行う。
3. `base_dir` を返す。
4. 条件 `path.is_absolute()` を判定し、真なら内部処理を行う。
5. `path` を返す。
6. `(base_dir / path).resolve()` を返す。

代表コード断片

```
def resolve_relative(base_dir: Path, raw: str) -> Path:
    path = Path(str(raw or "").strip())
    if not path:
        return base_dir
    if path.is_absolute():
        return path
    return (base_dir / path).resolve()
```

7.3 L144 関数 stage

- 行範囲: L144–L145
- 所属: module 直下
- 定義: `stage(message:str)->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `print`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `print(...)` を実行する。

代表コード断片

```
def stage(message: str) -> None:
    print(f"[generic-id] {message}", flush=True)
```

7.4 L148 関数 load_manifest

- 行範囲: L148–L165
- 所属: module 直下
- 定義: `load_manifest(package_dir:Path,manifest_arg:str|None)->Tuple[Path,dict]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path, cwd, is_absolute, is_file, next, open, resolve, safe_load, str, strip`
- 戻り値の要点: `(manifest_path, payload)`
- 主なローカル代入名: `candidates, default_path, f, manifest_path, path, payload, raw_path`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `default_path` に `package_dir / 'data' / 'identification' / 'identification_manifest.yaml'` の結果を代入する。
2. 条件 `manifest_arg` を判定し、真なら内部処理を行う。
3. `raw_path` に `Path(str(manifest_arg).strip())` の結果を代入する。
4. 条件 `raw_path.is_absolute()` を判定し、真なら内部処理を行う。
5. `manifest_path` に `raw_path` の結果を代入する。
6. 上の条件が偽の場合:
7. `candidates` に `[(package_dir / raw_path).resolve(), (ROOT / raw_path).resolve(), (Path.cwd() / raw_path).resolve()]` の結果を代入する。
8. `manifest_path` に `next((path for path in candidates if path.is_file()), candidates[0])` の結果を代入する。
9. 上の条件が偽の場合:
10. `manifest_path` に `default_path` の結果を代入する。
11. `with` 文で `manifest_path.open('r', encoding='utf-8')` を管理しながら処理する。
12. `payload` に `yaml.safe_load(f)` or `{}` の結果を代入する。
13. `(manifest_path, payload)` を返す。

代表コード断片

```
def load_manifest(package_dir: Path, manifest_arg: str | None) -> Tuple[Path, dict]:
    default_path = package_dir / "data" / "identification" / "identification_manifest.yaml"
    if manifest_arg:
        raw_path = Path(str(manifest_arg).strip())
        if raw_path.is_absolute():
            manifest_path = raw_path
        else:
            candidates = [
                (package_dir / raw_path).resolve(),
                (ROOT / raw_path).resolve(),
                (Path.cwd() / raw_path).resolve(),
            ]
            manifest_path = next((path for path in candidates if path.is_file()), candidates[0])
    else:
        manifest_path = default_path
    with manifest_path.open("r", encoding="utf-8") as f:
        payload = yaml.safe_load(f) or {}
    return manifest_path, payload
```

7.5 L168 関数 relpath_from

- 行範囲: L168–L174
- 所属: module 直下
- 定義: `relpath_from(base_dir:Path,target:Path|None)->str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `fspath`, `relpath`, `replace`
- 戻り値の要点: `"/os.path.relpath(target, base_dir).replace('\', '/') / os.fspath(target)`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 1

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `target is None` を判定し、真なら内部処理を行う。
2. `"/` を返す。
3. 例外処理を伴う try ブロックを実行する。
4. `os.path.relpath(target, base_dir).replace('\', '/')` を返す。
5. Exception を捕捉した場合:
6. `os.fspath(target)` を返す。

代表コード断片

```
def relpath_from(base_dir: Path, target: Path | None) -> str:
    if target is None:
        return ""
    try:
        return os.path.relpath(target, base_dir).replace("\\", "/")
    except Exception:
        return os.fspath(target)
```

7.6 L177 関数 `tex_path_fragment`

- 行範囲: L177–L196
- 所属: module 直下
- 定義: `tex_path_fragment(value:object)->str`
- docstring: Render ASCII and Unicode paths without losing CJK glyphs or TeX syntax.
- このブロックが直接呼ぶ主な関数・メソッド: `get`, `isascii`, `join`, `replace`, `str`
- 戻り値の要点: `f'\texttt{{{escaped}}}' / f'\path{{{text}}}'`
- 主なローカル代入名: `char`, `escaped`, `replacements`, `text`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `text` に `str(value)` の結果を代入する。
2. 条件 `text.isascii()` を判定し、真なら内部処理を行う。
3. `f'\path{{{text}}}'` を返す。
4. `replacements` に `{'\\': '\textbackslash', '{': '\{', '}': '\}', '$': '\$', '&': '\&', '#': '\#', '%': '\%', '_': '\_', '^': '\textasciicircum', '~': '\textasciitilde'}` の結果を代入する。
5. `escaped` に `".join((replacements.get(char, char) for char in text))` の結果を代入する。
6. `escaped` に `escaped.replace('/', '\allowbreak')` の結果を代入する。
7. `f'\texttt{{{escaped}}}'` を返す。

代表コード断片

```
def tex_path_fragment(value: object) -> str:
    """Render ASCII and Unicode paths without losing CJK glyphs or TeX syntax."""
    text = str(value)
    if text.isascii():
        return rf"\path{{{text}}}"
    replacements = {
        "\\": r"\textbackslash",
        "{": r"\{",
        "}": r"\}",
        "$": r"\$",
        "&": r"\&",
        "#": r"\#",
        "%": r"\%",
        "_": r"\_",
```

```

    "^": r"\textasciicircum{}",
    "~": r"\textasciitilde{}",
}
escaped = "".join(replacements.get(char, char) for char in text)
escaped = escaped.replace("/", r"\allowbreak{}")
return rf"\texttt{{{escaped}}}"

```

7.7 L199 関数 `tex_text_fragment`

- 行範囲: L199–L213
- 所属: module 直下
- 定義: `tex_text_fragment(value:object) -> str`
- docstring: Escape arbitrary report prose without path-style whitespace handling.
- このブロックが直接呼ぶ主な関数・メソッド: `get`, `join`, `str`
- 戻り値の要点: `".join((replacements.get(char, char) for char in str(value)))`
- 主なローカル代入名: `char`, `replacements`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `replacements` に `{'\'': '\textbackslash{}', '{': '\{', '}': '\}', '$': '\$', '&': '\&', '#': '\#', '%': '\%', '_': '\_', '^': '\textasciicircum{}', '~': '\textasciitilde{}'}` の結果を代入する。
2. `".join((replacements.get(char, char) for char in str(value)))` を返す。

代表コード断片

```

def tex_text_fragment(value: object) -> str:
    """Escape arbitrary report prose without path-style whitespace handling."""
    replacements = {
        "\\": r"\textbackslash{}",
        "{": r"\{",
        "}": r"\}",
        "$": r"\$",
        "&": r"\&",
        "#": r"\#",
        "%": r"\%",
        "_": r"\_",
        "^": r"\textasciicircum{}",
        "~": r"\textasciitilde{}",
    }
    return "".join(replacements.get(char, char) for char in str(value))

```

7.8 L216 関数 load_yaml_if_exists

- 行範囲: L216–L220
- 所属: module 直下
- 定義: `load_yaml_if_exists(path:Path|None)->dict`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `exists`, `open`, `safe_load`
- 戻り値の要点: `{}` / `yaml.safe_load(f)` or `{}`
- 主なローカル代入名: `f`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- with 文は context manager に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. 条件 `path is None or not path.exists()` を判定し、真なら内部処理を行う。
2. `{}` を返す。
3. with 文で `path.open('r', encoding='utf-8')` を管理しながら処理する。
4. `yaml.safe_load(f)` or `{}` を返す。

代表コード断片

```
def load_yaml_if_exists(path: Path | None) -> dict:
    if path is None or not path.exists():
        return {}
    with path.open("r", encoding="utf-8") as f:
        return yaml.safe_load(f) or {}
```

7.9 L223 関数 declared_control_stop_km

- 行範囲: L223–L242
- 所属: module 直下
- 定義: `declared_control_stop_km(profile_cfg:dict,profile_path:Path)->list[float]`
- docstring: Load control-stop distances declared by the vehicle package.
- このブロックが直接呼ぶ主な関数・メソッド: `append`, `bool`, `float`, `get`, `isinstance`, `load_yaml_if_exists`, `resolve_relative`, `set`, `sorted`, `str`, `strip`
- 戻り値の要点: `[]` / `sorted(set(distances))`

- 主なローカル代入名: `distances`, `item`, `key`, `paths`, `payload`, `raw`, `stops`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 2、`try` 1

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `paths` に `profile_cfg.get('paths', {})` if `isinstance(profile_cfg, dict)` else `{}` の結果を代入する。
2. `('actual_stop_yaml', 'stop_yaml')` を順に走査し、各要素を `key` に入れて処理する。
3. `raw` に `str(paths.get(key, '') or '').strip()` の結果を代入する。
4. 条件 `not raw` を判定し、真なら内部処理を行う。
5. `Continue` 文を実行する。
6. `payload` に `load_yaml_if_exists(resolve_relative(profile_path.parent, raw))` の結果を代入する。
7. `stops` に `payload.get('stops', [])` if `isinstance(payload, dict)` else `[]` の結果を代入する。
8. `distances` に `[]` の結果を代入する。
9. `stops` を順に走査し、各要素を `item` に入れて処理する。
10. 条件 `not isinstance(item, dict) or not bool(item.get('is_control_stop', False))` を判定し、真なら内部処理を行う。
11. `Continue` 文を実行する。
12. 例外処理を伴う `try` ブロックを実行する。
13. `distances.append(...)` を実行する。
14. `(KeyError, TypeError, ValueError)` を捕捉した場合:
15. `Continue` 文を実行する。
16. 条件 `distances` を判定し、真なら内部処理を行う。
17. `sorted(set(distances))` を返す。
を返す。

代表コード断片

```
def declared_control_stop_km(profile_cfg: dict, profile_path: Path) -> list[float]:
    """Load control-stop distances declared by the vehicle package."""
    paths = profile_cfg.get("paths", {}) if isinstance(profile_cfg, dict) else {}
    for key in ("actual_stop_yaml", "stop_yaml"):
        raw = str(paths.get(key, "") or "").strip()
        if not raw:
            continue
        payload = load_yaml_if_exists(resolve_relative(profile_path.parent, raw))
        stops = payload.get("stops", []) if isinstance(payload, dict) else []
        distances = []
        for item in stops:
            if not isinstance(item, dict) or not bool(item.get("is_control_stop", False)):
                continue
            try:
                distances.append(float(item["s_km"]))
            except (KeyError, TypeError, ValueError):
                continue
        if distances:
            return sorted(set(distances))
    return []
```

7.10 L245 関数 `_terminal_anchor_from_payload`

- 行範囲: L245–L289
- 所属: module 直下
- 定義: `_terminal_anchor_from_payload(payload:dict)->Dict[str,Any]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `bool`, `float`, `get`, `isinstance`, `str`, `strip`
- 戻り値の要点: `out / {} / {}`
- 主なローカル代入名: `anchor`, `key`, `out`, `raw`, `raw_time`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 6、ループ 3、`try` 1

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `not isinstance(payload, dict)` を判定し、真なら内部処理を行う。
2. `{}` を返す。
3. `anchor` に `payload.get('terminal_anchor', payload)` の結果を代入する。
4. 条件 `not isinstance(anchor, dict)` を判定し、真なら内部処理を行う。

5. {} を返す。
6. out に {} を代入する。
7. ('count', 's_km', 'voltage_v', 'current_a', 'temp_c', 'soc_target', 'soc_sigma', 'soc_evidence_min', 'soc_evidence_max', 'voltage_sigma_v', 'ocv_terminal_v', 'series_resistance_ohm', 'ocv_statistical_sigma_v', 'ocv_systematic_sigma_v', 'ocv_total_sigma_v') を順に走査し、各要素を key に入れて処理する。
8. raw に anchor.get(key, None) の結果を代入する。
9. 条件 raw in (None, "") を判定し、真なら内部処理を行う。
10. Continue 文を実行する。
11. 例外処理を伴う try ブロックを実行する。
12. out[key] に float(raw) の結果を代入する。
13. Exception を捕捉した場合:
14. Pass 文を実行する。
15. raw_time に str(anchor.get('time_utc', "") or "").strip() の結果を代入する。
16. 条件 raw_time を判定し、真なら内部処理を行う。
17. out['time_utc'] に raw_time の結果を代入する。
18. ('notes', 'source_documents', 'method', 'soc_target_basis') を順に走査し、各要素を key に入れて処理する。
19. 条件 key in anchor を判定し、真なら内部処理を行う。
20. out[key] に anchor[key] の結果を代入する。
21. ('quality_gate_pass', 'conditional_on_grounded_ocv_map', 'weak_channel_cross_consistency_gate_pass') を順に走査し、各要素を key に入れて処理する。
22. 条件 key in anchor を判定し、真なら内部処理を行う。
23. out[key] に bool(anchor[key]) の結果を代入する。
24. out を返す。

代表コード断片

```
def _terminal_anchor_from_payload(payload: dict) -> Dict[str, Any]:
    if not isinstance(payload, dict):
        return {}
    anchor = payload.get("terminal_anchor", payload)
    if not isinstance(anchor, dict):
        return {}
    out: Dict[str, Any] = {}
    for key in (
        "count",
        "s_km",
        "voltage_v",
        "current_a",
        "temp_c",
        "soc_target",
        "soc_sigma",
        "soc_evidence_min",
        "soc_evidence_max",
        "voltage_sigma_v",
        "ocv_terminal_v",
```



```

        "series_resistance_ohm",
        "ocv_statistical_sigma_v",
        "ocv_systematic_sigma_v",
        "ocv_total_sigma_v",
    ):
        raw = anchor.get(key, None)
        if raw in (None, ""):
            continue
        try:
            out[key] = float(raw)
        except Exception:
            pass
        raw_time = str(anchor.get("time_utc", "") or "").strip()
        if raw_time:
            out["time_utc"] = raw_time
        for key in ("notes", "source_documents", "method", "soc_target_basis"):
...

```

7.11 L292 関数 `_append_reason_column`

- 行範囲: L292–L297
- 所属: module 直下
- 定義: `_append_reason_column(frame:pd.DataFrame,mask:pd.Series,reason:str)->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `any`, `astype`, `fillna`, `len`, `where`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `current`, `merged`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `not mask.any()` を判定し、真なら内部処理を行う。
2. を返す。
3. `current` に `frame.loc[mask, 'exclude_reason'].fillna("").astype(str)` の結果を代入する。
4. `merged` に `np.where(current.str.len() > 0, current + ';' + reason, reason)` の結果を代入する。
5. `frame.loc[mask, 'exclude_reason']` に `merged` の結果を代入する。

代表コード断片

```
def _append_reason_column(frame: pd.DataFrame, mask: pd.Series, reason: str) -> None:
    if not mask.any():
        return
    current = frame.loc[mask, "exclude_reason"].fillna("").astype(str)
    merged = np.where(current.str.len() > 0, current + ";" + reason, reason)
    frame.loc[mask, "exclude_reason"] = merged
```

7.12 L300 関数 `resolve_manifest_context`

- 行範囲: L300–L410
- 所属: module 直下
- 定義: `resolve_manifest_context(package_dir:Path,manifest:dict)->dict`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `FileNotFoundError`, `Path`, `ValueError`, `_terminal_anchor_from_payload`, `append`, `difference`, `exists`, `get`, `is_absolute`, `isinstance`, `items`, `join`
- 戻り値の要点: `{'inputs': inputs, 'options': options, 'grounded_sources': grounded, 'evidence': evidence, 'actual_event_path': actual_event_path, 'counterfactual_event_path': counterfactual_event_path, 'terminal_anchor_path': terminal_anchor_path, 'grounded_summary_path': grounded_summary_path, 'source_inventory_path': source_inventory_path, 'notes_markdown_path': notes_markdown_path, 'explicit_grounded_assets': explicit_grounded_assets, 'grounded_summary_payload': grounded_summary_payload, 'actual_event_payload': actual_event_payload, 'terminal_anchor_override': terminal_anchor_override, 'external_documents': external_documents, 'declared_evidence': declared_evidence} / resolve_relative(package_dir, raw) / None`
- 主なローカル代入名: `actual_event_path`, `actual_event_payload`, `candidates`, `counterfactual_event_path`, `declared_evidence`, `evidence`, `explicit_grounded_assets`, `external_documents`, `grounded`, `grounded_summary_path`, `grounded_summary_payload`, `group_key`, `inputs`, `manifest_key`, `missing`, `missing_evidence`, `model_key`, `notes_markdown_path`, `options`, `path`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `FileNotFoundError('identification evidence manifest contains missing artifacts:\n- ' + '\n- '.join(missing_evidence))`, `ValueError('grounded_sources declares explicit maps but is missing required entries: ' + ', '.join(missing))`
- 制御構造の規模: 条件分岐 15、ループ 3、try 0

この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `inputs` に `manifest.get('inputs', {})` if `isinstance(manifest, dict)` else `{}` の結果を代入する。
2. `options` に `manifest.get('options', {})` if `isinstance(manifest, dict)` else `{}` の結果を代入する。
3. `grounded` に `manifest.get('grounded_sources', {})` if `isinstance(manifest, dict)` else `{}` の結果を代入する。

4. `evidence` に `manifest.get('evidence', {})` if `isinstance(manifest, dict)` else `{}` の結果を代入する。
5. 関数 `opt_path` を定義する。
6. `actual_event_path` に `opt_path(inputs.get('actual_event_yaml'))` の結果を代入する。
7. `counterfactual_event_path` に `opt_path(inputs.get('counterfactual_event_yaml'))` の結果を代入する。
8. `terminal_anchor_path` に `opt_path(inputs.get('terminal_anchor_yaml'))` の結果を代入する。
9. `grounded_summary_path` に `opt_path(grounded.get('grounded_map_summary_yaml', ''))` or `options.get('grounded_map_summary_yaml', '')` の結果を代入する。
10. `source_inventory_path` に `opt_path(evidence.get('source_inventory_json'))` の結果を代入する。
11. `notes_markdown_path` に `opt_path(evidence.get('notes_markdown'))` の結果を代入する。
12. `path_key_map` に `{'drive_eff_map': 'drive_eff_map_csv', 'drive_map_eco': 'drive_map_eco_csv', 'drive_map_power': 'drive_map_power_csv', 'regen_eff_map': 'regen_eff_map_csv', 'regen_map_eco': 'regen_map_eco_csv', 'regen_map_power': 'regen_map_power_csv', 'rint_map': 'rint_map_csv', 'panel_eff_map': 'panel_eff_map_csv', 'mppt_eff_map': 'mppt_eff_map_csv', 'ocv_soc_map': 'ocv_soc_map_csv'}` の結果を代入する。
13. `explicit_grounded_assets` に `{}` を代入する。
14. `path_key_map.items()` を順に走査し、各要素を `(model_key, manifest_key)` に入れて処理する。
15. `raw` に `str(grounded.get(manifest_key, ''))` or `''` の結果を代入する。
16. 条件 `raw` を判定し、真なら内部処理を行う。
17. `explicit_grounded_assets[model_key]` に `resolve_relative(package_dir, raw)` の結果を代入する。
18. 条件 `explicit_grounded_assets` を判定し、真なら内部処理を行う。
19. 条件 `'drive_map_eco' not in explicit_grounded_assets and 'drive_eff_map' in explicit_grounded_assets` を判定し、真なら内部処理を行う。
20. `explicit_grounded_assets['drive_map_eco']` に `explicit_grounded_assets['drive_eff_map']` の結果を代入する。
21. 条件 `'drive_map_power' not in explicit_grounded_assets and 'drive_eff_map' in explicit_grounded_assets` を判定し、真なら内部処理を行う。
22. `explicit_grounded_assets['drive_map_power']` に `explicit_grounded_assets['drive_eff_map']` の結果を代入する。
23. 条件 `'regen_map_eco' not in explicit_grounded_assets and 'regen_eff_map' in explicit_grounded_assets` を判定し、真なら内部処理を行う。
24. `explicit_grounded_assets['regen_map_eco']` に `explicit_grounded_assets['regen_eff_map']` の結果を代入する。
25. 条件 `'regen_map_power' not in explicit_grounded_assets and 'regen_eff_map' in explicit_grounded_assets` を判定し、真なら内部処理を行う。
26. `explicit_grounded_assets['regen_map_power']` に `explicit_grounded_assets['regen_eff_map']` の結果を代入する。
27. `required` に `{'drive_eff_map', 'regen_eff_map', 'rint_map', 'panel_eff_map', 'mppt_eff_map', 'ocv_soc_map'}` の結果を代入する。
28. `missing` に `sorted(required.difference(explicit_grounded_assets))` の結果を代入する。

29. 条件 `missing` を判定し、真なら内部処理を行う。
30. `ValueError('grounded_sources declares explicit maps but is missing required entries: ' + ', '.join(missing))` を送出する。
31. `grounded_summary_payload` に `load_yaml_if_exists(grounded_summary_path)` の結果を代入する。
32. 条件 `grounded_summary_path is not None` を判定し、真なら内部処理を行う。
33. `grounded_summary_payload['summary_yaml']` に `relpath_from(package_dir, grounded_summary_path)` の結果を代入する。
34. `actual_event_payload` に `load_yaml_if_exists(actual_event_path)` の結果を代入する。
35. `terminal_anchor_payload` に `load_yaml_if_exists(terminal_anchor_path)` の結果を代入する。
36. `terminal_anchor_override` に `_terminal_anchor_from_payload(terminal_anchor_payload)` の結果を代入する。
37. `external_documents` に `evidence.get('external_documents', [])` の結果を代入する。
38. 条件 `not isinstance(external_documents, list)` を判定し、真なら内部処理を行う。
39. `external_documents` に `[external_documents]` の結果を代入する。
40. `declared_evidence` に `{}` を代入する。
41. `missing_evidence` に `[]` を代入する。
42. `('field_tests', 'normalized_field_outputs', 'external_documents')` を順に走査し、各要素を `group_key` に入れて処理する。
43. `raw_values` に `evidence.get(group_key, [])` の結果を代入する。
44. 条件 `not isinstance(raw_values, list)` を判定し、真なら内部処理を行う。
45. `raw_values` に `[raw_values]` の結果を代入する。
46. `resolved_group` に `[]` を代入する。
47. `raw_values` を順に走査し、各要素を `raw_value` に入れて処理する。
48. `raw` に `str(raw_value or "").strip()` の結果を代入する。
49. 条件 `not raw` を判定し、真なら内部処理を行う。
50. `Continue` 文を実行する。
51. `raw_path` に `Path(raw)` の結果を代入する。
52. 条件 `raw_path.is_absolute()` を判定し、真なら内部処理を行う。
53. `resolved` に `raw_path` の結果を代入する。
54. 上の条件が偽の場合:
55. `candidates` に `[(package_dir / raw_path).resolve(), (ROOT / raw_path).resolve()]` の結果を代入する。
56. `resolved` に `next((path for path in candidates if path.exists()), candidates[0])` の結果を代入する。
57. `resolved_group.append(...)` を実行する。
58. 条件 `not resolved.exists()` を判定し、真なら内部処理を行う。
59. `missing_evidence.append(...)` を実行する。
60. `declared_evidence[group_key]` に `resolved_group` の結果を代入する。

61. 条件 `missing_evidence` を判定し、真なら内部処理を行う。
62. `FileNotFoundError('identification evidence manifest contains missing artifacts:\n- ' + '\n- '.join(missing_evidence))` を送出する。
63. `{'inputs': inputs, 'options': options, 'grounded_sources': grounded, 'evidence': evidence, 'actual_event_path': actual_event_path, 'counterfactual_event_path': counterfactual_event_path, 'terminal_anchor_path': terminal_anchor_path, 'grounded_summary_path': grounded_summary_path, 'source_inventory_path': source_inventory_path, 'notes_markdown_path': notes_markdown_path, 'explicit_grounded_assets': explicit_grounded_assets, 'grounded_summary_payload': grounded_summary_payload, 'actual_event_payload': actual_event_payload, 'terminal_anchor_override': terminal_anchor_override, 'external_documents': external_documents, 'declared_evidence': declared_evidence}` を返す。

代表コード断片

```
def resolve_manifest_context(package_dir: Path, manifest: dict) -> dict:
    inputs = manifest.get("inputs", {}) if isinstance(manifest, dict) else {}
    options = manifest.get("options", {}) if isinstance(manifest, dict) else {}
    grounded = manifest.get("grounded_sources", {}) if isinstance(manifest, dict) else {}
    evidence = manifest.get("evidence", {}) if isinstance(manifest, dict) else {}

    def opt_path(raw_value) -> Path | None:
        raw = str(raw_value or "").strip()
        if not raw:
            return None
        return resolve_relative(package_dir, raw)

    actual_event_path = opt_path(inputs.get("actual_event_yaml"))
    counterfactual_event_path = opt_path(inputs.get("counterfactual_event_yaml"))
    terminal_anchor_path = opt_path(inputs.get("terminal_anchor_yaml"))
    grounded_summary_path = opt_path(grounded.get("grounded_map_summary_yaml", "") or
options.get("grounded_map_summary_yaml", ""))
    source_inventory_path = opt_path(evidence.get("source_inventory_json"))
    notes_markdown_path = opt_path(evidence.get("notes_markdown"))

    path_key_map = {
        "drive_eff_map": "drive_eff_map_csv",
        "drive_map_eco": "drive_map_eco_csv",
        "drive_map_power": "drive_map_power_csv",
        "regen_eff_map": "regen_eff_map_csv",
        "regen_map_eco": "regen_map_eco_csv",
        "regen_map_power": "regen_map_power_csv",
        "rint_map": "rint_map_csv",
        "panel_eff_map": "panel_eff_map_csv",
        "mppt_eff_map": "mppt_eff_map_csv",
        "ocv_soc_map": "ocv_soc_map_csv",
    }
    explicit_grounded_assets: Dict[str, Path] = {}
    for model_key, manifest_key in path_key_map.items():
        raw = str(grounded.get(manifest_key, "") or "").strip()
        if raw:
            ...
```

7.13 L306 関数 `resolve_manifest_context.opt_path`

- 行範囲: L306–L310
- 所属: `resolve_manifest_context`
- 定義: `opt_path(raw_value) -> Path | None`
- docstring: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: `resolve_relative`, `str`, `strip`
- 戻り値の要点: `resolve_relative(package_dir, raw) / None`
- 主なローカル代入名: `raw`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `raw` に `str(raw_value or "").strip()` の結果を代入する。
2. 条件 `not raw` を判定し、真なら内部処理を行う。
3. `None` を返す。
4. `resolve_relative(package_dir, raw)` を返す。

代表コード断片

```
def opt_path(raw_value) -> Path | None:
    raw = str(raw_value or "").strip()
    if not raw:
        return None
    return resolve_relative(package_dir, raw)
```

7.14 L413 関数 `hampel_mask`

- 行範囲: L413–L422
- 所属: `module` 直下
- 定義: `hampel_mask(series:pd.Series,*,window:int,n_sigma:float,min_abs:float)->pd.Series`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `astype`, `clip`, `float`, `int`, `max`, `maximum`, `median`, `rolling`, `to_numeric`
- 戻り値の要点: `abs_dev > np.maximum(float(min_abs), float(n_sigma) * scale)`
- 主なローカル代入名: `abs_dev`, `mad`, `med`, `scale`, `window`, `x`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. x に `pd.to_numeric(series, errors='coerce').astype(float)` の結果を代入する。
2. window に `max(3, int(window))` の結果を代入する。
3. 条件 `window % 2 == 0` を判定し、真なら内部処理を行う。
4. window を Add で更新する。
5. med に `x.rolling(window, center=True, min_periods=1).median()` の結果を代入する。
6. abs_dev に `(x - med).abs()` の結果を代入する。
7. mad に `abs_dev.rolling(window, center=True, min_periods=1).median()` の結果を代入する。
8. scale に `(1.4826 * mad).clip(lower=max(1e-06, float(min_abs) * 0.25))` の結果を代入する。
9. `abs_dev > np.maximum(float(min_abs), float(n_sigma) * scale)` を返す。

代表コード断片

```
def hampel_mask(series: pd.Series, *, window: int, n_sigma: float, min_abs: float) -> pd.Series:
    x = pd.to_numeric(series, errors="coerce").astype(float)
    window = max(3, int(window))
    if window % 2 == 0:
        window += 1
    med = x.rolling(window, center=True, min_periods=1).median()
    abs_dev = (x - med).abs()
    mad = abs_dev.rolling(window, center=True, min_periods=1).median()
    scale = (1.4826 * mad).clip(lower=max(1.0e-6, float(min_abs) * 0.25))
    return abs_dev > np.maximum(float(min_abs), float(n_sigma) * scale)
```

7.15 L425 関数 `apply_sensor_quality_annotations`

- 行範囲: L425–L517
- 所属: module 直下
- 定義: `apply_sensor_quality_annotations(work:pd.DataFrame,*,base_model,options:dict|None=None)->pd.DataFrame`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_append_reason_column`, `abs`, `any`, `astype`, `copy`, `diff`, `fillna`, `float`, `get`, `getattr`, `hampel_mask`, `int`
- 戻り値の要点: `out / work`
- 主なローカル代入名: `charge_floor_a`, `current`, `current_limit_margin_a`, `current_spike`, `current_spike_min_abs_a`, `current_spike_sigma`, `current_spike_window`, `dt_sec`, `impossible_charge`, `impossible_voltage_slew`, `invalid_v`, `invalid_v_threshold`, `key`, `low_current_pair`, `options`, `out`, `power_spike`, `power_spike_min_abs_w`, `power_spike_sigma`, `power_spike_window`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 9、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. options に options if isinstance(options, dict) else {} の結果を代入する。
2. sensor_cfg に options.get('sensor_filter', {}) if isinstance(options.get('sensor_filter', {}), dict) else {} の結果を代入する。
3. 条件 sensor_cfg.get('enabled', True) is False を判定し、真なら内部処理を行う。
4. work を返す。
5. out に work.copy() の結果を代入する。
6. ('exclude_power_fit', 'exclude_voltage_fit', 'exclude_weather_fit') を順に走査し、各要素を key に入れて処理する。
7. 条件 key not in out.columns を判定し、真なら内部処理を行う。
8. out[key] に False の結果を代入する。
9. out[key] に out[key].fillna(False).astype(bool) の結果を代入する。
10. 条件 'exclude_reason' not in out.columns を判定し、真なら内部処理を行う。
11. out['exclude_reason'] に” の結果を代入する。
12. invalid_v_threshold に float(sensor_cfg.get('invalid_voltage_threshold_v', INVALID_PACK_VOLTAGE_MIN_V)) の結果を代入する。
13. current_limit_margin_a に float(sensor_cfg.get('charge_current_limit_margin_a', 2.0)) の結果を代入する。
14. current_spike_window に int(sensor_cfg.get('current_spike_window', 9)) の結果を代入する。
15. current_spike_sigma に float(sensor_cfg.get('current_spike_sigma', 4.0)) の結果を代入する。
16. current_spike_min_abs_a に float(sensor_cfg.get('current_spike_min_abs_a', 4.0)) の結果を代入する。
17. power_spike_window に int(sensor_cfg.get('power_spike_window', 9)) の結果を代入する。
18. power_spike_sigma に float(sensor_cfg.get('power_spike_sigma', 4.0)) の結果を代入する。
19. power_spike_min_abs_w に float(sensor_cfg.get('power_spike_min_abs_w', 400.0)) の結果を代入する。
20. voltage_slew_max_vps に float(sensor_cfg.get('voltage_slew_max_vps', 0.2)) の結果を代入する。
21. voltage_slew_low_current_a に float(sensor_cfg.get('voltage_slew_low_current_a', 2.5)) の結果を代入する。
22. voltage_spike_window に int(sensor_cfg.get('voltage_spike_window', 9)) の結果を代入する。
23. voltage_spike_sigma に float(sensor_cfg.get('voltage_spike_sigma', 4.0)) の結果を代入する。
24. voltage_spike_min_abs_v に float(sensor_cfg.get('voltage_spike_min_abs_v', 2.0)) の結果を代入する。
25. invalid_v に np.isfinite(out['battery_voltage_v']) & (out['battery_voltage_v'] < invalid_v_threshold) の結果を代入する。
26. 条件 invalid_v.any() を判定し、真なら内部処理を行う。
27. out.loc[invalid_v, 'exclude_voltage_fit'] に True の結果を代入する。

28. `out.loc[invalid_v, 'exclude_power_fit']` に `True` の結果を代入する。
29. `_append_reason_column(...)` を実行する。
30. `voltage` に `pd.to_numeric(out['battery_voltage_v'], errors='coerce')` の結果を代入する。
31. `current` に `pd.to_numeric(out['battery_current_a'], errors='coerce')` の結果を代入する。
32. `dt_sec` に `pd.to_numeric(out['dt_sec'], errors='coerce').replace(0.0, np.nan)` の結果を代入する。
33. `voltage_slew_vps` に `voltage.diff().abs() / dt_sec` の結果を代入する。
34. `low_current_pair` に `(current.abs() <= voltage_slew_low_current_a) & (current.shift(1).abs() <= voltage_slew_low_current_a)` の結果を代入する。
35. `impossible_voltage_slew` に `(voltage_slew_vps > voltage_slew_max_vps) & low_current_pair & ~invalid_v & ~invalid_v.shift(1, fill_value=True)` の結果を代入する。
36. 条件 `impossible_voltage_slew.any()` を判定し、真なら内部処理を行う。
37. `out.loc[impossible_voltage_slew, 'exclude_voltage_fit']` に `True` の結果を代入する。
38. `out.loc[impossible_voltage_slew, 'exclude_power_fit']` に `True` の結果を代入する。
39. `_append_reason_column(...)` を実行する。
40. `voltage_spike` に `hampel_mask(voltage, window=voltage_spike_window, n_sigma=voltage_spike_sigma, min_abs=voltage_spike_min_abs_v) & (current.abs() <= voltage_slew_low_current_a) & ~invalid_v` の結果を代入する。
41. 条件 `voltage_spike.any()` を判定し、真なら内部処理を行う。
42. `out.loc[voltage_spike, 'exclude_voltage_fit']` に `True` の結果を代入する。
43. `out.loc[voltage_spike, 'exclude_power_fit']` に `True` の結果を代入する。
44. `_append_reason_column(...)` を実行する。
45. `charge_floor_a` に `float(getattr(base_model.p, 'I_chg_min', -16.5)) - current_limit_margin_a` の結果を代入する。
46. `impossible_charge` に `np.isfinite(out['battery_current_a']) & (out['battery_current_a'] < charge_floor_a)` の結果を代入する。
47. 条件 `impossible_charge.any()` を判定し、真なら内部処理を行う。
48. `out.loc[impossible_charge, 'exclude_power_fit']` に `True` の結果を代入する。
49. `_append_reason_column(...)` を実行する。
50. `current_spike` に `hampel_mask(out['battery_current_a'], window=current_spike_window, n_sigma=current_spike_sigma, min_abs=current_spike_min_abs_a) & np.isfinite(out['battery_current_a'])` の結果を代入する。
51. 条件 `current_spike.any()` を判定し、真なら内部処理を行う。
52. `out.loc[current_spike, 'exclude_power_fit']` に `True` の結果を代入する。
53. `_append_reason_column(...)` を実行する。
54. `power_spike` に `hampel_mask(out['battery_power_w_obs'], window=power_spike_window, n_sigma=power_spike_sigma, min_abs=power_spike_min_abs_w) & np.isfinite(out['battery_power_w_obs'])` の結果を代入する。
55. 条件 `power_spike.any()` を判定し、真なら内部処理を行う。
56. `out.loc[power_spike, 'exclude_power_fit']` に `True` の結果を代入する。
57. `_append_reason_column(...)` を実行する。
58. `out` を返す。

代表コード断片

```
def apply_sensor_quality_annotations(
    work: pd.DataFrame,
    *,
    base_model,
    options: dict | None = None,
) -> pd.DataFrame:
    options = options if isinstance(options, dict) else {}
    sensor_cfg = options.get("sensor_filter", {}) if isinstance(options.get("sensor_filter", {}), dict) else {}
    if sensor_cfg.get("enabled", True) is False:
        return work

    out = work.copy()
    for key in ("exclude_power_fit", "exclude_voltage_fit", "exclude_weather_fit"):
        if key not in out.columns:
            out[key] = False
        out[key] = out[key].fillna(False).astype(bool)
    if "exclude_reason" not in out.columns:
        out["exclude_reason"] = ""

    invalid_v_threshold = float(sensor_cfg.get("invalid_voltage_threshold_v",
INVALID_PACK_VOLTAGE_MIN_V))
    current_limit_margin_a = float(sensor_cfg.get("charge_current_limit_margin_a", 2.0))
    current_spike_window = int(sensor_cfg.get("current_spike_window", 9))
    current_spike_sigma = float(sensor_cfg.get("current_spike_sigma", 4.0))
    current_spike_min_abs_a = float(sensor_cfg.get("current_spike_min_abs_a", 4.0))
    power_spike_window = int(sensor_cfg.get("power_spike_window", 9))
    power_spike_sigma = float(sensor_cfg.get("power_spike_sigma", 4.0))
    power_spike_min_abs_w = float(sensor_cfg.get("power_spike_min_abs_w", 400.0))
    voltage_slew_max_vps = float(sensor_cfg.get("voltage_slew_max_vps", 0.20))
    voltage_slew_low_current_a = float(sensor_cfg.get("voltage_slew_low_current_a", 2.5))
    voltage_spike_window = int(sensor_cfg.get("voltage_spike_window", 9))
    voltage_spike_sigma = float(sensor_cfg.get("voltage_spike_sigma", 4.0))
    voltage_spike_min_abs_v = float(sensor_cfg.get("voltage_spike_min_abs_v", 2.0))

    invalid_v = np.isfinite(out["battery_voltage_v"]) & (out["battery_voltage_v"] <
invalid_v_threshold)
    if invalid_v.any():
...

```

7.16 L520 関数 polarization_current_trace

- 行範囲: L520–L541
- 所属: module 直下
- 定義: `polarization_current_trace(replay:pd.DataFrame,*,current_column:str,tau_sec:float)->np.ndarray`
- docstring: Return the 1-RC branch current state immediately before each sample.
- このブロックが直接呼ぶ主な関数・メソッド: `diff`, `exp`, `float`, `isfinite`, `len`, `max`, `min`, `range`, `replay_segment_start_mask`, `to_datetime`, `to_numeric`, `to_numpy`
- 戻り値の要点: `state`
- 主なローカル代入名: `alpha`, `current`, `dt`, `dt_sec`, `idx`, `segment_starts`, `state`, `tau`, `time_utc`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。

- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 1、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. current に `pd.to_numeric(replay[current_column], errors='coerce').to_numpy(dtype=float)` の結果を代入する。
2. current に `np.where(np.isfinite(current), current, 0.0)` の結果を代入する。
3. time_utc に `pd.to_datetime(replay['time_utc'], utc=True, errors='coerce')` の結果を代入する。
4. dt_sec に `time_utc.diff().dt.total_seconds().to_numpy(dtype=float)` の結果を代入する。
5. segment_starts に `replay_segment_start_mask(replay)` の結果を代入する。
6. state に `np.zeros(len(replay), dtype=float)` の結果を代入する。
7. tau に `max(float(tau_sec), 1e-06)` の結果を代入する。
8. `range(1, len(replay))` を順に走査し、各要素を idx に入れて処理する。
9. 条件 `segment_starts[idx] or not np.isfinite(dt_sec[idx]) or dt_sec[idx] <= 0.0` を判定し、真なら内部処理を行う。
10. state[idx] に 0.0 の結果を代入する。
11. Continue 文を実行する。
12. dt に `min(float(dt_sec[idx]), 60.0)` の結果を代入する。
13. alpha に `math.exp(-dt / tau)` の結果を代入する。
14. state[idx] に `alpha * state[idx - 1] + (1.0 - alpha) * current[idx - 1]` の結果を代入する。
15. state を返す。

代表コード断片

```
def polarization_current_trace(
    replay: pd.DataFrame,
    *,
    current_column: str,
    tau_sec: float,
) -> np.ndarray:
    """Return the 1-RC branch current state immediately before each sample."""
    current = pd.to_numeric(replay[current_column], errors="coerce").to_numpy(dtype=float)
    current = np.where(np.isfinite(current), current, 0.0)
    time_utc = pd.to_datetime(replay["time_utc"], utc=True, errors="coerce")
    dt_sec = time_utc.diff().dt.total_seconds().to_numpy(dtype=float)
    segment_starts = replay_segment_start_mask(replay)
    state = np.zeros(len(replay), dtype=float)
    tau = max(float(tau_sec), 1.0e-6)
    for idx in range(1, len(replay)):
        if segment_starts[idx] or not np.isfinite(dt_sec[idx]) or dt_sec[idx] <= 0.0:
            state[idx] = 0.0
            continue
        dt = min(float(dt_sec[idx]), 60.0)
```

```

    alpha = math.exp(-dt / tau)
    state[idx] = alpha * state[idx - 1] + (1.0 - alpha) * current[idx - 1]
return state

```

7.17 L544 関数 fit_battery_polarization

- 行範囲: L544–L679
- 所属: module 直下
- 定義: `fit_battery_polarization(replay:pd.DataFrame)->Dict[str,Any]`
- docstring: Fit a bounded one-RC branch and gate it on the last independent day.
- このブロックが直接呼ぶ主な関数・メソッド: Series, astype, bool, clip, dot, fillna, float, geomspace, get, int, isfinite, len
- 戻り値の要点: `{'adopted': adopted, 'reason': 'bounded_1rc_improves_training_and_last_day_holdout_rmse' if adopted else 'training_or_last_day_holdout_gate_failed', 'sample_count': int(base_valid.sum()), 'training_sample_count': int(training_valid.sum()), 'validation_sample_count': int(validation_valid.sum()), 'holdout_group': holdout_group, 'r_polarization_ohm': float(best['r_polarization_ohm']) if adopted else 0.0, 'tau_sec': float(best['tau_sec']), 'rmse_before_v': rmse_before, 'rmse_after_v': float(best['rmse_after_v']) if adopted else rmse_before, 'rmse_improvement_v': float(improvement if adopted else 0.0), 'validation_rmse_before_v': validation_before, 'validation_rmse_after_v': validation_after, 'validation_rmse_ratio': validation_ratio, 'validation_rmse_ratio_max': 1.0, 'method': 'bounded deterministic tau grid with closed-form least-squares Rp on earlier race days; last race day is an untouched adoption holdout'}` / `{'adopted': False, 'reason': 'insufficient_valid_voltage_samples', 'sample_count': int(base_valid.sum()), 'r_polarization_ohm': 0.0, 'tau_sec': 60.0, 'rmse_before_v': float('nan'), 'rmse_after_v': float('nan')}` / `{'adopted': False, 'reason': 'no_independent_day_holdout', 'sample_count': int(base_valid.sum()), 'training_sample_count': 0, 'validation_sample_count': 0, 'r_polarization_ohm': 0.0, 'tau_sec': 60.0, 'rmse_before_v': float(np.sqrt(np.mean(residual[base_valid]**2))), 'rmse_after_v': float(np.sqrt(np.mean(residual[base]**2)))}` / `{'adopted': False, 'reason': 'insufficient_independent_day_holdout_samples', 'sample_count': int(base_valid.sum()), 'training_sample_count': int(training_valid.sum()), 'validation_sample_count': int(validation_valid.sum()), 'holdout_group': holdout_group, 'r_polarization_ohm': 0.0, 'tau_sec': 60.0, 'rmse_before_v': float(np.sqrt(np.mean(residual[base_valid]**2))), 'rmse_after_v': float(np.sqrt(np.mean(residual[base]**2)))}`
- 主なローカル代入名: adopted, base_valid, best, best_state, corrected, denom, excluded, groups, holdout_group, improvement, residual, rmse, rmse_before, rp_ohm, state, tau_sec, time_local, training_valid, unique_groups, valid
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 7、ループ 1、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して collection を作る。

上から順の処理

1. `residual` に `(pd.to_numeric(replay['battery_voltage_v_obs'], errors='coerce') - pd.to_numeric(replay['battery_voltage_v_obs'], errors='coerce')).to_numpy(dtype=float)` の結果を代入する。
2. `excluded` に `replay.get('exclude_voltage_fit', pd.Series(False, index=replay.index)).fillna(True).astype(bool).to_numpy()` の結果を代入する。
3. `base_valid` に `np.isfinite(residual) & ~excluded` の結果を代入する。
4. 条件 `int(base_valid.sum()) < 500` を判定し、真なら内部処理を行う。
5. `{'adopted': False, 'reason': 'insufficient_valid_voltage_samples', 'sample_count': int(base_valid.sum()), 'r_polarization_ohm': 0.0, 'tau_sec': 60.0, 'rmse_before_v': float('nan'), 'rmse_after_v': float('nan')}` を返す。
6. 条件 `'day' in replay.columns` を判定し、真なら内部処理を行う。
7. `groups` に `pd.to_numeric(replay['day'], errors='coerce').to_numpy(dtype=float)` の結果を代入する。
8. 上の条件が偽の場合:
9. `time_local` に `pd.to_datetime(replay['time_utc'], utc=True, errors='coerce').dt.tz_convert(TIMEZONE_LOCAL)` の結果を代入する。
10. `groups` に `time_local.dt.strftime('%Y%m%d').astype(float).to_numpy()` の結果を代入する。
11. `unique_groups` に `sorted((float(value) for value in np.unique(groups[base_valid & np.isfinite(groups)])))` の結果を代入する。
12. 条件 `len(unique_groups) < 2` を判定し、真なら内部処理を行う。
13. `{'adopted': False, 'reason': 'no_independent_day_holdout', 'sample_count': int(base_valid.sum()), 'training_sample_count': 0, 'validation_sample_count': 0, 'r_polarization_ohm': 0.0, 'tau_sec': 60.0, 'rmse_before_v': float(np.sqrt(np.mean(residual[base_valid]**2))), 'rmse_after_v': float(np.sqrt(np.mean(residual[base_valid]**2)))}` を返す。
14. `holdout_group` に `unique_groups[-1]` の結果を代入する。
15. `training_valid` に `base_valid & np.isfinite(groups) & (groups != holdout_group)` の結果を代入する。
16. `validation_valid` に `base_valid & np.isfinite(groups) & (groups == holdout_group)` の結果を代入する。
17. 条件 `int(training_valid.sum()) < 500 or int(validation_valid.sum()) < 100` を判定し、真なら内部処理を行う。
18. `{'adopted': False, 'reason': 'insufficient_independent_day_holdout_samples', 'sample_count': int(base_valid.sum()), 'training_sample_count': int(training_valid.sum()), 'validation_sample_count': int(validation_valid.sum()), 'holdout_group': holdout_group, 'r_polarization_ohm': 0.0, 'tau_sec': 60.0, 'rmse_before_v': float(np.sqrt(np.mean(residual[base_valid]**2))), 'rmse_after_v': float(np.sqrt(np.mean(residual[base_valid]**2)))}` を返す。
19. `best` に `None` を代入する。
20. `np.geomspace(10.0, 600.0, 81)` を順に走査し、各要素を `tau_sec` に入れて処理する。
21. `state` に `polarization_current_trace(replay, current_column='battery_current_a_obs', tau_sec=float(tau_sec))` の結果を代入する。
22. `valid` に `training_valid & np.isfinite(state)` の結果を代入する。
23. `denom` に `float(np.dot(state[valid], state[valid]))` の結果を代入する。
24. 条件 `denom <= 1e-12` を判定し、真なら内部処理を行う。
25. `Continue` 文を実行する。

26. `rp_ohm` に `float(np.clip(-np.dot(residual[valid], state[valid]) / denom, 0.0, 0.12))` の結果を代入する。
27. `corrected` に `residual[valid] + rp_ohm * state[valid]` の結果を代入する。
28. `rmse` に `float(np.sqrt(np.mean(corrected ** 2)))` の結果を代入する。
29. 条件 `best is None or rmse < best['rmse_after_v']` を判定し、真なら内部処理を行う。
30. `best` に `{'r_polarization_ohm': rp_ohm, 'tau_sec': float(tau_sec), 'rmse_after_v': rmse}` の結果を代入する。
31. `rmse_before` に `float(np.sqrt(np.mean(residual[training_valid] ** 2)))` の結果を代入する。
32. 条件 `best is None` を判定し、真なら内部処理を行う。
33. `{'adopted': False, 'reason': 'no_finite_candidate', 'sample_count': int(base_valid.sum()), 'training_sample_count': int(training_valid.sum()), 'validation_sample_count': int(validation_valid.sum()), 'holdout_group': holdout_group, 'r_polarization_ohm': 0.0, 'tau_sec': 60.0, 'rmse_before_v': rmse_before, 'rmse_after_v': rmse_before}` を返す。
34. `validation_before` に `float(np.sqrt(np.mean(residual[validation_valid] ** 2)))` の結果を代入する。
35. `best_state` に `polarization_current_trace(replay, current_column='battery_current_a_obs', tau_sec=float(best['tau_sec']))` の結果を代入する。
36. `validation_corrected` に `residual[validation_valid] + float(best['r_polarization_ohm']) * best_state[validation_valid]` の結果を代入する。
37. `validation_after` に `float(np.sqrt(np.mean(validation_corrected ** 2)))` の結果を代入する。
38. `validation_ratio` に `validation_after / max(validation_before, 1e-12)` の結果を代入する。
39. `improvement` に `rmse_before - float(best['rmse_after_v'])` の結果を代入する。
40. `adopted` に `bool(best['r_polarization_ohm'] >= 0.001 and improvement >= 0.005 and np.isfinite(validation_ratio) and (validation_ratio <= 1.0))` の結果を代入する。
41. `{'adopted': adopted, 'reason': 'bounded_1rc_improves_training_and_last_day_holdout_rmse' if adopted else 'training_or_last_day_holdout_gate_failed', 'sample_count': int(base_valid.sum()), 'training_sample_count': int(training_valid.sum()), 'validation_sample_count': int(validation_valid.sum()), 'holdout_group': holdout_group, 'r_polarization_ohm': float(best['r_polarization_ohm'] if adopted else 0.0), 'tau_sec': float(best['tau_sec']), 'rmse_before_v': rmse_before, 'rmse_after_v': float(best['rmse_after_v'] if adopted else rmse_before), 'rmse_improvement_v': float(improvement if adopted else 0.0), 'validation_rmse_before_v': validation_before, 'validation_rmse_after_v': validation_after, 'validation_rmse_ratio': validation_ratio, 'validation_rmse_ratio_max': 1.0, 'method': 'bounded deterministic tau grid with closed-form least-squares Rp on earlier race days; last race day is an untouched adoption holdout'}` を返す。

代表コード断片

```
def fit_battery_polarization(replay: pd.DataFrame) -> Dict[str, Any]:
    """Fit a bounded one-RC branch and gate it on the last independent day."""
    residual = (
        pd.to_numeric(replay["battery_voltage_v_obs"], errors="coerce")
        - pd.to_numeric(replay["battery_voltage_v_pred"], errors="coerce")
    ).to_numpy(dtype=float)
    excluded = replay.get("exclude_voltage_fit", pd.Series(False, index=replay.index)).
    fillna(True).astype(bool).to_numpy()
    base_valid = np.isfinite(residual) & ~excluded
    if int(base_valid.sum()) < 500:
        return {
            "adopted": False,
            "reason": "insufficient_valid_voltage_samples",
            "sample_count": int(base_valid.sum()),
            "r_polarization_ohm": 0.0,
            "tau_sec": 60.0,
```

```

        "rmse_before_v": float("nan"),
        "rmse_after_v": float("nan"),
    }

    if "day" in replay.columns:
        groups = pd.to_numeric(replay["day"], errors="coerce").to_numpy(dtype=float)
    else:
        time_local = pd.to_datetime(replay["time_utc"], utc=True, errors="coerce").dt.
        tz_convert(
            TIMEZONE_LOCAL
        )
        groups = time_local.dt.strftime("%Y%m%d").astype(float).to_numpy()
    unique_groups = sorted(float(value) for value in np.unique(groups[base_valid & np.
    isfinite(groups)]))
    if len(unique_groups) < 2:
        return {
            "adopted": False,
            "reason": "no_independent_day_holdout",
            "sample_count": int(base_valid.sum()),
            "training_sample_count": 0,
            "validation_sample_count": 0,
            "r_polarization_ohm": 0.0,
        }
    ...

```

7.18 L682 関数 apply_battery_polarization

- 行範囲: L682–L703
- 所属: module 直下
- 定義: `apply_battery_polarization(replay:pd.DataFrame,dynamic_fit:Dict[str,Any],*,current_column:str)->pd.DataFrame`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `bool, copy, float, get, polarization_current_trace, to_numeric`
- 戻り値の要点: `out / out`
- 主なローカル代入名: `out, polarization_v, state`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `out` に `replay.copy()` の結果を代入する。
2. `out['battery_voltage_v_pred_static']` に `pd.to_numeric(out['battery_voltage_v_pred'], errors='coerce')` の結果を代入する。

3. 条件 `not bool(dynamic_fit.get('adopted', False))` を判定し、真なら内部処理を行う。
4. `out['battery_polarization_v']` に 0.0 の結果を代入する。
5. `out` を返す。
6. `state` に `polarization_current_trace(out, current_column=current_column, tau_sec=float(dynamic_fit['tau_sec']))` の結果を代入する。
7. `polarization_v` に `float(dynamic_fit['r_polarization_ohm']) * state` の結果を代入する。
8. `out['battery_polarization_v']` に `polarization_v` の結果を代入する。
9. `out['battery_voltage_v_pred']` に `out['battery_voltage_v_pred_static'] - polarization_v` の結果を代入する。
10. `out` を返す。

代表コード断片

```
def apply_battery_polarization(
    replay: pd.DataFrame,
    dynamic_fit: Dict[str, Any],
    *,
    current_column: str,
) -> pd.DataFrame:
    out = replay.copy()
    out["battery_voltage_v_pred_static"] = pd.to_numeric(
        out["battery_voltage_v_pred"], errors="coerce"
    )
    if not bool(dynamic_fit.get("adopted", False)):
        out["battery_polarization_v"] = 0.0
        return out
    state = polarization_current_trace(
        out,
        current_column=current_column,
        tau_sec=float(dynamic_fit["tau_sec"]),
    )
    polarization_v = float(dynamic_fit["r_polarization_ohm"]) * state
    out["battery_polarization_v"] = polarization_v
    out["battery_voltage_v_pred"] = out["battery_voltage_v_pred_static"] - polarization_v
    return out
```

7.19 L706 関数 `resolve_fit_plan`

- 行範囲: L706–L747
- 所属: module 直下
- 定義: `resolve_fit_plan(options:dict|None,*,quality:str)->Dict[str,Any]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `dict`, `float`, `get`, `isinstance`, `items`, `lower`, `str`, `strip`
- 戻り値の要点: `plan`
- 主なローカル代入名: `key_map`, `opt_key`, `options`, `out_key`, `plan`, `quality_norm`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. options に options if isinstance(options, dict) else {} の結果を代入する。
2. quality_norm に str(quality or options.get('fit_quality', 'standard')).strip().lower() の結果を代入する。
3. 条件 quality_norm not in FIT_QUALITY_PRESETS を判定し、真なら内部処理を行う。
4. quality_norm に'standard' の結果を代入する。
5. plan に dict(FIT_QUALITY_PRESETS[quality_norm]) の結果を代入する。
6. key_map に {'battery_restart_count': 'battery_restart_count', 'battery_maxiter': 'battery_maxiter', 'motion_restart_count': 'motion_restart_count', 'motion_maxiter': 'motion_maxiter', 'joint_restart_count': 'joint_restart_count', 'joint_random_start_count': 'joint_random_start_count', 'joint_local_topk': 'joint_local_topk', 'joint_maxiter': 'joint_maxiter', 'fit_stride': 'fit_stride', 'allow_map_shape_fit': 'allow_map_shape_fit', 'post_refine_enabled': 'post_refine_enabled'} の結果を代入する。
7. key_map.items() を順に走査し、各要素を (out_key, opt_key) に入れて処理する。
8. 条件 opt_key in options を判定し、真なら内部処理を行う。
9. plan[out_key] に options[opt_key] の結果を代入する。
10. plan['panel_deployment_stopped_speed_kmh'] に float(options.get('panel_deployment_stopped_speed_kmh', 2.0)) の結果を代入する。
11. plan['panel_deployment_min_dwell_sec'] に float(options.get('panel_deployment_min_dwell_sec', 300.0)) の結果を代入する。
12. plan['panel_deployment_max_sample_gap_sec'] に float(options.get('panel_deployment_max_sample_gap_sec', 60.0)) の結果を代入する。
13. plan['panel_control_stop_tolerance_kmh'] に float(options.get('panel_control_stop_tolerance_kmh', 1.0)) の結果を代入する。
14. plan['quality'] に quality_norm の結果を代入する。
15. plan['terminal_anchor_role'] に str(options.get('terminal_anchor_role', 'independent_consensus')) の結果を代入する。
16. plan['sensor_filter'] に options.get('sensor_filter', {}) の結果を代入する。
17. plan['acceleration_observation'] に options.get('acceleration_observation', {}) の結果を代入する。
18. plan['grade_observation'] に options.get('grade_observation', {}) の結果を代入する。
19. plan を返す。

代表コード断片

```
def resolve_fit_plan(options: dict | None, *, quality: str) -> Dict[str, Any]:
    options = options if isinstance(options, dict) else {}
    quality_norm = str(quality or options.get("fit_quality", "standard")).strip().lower()
    if quality_norm not in FIT_QUALITY_PRESETS:
        quality_norm = "standard"
    plan = dict(FIT_QUALITY_PRESETS[quality_norm])
    key_map = {
        "battery_restart_count": "battery_restart_count",
```

```

        "battery_maxiter": "battery_maxiter",
        "motion_restart_count": "motion_restart_count",
        "motion_maxiter": "motion_maxiter",
        "joint_restart_count": "joint_restart_count",
        "joint_random_start_count": "joint_random_start_count",
        "joint_local_topk": "joint_local_topk",
        "joint_maxiter": "joint_maxiter",
        "fit_stride": "fit_stride",
        "allow_map_shape_fit": "allow_map_shape_fit",
        "post_refine_enabled": "post_refine_enabled",
    }
    for out_key, opt_key in key_map.items():
        if opt_key in options:
            plan[out_key] = options[opt_key]
    plan["panel_deployment_stopped_speed_kmh"] = float(
        options.get("panel_deployment_stopped_speed_kmh", 2.0)
    )
    plan["panel_deployment_min_dwell_sec"] = float(
        options.get("panel_deployment_min_dwell_sec", 300.0)
    )
    plan["panel_deployment_max_sample_gap_sec"] = float(
        options.get("panel_deployment_max_sample_gap_sec", 60.0)
    )
    plan["panel_control_stop_tolerance_km"] = float(
        options.get("panel_control_stop_tolerance_km", 1.0)
    )
    plan["quality"] = quality_norm
...

```

7.20 L750 関数 `resolve_identification_output_layout`

- 行範囲: L750–L774
- 所属: module 直下
- 定義: `resolve_identification_output_layout(package_dir:Path,profile_cfg:dict,*, output_tag_override:str|None=None)->Dict[str,Path|str]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `get`, `resolve_relative`, `str`, `strip`, `sub`
- 戻り値の要点: `{'tag': tag, 'output_root': output_root, 'run_root': run_root, 'report_root': report_root, 'grounded_maps': run_root / 'grounded_base_maps', 'adopted_maps': run_root / 'adopted_maps'}`
- 主なローカル代入名: `identification_cfg`, `output_root`, `raw_tag`, `report_root`, `run_root`, `tag`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `identification_cfg` に `profile_cfg.get('identification', {})` or `{}` の結果を代入する。
2. `output_root` に `resolve_relative(package_dir, str(identification_cfg.get('output_dir', 'outputs/identification') or 'outputs/identification'))` の結果を代入する。
3. `raw_tag` に `output_tag_override` の結果を代入する。
4. 条件 `raw_tag is None` を判定し、真なら内部処理を行う。
5. `raw_tag` に `str(identification_cfg.get('output_tag', '') or '')` の結果を代入する。
6. `tag` に `re.sub('[^A-Za-z0-9_-]+', '_', str(raw_tag).strip()).strip('.')` の結果を代入する。
7. `run_root` に `output_root / 'runs' / tag` if `tag` else `output_root` の結果を代入する。
8. `report_root` に `run_root / 'reports'` if `tag` else `package_dir / 'outputs' / 'reports'` の結果を代入する。
9. `{'tag': tag, 'output_root': output_root, 'run_root': run_root, 'report_root': report_root, 'grounded_maps': run_root / 'grounded_base_maps', 'adopted_maps': run_root / 'adopted_maps'}` を返す。

代表コード断片

```
def resolve_identification_output_layout(
    package_dir: Path,
    profile_cfg: dict,
    *,
    output_tag_override: str | None = None,
) -> Dict[str, Path | str]:
    identification_cfg = profile_cfg.get("identification", {}) or {}
    output_root = resolve_relative(
        package_dir,
        str(identification_cfg.get("output_dir", "outputs/identification") or "outputs/identification"),
    )
    raw_tag = output_tag_override
    if raw_tag is None:
        raw_tag = str(identification_cfg.get("output_tag", "") or "")
    tag = re.sub(r"[^A-Za-z0-9_-]+", "_", str(raw_tag).strip()).strip(".")
    run_root = output_root / "runs" / tag if tag else output_root
    report_root = run_root / "reports" if tag else package_dir / "outputs" / "reports"
    return {
        "tag": tag,
        "output_root": output_root,
        "run_root": run_root,
        "report_root": report_root,
        "grounded_maps": run_root / "grounded_base_maps",
        "adopted_maps": run_root / "adopted_maps",
    }
```

7.21 L777 関数 `identification_profile_output_path`

- 行範囲: L777–L786
- 所属: module 直下
- 定義: `identification_profile_output_path(canonical_profile:Path,run_output_dir:Path,*,output_tag:str,adopt_profile:bool)->Path`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `str`, `strip`

- 戻り値の要点: `Path(canonical_profile) / Path(run_output_dir) / 'profile_candidate.yaml'`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `str(output_tag).strip() and (not adopt_profile)` を判定し、真なら内部処理を行う。
2. `Path(run_output_dir) / 'profile_candidate.yaml'` を返す。
3. `Path(canonical_profile)` を返す。

代表コード断片

```
def identification_profile_output_path(
    canonical_profile: Path,
    run_output_dir: Path,
    *,
    output_tag: str,
    adopt_profile: bool,
) -> Path:
    if str(output_tag).strip() and not adopt_profile:
        return Path(run_output_dir) / "profile_candidate.yaml"
    return Path(canonical_profile)
```

7.22 L789 関数 `load_ocv_df`

- 行範囲: L789–L794
- 所属: module 直下
- 定義: `load_ocv_df(profile_cfg:dict,profile_yaml:Path)->pd.DataFrame`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `FileNotFoundError`, `get`, `read_csv`, `resolve_relative`, `str`, `strip`
- 戻り値の要点: `pd.read_csv(ocv_path)`
- 主なローカル代入名: `ocv_path`, `raw`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `FileNotFoundError('profile.paths.ocv_soc_map is required')`
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. raw に `str((profile_cfg.get('paths', {}) or {}).get('ocv_soc_map', '') or '').strip()` の結果を代入する。
2. 条件 `not raw` を判定し、真なら内部処理を行う。
3. `FileNotFoundError('profile.paths.ocv_soc_map is required')` を送出する。
4. `ocv_path` に `resolve_relative(profile_yaml.parent, raw)` の結果を代入する。
5. `pd.read_csv(ocv_path)` を返す。

代表コード断片

```
def load_ocv_df(profile_cfg: dict, profile_yaml: Path) -> pd.DataFrame:
    raw = str((profile_cfg.get("paths", {}) or {}).get("ocv_soc_map", "") or "").strip()
    if not raw:
        raise FileNotFoundError("profile.paths.ocv_soc_map is required")
    ocv_path = resolve_relative(profile_yaml.parent, raw)
    return pd.read_csv(ocv_path)
```

7.23 L797 関数 build_source_map_assets

- 行範囲: L797–L818
- 所属: module 直下
- 定義: `build_source_map_assets(profile_cfg:dict,profile_yaml:Path)->Dict[str,Path]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `FileNotFoundError`, `get`, `rel`, `resolve_relative`, `str`, `strip`
- 戻り値の要点: `{'drive_eff_map': rel('drive_eff_map'), 'drive_map_eco': rel('drive_map_eco', paths.get('drive_eff_map', '')), 'drive_map_power': rel('drive_map_power', paths.get('drive_eff_map', '')), 'regen_eff_map': rel('regen_eff_map', 'regen_map_eco': rel('regen_map_eco', paths.get('regen_eff_map', '')), 'regen_map_power': rel('regen_map_power', paths.get('regen_eff_map', '')), 'rint_map': rel('rint_map'), 'panel_eff_map': rel('panel_eff_map'), 'mppt_eff_map': rel('mppt_eff_map'), 'ocv_soc_map': rel('ocv_soc_map')}` / `resolve_relative(base_dir, raw)`
- 主なローカル代入名: `base_dir`, `paths`, `raw`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `FileNotFoundError(f'profile.paths.{key} is required')`
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `base_dir` に `profile_yaml.parent` の結果を代入する。
2. `paths` に `profile_cfg.get('paths', {})` or `{}` の結果を代入する。
3. 関数 `rel` を定義する。
4. `{'drive_eff_map': rel('drive_eff_map'), 'drive_map_eco': rel('drive_map_eco', paths.get('drive_eff_map', "")), 'drive_map_power': rel('drive_map_power', paths.get('drive_eff_map', "")), 'regen_eff_map': rel('regen_eff_map'), 'regen_map_eco': rel('regen_map_eco', paths.get('regen_eff_map', "")), 'regen_map_power': rel('regen_map_power', paths.get('regen_eff_map', "")), 'rint_map': rel('rint_map'), 'panel_eff_map': rel('panel_eff_map'), 'mppt_eff_map': rel('mppt_eff_map'), 'ocv_soc_map': rel('ocv_soc_map')}` を返す。

代表コード断片

```
def build_source_map_assets(profile_cfg: dict, profile_yaml: Path) -> Dict[str, Path]:
    base_dir = profile_yaml.parent
    paths = profile_cfg.get("paths", {}) or {}

    def rel(key: str, fallback: str | None = None) -> Path:
        raw = str(paths.get(key, fallback or "") or "").strip()
        if not raw:
            raise FileNotFoundError(f"profile.paths.{key} is required")
        return resolve_relative(base_dir, raw)

    return {
        "drive_eff_map": rel("drive_eff_map"),
        "drive_map_eco": rel("drive_map_eco", paths.get("drive_eff_map", "")),
        "drive_map_power": rel("drive_map_power", paths.get("drive_eff_map", "")),
        "regen_eff_map": rel("regen_eff_map"),
        "regen_map_eco": rel("regen_map_eco", paths.get("regen_eff_map", "")),
        "regen_map_power": rel("regen_map_power", paths.get("regen_eff_map", "")),
        "rint_map": rel("rint_map"),
        "panel_eff_map": rel("panel_eff_map"),
        "mppt_eff_map": rel("mppt_eff_map"),
        "ocv_soc_map": rel("ocv_soc_map"),
    }
```

7.24 L801 関数 `build_source_map_assets.rel`

- 行範囲: L801–L805
- 所属: `build_source_map_assets`
- 定義: `rel(key:str, fallback:str|None=None) -> Path`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `FileNotFoundError`, `get`, `resolve_relative`, `str`, `strip`
- 戻り値の要点: `resolve_relative(base_dir, raw)`
- 主なローカル代入名: `raw`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `FileNotFoundError(f'profile.paths.{key} is required')`
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. raw に `str(paths.get(key, fallback or "") or "").strip()` の結果を代入する。
2. 条件 `not raw` を判定し、真なら内部処理を行う。
3. `FileNotFoundError(f"profile.paths.{key} is required")` を送出する。
4. `resolve_relative(base_dir, raw)` を返す。

代表コード断片

```
def rel(key: str, fallback: str | None = None) -> Path:
    raw = str(paths.get(key, fallback or "") or "").strip()
    if not raw:
        raise FileNotFoundError(f"profile.paths.{key} is required")
    return resolve_relative(base_dir, raw)
```

7.25 L821 関数 `apply_actual_event_annotations`

- 行範囲: L821–L860
- 所属: module 直下
- 定義: `apply_actual_event_annotations(work:pd.DataFrame,payload:dict|None)->pd.DataFrame`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Timestamp`, `_append_reason_column`, `any`, `astype`, `bool`, `copy`, `fillna`, `get`, `hasattr`, `isinstance`, `str`, `strip`
- 戻り値の要点: `out / work / work`
- 主なローカル代入名: `end_utc`, `event`, `events`, `fit_flags`, `key`, `mask`, `out`, `raw_end`, `raw_start`, `reason`, `start_utc`, `time_utc`, `touched`, `tz_name`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 9、ループ 3、`try` 1

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `not isinstance(payload, dict)` を判定し、真なら内部処理を行う。
2. `work` を返す。
3. `events` に `payload.get('events', payload)` の結果を代入する。
4. 条件 `not isinstance(events, list) or not events` を判定し、真なら内部処理を行う。
5. `work` を返す。
6. `out` に `work.copy()` の結果を代入する。
7. ('exclude_power_fit', 'exclude_voltage_fit', 'exclude_weather_fit') を順に走査し、各要素を `key` に入れて処理する。
8. 条件 `key not in out.columns` を判定し、真なら内部処理を行う。
9. `out[key]` に `False` の結果を代入する。
10. `out[key]` に `out[key].fillna(False).astype(bool)` の結果を代入する。
11. 条件 'exclude_reason' not in out.columns を判定し、真なら内部処理を行う。
12. `out['exclude_reason']` に "" の結果を代入する。
13. `time_utc` に `pd.to_datetime(out['time_utc'], format='mixed', utc=True, errors='coerce')` の結果を代入する。
14. `events` を順に走査し、各要素を `event` に入れて処理する。
15. 条件 `not isinstance(event, dict)` を判定し、真なら内部処理を行う。
16. `Continue` 文を実行する。
17. `raw_start` に `str(event.get('start_local', '') or '').strip()` の結果を代入する。
18. `raw_end` に `str(event.get('end_local', '') or '').strip()` の結果を代入する。
19. 条件 `not raw_start or not raw_end` を判定し、真なら内部処理を行う。
20. `Continue` 文を実行する。
21. `tz_name` に `str(event.get('timezone', TIMEZONE_LOCAL.key if hasattr(TIMEZONE_LOCAL, 'key') else 'Australia/Darwin') or 'Australia/Darwin')` の結果を代入する。
22. 例外処理を伴う `try` ブロックを実行する。
23. `start_utc` に `pd.Timestamp(raw_start).tz_localize(tz_name).tz_convert('UTC')` の結果を代入する。
24. `end_utc` に `pd.Timestamp(raw_end).tz_localize(tz_name).tz_convert('UTC')` の結果を代入する。
25. `Exception` を捕捉した場合:
26. `Continue` 文を実行する。
27. `mask` に `(time_utc >= start_utc) & (time_utc <= end_utc)` の結果を代入する。
28. 条件 `not mask.any()` を判定し、真なら内部処理を行う。
29. `Continue` 文を実行する。
30. `fit_flags` に `event.get('fit_flags', {})` if `isinstance(event.get('fit_flags', {}), dict)` else `{}` の結果を代入する。
31. `reason` に `str(event.get('label', event.get('kind', 'actual_event')) or 'actual_event')` の結果を代入する。

32. touched に False の結果を代入する。
33. ('exclude_power_fit', 'exclude_voltage_fit', 'exclude_weather_fit') を順に走査し、各要素を key に入れて処理する。
34. 条件 bool(fit_flags.get(key, False)) を判定し、真なら内部処理を行う。
35. out.loc[mask, key] に True の結果を代入する。
36. touched に True の結果を代入する。
37. 条件 touched を判定し、真なら内部処理を行う。
38. _append_reason_column(...) を実行する。
39. out を返す。

代表コード断片

```
def apply_actual_event_annotations(work: pd.DataFrame, payload: dict | None) -> pd.DataFrame:
    :
    if not isinstance(payload, dict):
        return work
    events = payload.get("events", payload)
    if not isinstance(events, list) or not events:
        return work
    out = work.copy()
    for key in ("exclude_power_fit", "exclude_voltage_fit", "exclude_weather_fit"):
        if key not in out.columns:
            out[key] = False
        out[key] = out[key].fillna(False).astype(bool)
    if "exclude_reason" not in out.columns:
        out["exclude_reason"] = ""
    time_utc = pd.to_datetime(out["time_utc"], format="mixed", utc=True, errors="coerce")
    for event in events:
        if not isinstance(event, dict):
            continue
        raw_start = str(event.get("start_local", "") or "").strip()
        raw_end = str(event.get("end_local", "") or "").strip()
        if not raw_start or not raw_end:
            continue
        tz_name = str(event.get("timezone", TIMEZONE_LOCAL.key if hasattr(TIMEZONE_LOCAL, "key")
                                else "Australia/Darwin") or "Australia/Darwin")
        try:
            start_utc = pd.Timestamp(raw_start).tz_localize(tz_name).tz_convert("UTC")
            end_utc = pd.Timestamp(raw_end).tz_localize(tz_name).tz_convert("UTC")
        except Exception:
            continue
        mask = (time_utc >= start_utc) & (time_utc <= end_utc)
        if not mask.any():
            continue
        fit_flags = event.get("fit_flags", {}) if isinstance(event.get("fit_flags", {}), dict) else {}
        reason = str(event.get("label", event.get("kind", "actual_event")) or "actual_event")
    )
    touched = False
    for key in ("exclude_power_fit", "exclude_voltage_fit", "exclude_weather_fit"):
        if bool(fit_flags.get(key, False)):
    ...
```

7.26 L863 関数 truncate_at_retire_event

- 行範囲: L863–L889

- 所属: module 直下
- 定義: `truncate_at_retire_event(work:pd.DataFrame,payload:dict|None)->pd.DataFrame`
- docstring: End historical replay at the first authoritative retirement timestamp.
- このブロックが直接呼ぶ主な関数・メソッド: `Timestamp`, `ValueError`, `append`, `copy`, `get`, `isinstance`, `isoformat`, `lower`, `min`, `reset_index`, `str`, `strip`
- 戻り値の要点: `retained.reset_index(drop=True) / work / work / work`
- 主なローカル代入名: `cutoff`, `cutoffs`, `event`, `events`, `raw_start`, `retained`, `time_utc`, `tz_name`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError(f'retire-event cutoff {cutoff.isoformat()} removed the complete replay')`
- 制御構造の規模: 条件分岐 6、ループ 1、try 1

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `not isinstance(payload, dict)` を判定し、真なら内部処理を行う。
2. `work` を返す。
3. `events` に `payload.get('events', payload)` の結果を代入する。
4. 条件 `not isinstance(events, list)` を判定し、真なら内部処理を行う。
5. `work` を返す。
6. `cutoffs` に `[]` を代入する。
7. `events` を順に走査し、各要素を `event` に入れて処理する。
8. 条件 `not isinstance(event, dict) or str(event.get('kind', '')).strip().lower() != 'retire_event'` を判定し、真なら内部処理を行う。
9. Continue 文を実行する。
10. `raw_start` に `str(event.get('start_local', '') or '').strip()` の結果を代入する。
11. 条件 `not raw_start` を判定し、真なら内部処理を行う。
12. Continue 文を実行する。
13. `tz_name` に `str(event.get('timezone', 'Australia/Darwin') or 'Australia/Darwin')` の結果を代入する。
14. 例外処理を伴う try ブロックを実行する。
15. `cutoffs.append(...)` を実行する。
16. Exception を捕捉した場合:
17. Continue 文を実行する。

18. 条件 `not cutoffs` を判定し、真なら内部処理を行う。
19. `work` を返す。
20. `cutoff` に `min(cutoffs)` の結果を代入する。
21. `time_utc` に `pd.to_datetime(work['time_utc'], format='mixed', utc=True, errors='coerce')` の結果を代入する。
22. `retained` に `work.loc[time_utc <= cutoff].copy()` の結果を代入する。
23. 条件 `retained.empty` を判定し、真なら内部処理を行う。
24. `ValueError(f'retire-event cutoff {cutoff.isoformat()} removed the complete replay')` を送出する。
25. `retained.reset_index(drop=True)` を返す。

代表コード断片

```
def truncate_at_retire_event(work: pd.DataFrame, payload: dict | None) -> pd.DataFrame:
    """End historical replay at the first authoritative retirement timestamp."""
    if not isinstance(payload, dict):
        return work
    events = payload.get("events", payload)
    if not isinstance(events, list):
        return work
    cutoffs: list[pd.Timestamp] = []
    for event in events:
        if not isinstance(event, dict) or str(event.get("kind", "")).strip().lower() != "retire_event":
            continue
        raw_start = str(event.get("start_local", "") or "").strip()
        if not raw_start:
            continue
        tz_name = str(event.get("timezone", "Australia/Darwin") or "Australia/Darwin")
        try:
            cutoffs.append(pd.Timestamp(raw_start).tz_localize(tz_name).tz_convert("UTC"))
        except Exception:
            continue
    if not cutoffs:
        return work
    cutoff = min(cutoffs)
    time_utc = pd.to_datetime(work["time_utc"], format="mixed", utc=True, errors="coerce")
    retained = work.loc[time_utc <= cutoff].copy()
    if retained.empty:
        raise ValueError(f"retire-event cutoff {cutoff.isoformat()} removed the complete replay")
    return retained.reset_index(drop=True)
```

7.27 L892 関数 `normalize_generic_log`

- 行範囲: L892–L991
- 所属: module 直下
- 定義: `normalize_generic_log(log_csv: Path, *, actual_event_payload: dict | None = None, base_model = None, options: dict | None = None) -> pd.DataFrame`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `apply_actual_event_annotations`, `apply_sensor_quality_annotations`, `astype`, `clip`, `copy`, `diff`, `dropna`, `extend`, `fillna`, `float`, `items`

- 戻り値の要点: work
- 主なローカル代入名: accel, column, df, dt, key, numeric_cols, required_defaults, segment_start, speed_ms, time_gap, value, work
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: ValueError('normalized replay log must contain time_utc')
- 制御構造の規模: 条件分岐 12、ループ 3、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して collection を作る。

上から順の処理

1. df に pd.read_csv(log_csv, low_memory=False) の結果を代入する。
2. 条件 'time_utc' not in df.columns を判定し、真なら内部処理を行う。
3. ValueError('normalized replay log must contain time_utc') を送出する。
4. work に df.copy() の結果を代入する。
5. work['time_utc'] に pd.to_datetime(work['time_utc'], format='mixed', utc=True) の結果を代入する。
6. work に work.sort_values('time_utc').reset_index(drop=True) の結果を代入する。
7. required_defaults に {'s_km': np.nan, 'speed_kmh': 0.0, 'slope_pct': 0.0, 'route_heading_deg': 0.0, 'headwind_archive_ms': 0.0, 'GHI_archive': 0.0, 'Tamb_archive_C': 25.0, 'solar_power_w_obs': 0.0, 'battery_power_w_obs': 0.0, 'battery_current_a': 0.0, 'battery_voltage_v': np.nan, 'lat': np.nan, 'lon': np.nan, 'alt_m': 0.0} の結果を代入する。
8. required_defaults.items() を順に走査し、各要素を (key, value) に入れて処理する。
9. 条件 key not in work.columns を判定し、真なら内部処理を行う。
10. work[key] に value の結果を代入する。
11. 条件 'dt_sec' not in work.columns を判定し、真なら内部処理を行う。
12. dt に work['time_utc'].diff().dt.total_seconds().fillna(0.0) の結果を代入する。
13. 条件 len(dt) >= 2 and dt.iloc[0] <= 0.0 を判定し、真なら内部処理を行う。
14. dt.iloc[0] に float(np.nanmedian(dt.iloc[1:].replace(0.0, np.nan).dropna())) if len(dt.iloc[1:].dropna()) else 5.0 の結果を代入する。
15. work['dt_sec'] に dt.clip(lower=0.0).fillna(5.0) の結果を代入する。
16. 条件 'time_local' not in work.columns を判定し、真なら内部処理を行う。
17. work['time_local'] に work['time_utc'].dt.tz_convert(TIMEZONE_LOCAL).astype(str) の結果を代入する。
18. 条件 'day' not in work.columns を判定し、真なら内部処理を行う。
19. work['day'] に (work['time_utc'].dt.normalize() - work['time_utc'].dt.normalize().min()).dt.days + 1 の結果を代入する。

20. ('exclude_power_fit', 'exclude_voltage_fit', 'exclude_weather_fit') を順に走査し、各要素を key に入れて処理する。
21. 条件 `key not in work.columns` を判定し、真なら内部処理を行う。
22. `work[key]` に False の結果を代入する。
23. `work[key]` に `work[key].fillna(False).astype(bool)` の結果を代入する。
24. 条件 'exclude_reason' not in work.columns を判定し、真なら内部処理を行う。
25. `work['exclude_reason']` に "" の結果を代入する。
26. 条件 'GHI_effective' not in work.columns を判定し、真なら内部処理を行う。
27. `work['GHI_effective']` に `work['GHI_archive']` の結果を代入する。
28. 条件 'Tcell_effective_C' not in work.columns を判定し、真なら内部処理を行う。
29. `work['Tcell_effective_C']` に `work['Tamb_archive_C'] + 0.03 * pd.to_numeric(work['GHI_effective'], errors='coerce').fillna(0.0)` の結果を代入する。
30. 条件 'headwind_effective_ms' not in work.columns を判定し、真なら内部処理を行う。
31. `work['headwind_effective_ms']` に `work['headwind_archive_ms']` の結果を代入する。
32. `numeric_cols` に ['s_kmh', 'speed_kmh', 'slope_pct', 'route_heading_deg', 'headwind_archive_ms', 'headwind_effective_ms', 'GHI_archive', 'GHI_effective', 'Tamb_archive_C', 'Tcell_effective_C', 'solar_power_w_obs', 'battery_power_w_obs', 'battery_current_a', 'battery_voltage_v', 'dt_sec', 'lat', 'lon', 'alt_m'] の結果を代入する。
33. `numeric_cols.extend(...)` を実行する。
34. `numeric_cols` を順に走査し、各要素を key に入れて処理する。
35. `work[key]` に `pd.to_numeric(work[key], errors='coerce')` の結果を代入する。
36. `speed_ms` に `work['speed_kmh'].fillna(0.0) / 3.6` の結果を代入する。
37. `dt` に `work['dt_sec'].replace(0.0, np.nan)` の結果を代入する。
38. `accel` に `speed_ms.diff() / dt` の結果を代入する。
39. `segment_start` に `work['day'].ne(work['day'].shift(1))` の結果を代入する。
40. `time_gap` に `work['time_utc'].diff().dt.total_seconds().fillna(0.0) > 7200.0` の結果を代入する。
41. `accel.loc[segment_start | time_gap]` に 0.0 の結果を代入する。
42. `work['accel_ms2']` に `accel.replace([np.inf, -np.inf], np.nan).rolling(5, center=True, min_periods=1).median().fillna(0.0).clip(1.5, upper=1.5)` の結果を代入する。
43. `work` に `apply_actual_event_annotations(work, actual_event_payload)` の結果を代入する。
44. `work` に `truncate_at_retire_event(work, actual_event_payload)` の結果を代入する。
45. 条件 `base_model is not None` を判定し、真なら内部処理を行う。
46. `work` に `apply_sensor_quality_annotations(work, base_model=base_model, options=options)` の結果を代入する。
47. `work` を返す。

代表コード断片

```
def normalize_generic_log(
    log_csv: Path,
    *,
    actual_event_payload: dict | None = None,
    base_model=None,
    options: dict | None = None,
) -> pd.DataFrame:
    df = pd.read_csv(log_csv, low_memory=False)
    if "time_utc" not in df.columns:
        raise ValueError("normalized replay log must contain time_utc")
    work = df.copy()
    work["time_utc"] = pd.to_datetime(work["time_utc"], format="mixed", utc=True)
    work = work.sort_values("time_utc").reset_index(drop=True)

    required_defaults = {
        "s_km": np.nan,
        "speed_kmh": 0.0,
        "slope_pct": 0.0,
        "route_heading_deg": 0.0,
        "headwind_archive_ms": 0.0,
        "GHI_archive": 0.0,
        "Tamb_archive_C": 25.0,
        "solar_power_w_obs": 0.0,
        "battery_power_w_obs": 0.0,
        "battery_current_a": 0.0,
        "battery_voltage_v": np.nan,
        "lat": np.nan,
        "lon": np.nan,
        "alt_m": 0.0,
    }
    for key, value in required_defaults.items():
        if key not in work.columns:
            work[key] = value

    if "dt_sec" not in work.columns:
    ...
```

7.28 L994 関数 build_terminal_anchor

- 行範囲: L994–L1026
- 所属: module 直下
- 定義: `build_terminal_anchor(log_df:pd.DataFrame,ocv_df:pd.DataFrame,base_model,anchor_km:float)->Dict[str,float]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `asarray`, `astype`, `copy`, `fillna`, `float`, `infer_soc_from_loaded_state`, `int`, `isfinite`, `itertuples`, `len`, `median`
- 戻り値の要点: `{'count': int(len(window)), 's_km': float(window['s_km'].median()), 'time_utc': pd.to_datetime(window['time_utc'].iloc[-1], utc=True).strftime('%Y-%m-%dT%H:%M:%SZ'), 'voltage_v': float(window['battery_voltage_v'].median()), 'current_a': float(window['battery_current_a'].median()), 'temp_c': float(window['Tamb_archive_C'].median()), 'soc_target': float(np.nanmedian(np.asarray(soc_targets, dtype=float)))}`
- 主なローカル代入名: `row`, `soc_targets`, `valid`, `window`
- 読み取る主なインスタンス属性: 特になし。

- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `valid` に `np.isfinite(log_df['battery_voltage_v']) & np.isfinite(log_df['battery_current_a']) & np.isfinite(log_df['Tamb_archive_C']) & np.isfinite(log_df['s_km']) & (log_df['battery_voltage_v'] >= INVALID_PACK_VOLTAGE_MIN_V) & ~log_df['exclude_voltage_fit'].fillna(False).astype(bool)` の結果を代入する。
2. `window` に `log_df.loc[valid & (np.abs(log_df['s_km'] - float(anchor_km)) <= 1.5)].copy()` の結果を代入する。
3. 条件 `window.empty` を判定し、真なら内部処理を行う。
4. `window` に `log_df.loc[valid].tail(6).copy()` の結果を代入する。
5. 上の条件が偽の場合:
6. `window` に `window.tail(min(6, len(window))).copy()` の結果を代入する。
7. `soc_targets` に `[infer_soc_from_loaded_state(float(row.battery_voltage_v), float(row.battery_current_a), float(row.Tamb_archive_C), ocv_df, base_model) for row in window.itertuples(index=False)]` の結果を代入する。
8. `{'count': int(len(window)), 's_km': float(window['s_km'].median()), 'time_utc': pd.to_datetime(window['time_utc'].iloc[1], utc=True).strftime('%Y-%m-%dT%H:%M:%SZ'), 'voltage_v': float(window['battery_voltage_v'].median()), 'current_a': float(window['battery_current_a'].median()), 'temp_c': float(window['Tamb_archive_C'].median()), 'soc_target': float(np.nanmedian(np.asarray(soc_targets, dtype=float)))}` を返す。

代表コード断片

```
def build_terminal_anchor(log_df: pd.DataFrame, ocv_df: pd.DataFrame, base_model, anchor_km: float) -> Dict[str, float]:
    valid = (
        np.isfinite(log_df["battery_voltage_v"])
        & np.isfinite(log_df["battery_current_a"])
        & np.isfinite(log_df["Tamb_archive_C"])
        & np.isfinite(log_df["s_km"])
        & (log_df["battery_voltage_v"] >= INVALID_PACK_VOLTAGE_MIN_V)
        & (~log_df["exclude_voltage_fit"].fillna(False).astype(bool))
    )
    window = log_df.loc[valid & (np.abs(log_df["s_km"] - float(anchor_km)) <= 1.5)].copy()
    if window.empty:
        window = log_df.loc[valid].tail(6).copy()
    else:
        window = window.tail(min(6, len(window))).copy()
    soc_targets = [
        infer_soc_from_loaded_state(
            float(row.battery_voltage_v),
            float(row.battery_current_a),
            float(row.Tamb_archive_C),
            ocv_df,
            base_model,
        )
    ]
```

```

    for row in window.itertuples(index=False)
]
return {
    "count": int(len(window)),
    "s_km": float(window["s_km"].median()),
    "time_utc": pd.to_datetime(window["time_utc"].iloc[-1], utc=True).strftime("%Y-%m-%dT%H:%M:%SZ"),
    "voltage_v": float(window["battery_voltage_v"].median()),
    "current_a": float(window["battery_current_a"].median()),
    "temp_c": float(window["Tamb_archive_C"].median()),
    "soc_target": float(np.nanmedian(np.asarray(soc_targets, dtype=float))),
}

```

7.29 L1029 関数 terminal_metrics

- 行範囲: L1029–L1075
- 所属: module 直下
- 定義: `terminal_metrics(replay_df:pd.DataFrame,ocv_df:pd.DataFrame,base_model,batt:BatteryFitResult,anchor_km:float,*,terminal_anchor:dict|None=None)->Dict[str,float]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs, asarray, astype, copy, fillna, float, get, infer_soc_from_loaded_state, isfinite, isinstance, itertuples, len`
- 戻り値の要点: `{'retire_anchor_s_km': float(window['s_km'].median()), 'retire_anchor_voltage_obs_v': float(window['battery_voltage_v_obs'].median()), 'retire_anchor_voltage_pred_v': float(window['battery_voltage_v_pred'].median()), 'retire_anchor_soc_obs': soc_obs_value, 'retire_anchor_soc_pred': float(window['soc_pred'].median()), 'retire_anchor_soc_error': float(window['soc_pred'].median() - soc_obs_value)}`
- 主なローカル代入名: `row, soc_obs, soc_obs_value, terminal_anchor, valid, window`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `valid` に `np.isfinite(replay_df['battery_voltage_v_obs']) & np.isfinite(replay_df['battery_current_a_obs']) & np.isfinite(replay_df['Tamb_C']) & np.isfinite(replay_df['s_km']) & (replay_df['battery_voltage_v_obs'] >= INVALID_PACK_VOLTAGE_MIN_V) & ~replay_df['exclude_voltage_fit'].fillna(False).astype(bool)` の結果を代入する。
2. `window` に `replay_df.loc[valid & (np.abs(replay_df['s_km'] - float(anchor_km)) <= 1.5)].copy()` の結果を代入する。

3. 条件 `window.empty` を判定し、真なら内部処理を行う。
4. `window` に `replay_df.loc[valid].tail(6).copy()` の結果を代入する。
5. 上の条件が偽の場合:
6. `window` に `window.tail(min(6, len(window))).copy()` の結果を代入する。
7. `terminal_anchor` に `terminal_anchor if isinstance(terminal_anchor, dict) else {}` の結果を代入する。
8. 条件 `'soc_target' in terminal_anchor and terminal_anchor.get('soc_target') not in (None, '')` を判定し、真なら内部処理を行う。
9. `soc_obs_value` に `float(terminal_anchor['soc_target'])` の結果を代入する。
10. 上の条件が偽の場合:
11. `soc_obs` に `[infer_soc_from_loaded_state(float(row.battery_voltage_v_obs), float(row.battery_current_a_obs), float(row.Tamb_C), ocv_df, base_model, rint_scale=float(batt.rint_scale), r_line_ohm=float(batt.r_line_ohm)) for row in window.itertuples(index=False)]` の結果を代入する。
12. `soc_obs_value` に `float(np.nanmedian(np.asarray(soc_obs, dtype=float)))` の結果を代入する。
13. `{'retire_anchor_s_km': float(window['s_km'].median()), 'retire_anchor_voltage_obs_v': float(window['battery_voltage_obs_v'].median()), 'retire_anchor_voltage_pred_v': float(window['battery_voltage_v_pred'].median()), 'retire_anchor_soc_obs': soc_obs_value, 'retire_anchor_soc_pred': float(window['soc_pred'].median()), 'retire_anchor_soc_error': float(window['soc_pred'].median() - soc_obs_value)}` を返す。

代表コード断片

```
def terminal_metrics(
    replay_df: pd.DataFrame,
    ocv_df: pd.DataFrame,
    base_model,
    batt: BatteryFitResult,
    anchor_km: float,
    *,
    terminal_anchor: dict | None = None,
) -> Dict[str, float]:
    valid = (
        np.isfinite(replay_df["battery_voltage_v_obs"])
        & np.isfinite(replay_df["battery_current_a_obs"])
        & np.isfinite(replay_df["Tamb_C"])
        & np.isfinite(replay_df["s_km"])
        & (replay_df["battery_voltage_v_obs"] >= INVALID_PACK_VOLTAGE_MIN_V)
        & (~replay_df["exclude_voltage_fit"].fillna(False).astype(bool))
    )
    window = replay_df.loc[valid & (np.abs(replay_df["s_km"] - float(anchor_km)) <= 1.5)].copy()
    if window.empty:
        window = replay_df.loc[valid].tail(6).copy()
    else:
        window = window.tail(min(6, len(window))).copy()
    terminal_anchor = terminal_anchor if isinstance(terminal_anchor, dict) else {}
    if "soc_target" in terminal_anchor and terminal_anchor.get("soc_target") not in (None, ""):
        soc_obs_value = float(terminal_anchor["soc_target"])
    else:
        soc_obs = [
            infer_soc_from_loaded_state(
                float(row.battery_voltage_v_obs),
                float(row.battery_current_a_obs),
                float(row.Tamb_C),
                ocv_df,
```

```
base_model,
rint_scale=float(batt.rint_scale),
r_line_ohm=float(batt.r_line_ohm),
...
```

7.30 L1078 関数 `replay_day_metrics`

- 行範囲: L1078–L1096
- 所属: module 直下
- 定義: `replay_day_metrics(replay_df:pd.DataFrame)->list[dict]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `append`, `astype`, `fillna`, `float`, `groupby`, `int`, `len`, `max`, `mean`, `notna`, `sqrt`, `sum`
- 戻り値の要点: `rows`
- 主なローカル代入名: `day`, `group`, `power_mask`, `power_resid`, `rows`, `volt_mask`, `volt_resid`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `rows` に `[]` を代入する。
2. `replay_df.groupby('day', dropna=False)` を順に走査し、各要素を `(day, group)` に入れて処理する。
3. `power_mask` に `~group['exclude_power_fit']` の結果を代入する。
4. `volt_mask` に `~group['exclude_voltage_fit']` の結果を代入する。
5. `power_resid` に `group.loc[power_mask, 'battery_power_w_obs'] - group.loc[power_mask, 'battery_power_w_pred']` の結果を代入する。
6. `volt_resid` に `group.loc[volt_mask, 'battery_voltage_v_obs'] - group.loc[volt_mask, 'battery_voltage_v_pred']` の結果を代入する。
7. `rows.append(...)` を実行する。
8. `rows` を返す。

代表コード断片

```
def replay_day_metrics(replay_df: pd.DataFrame) -> list[dict]:
    rows: list[dict] = []
    for day, group in replay_df.groupby("day", dropna=False):
        power_mask = ~group["exclude_power_fit"]
        volt_mask = ~group["exclude_voltage_fit"]
        power_resid = group.loc[power_mask, "battery_power_w_obs"] - group.loc[power_mask, "battery_power_w_pred"]
        volt_resid = group.loc[volt_mask, "battery_voltage_v_obs"] - group.loc[volt_mask, "battery_voltage_v_pred"]
        rows.append(
            {
                "day": int(day) if pd.notna(day) else -1,
                "distance_end_km": float(pd.to_numeric(group["s_km"], errors="coerce").max()),
                "final_soc_pred": float(pd.to_numeric(group["soc_pred"], errors="coerce").iloc[-1]),
                "power_rmse_clean_w": float(np.sqrt(np.mean(power_resid.to_numpy(dtype=float) ** 2))) if len(power_resid) else float("nan"),
                "voltage_rmse_clean_v": float(np.sqrt(np.mean(volt_resid.to_numpy(dtype=float) ** 2))) if len(volt_resid) else float("nan"),
                "excluded_power_points": int(pd.to_numeric(group["exclude_power_fit"], errors="coerce").fillna(False).astype(bool).sum()),
                "excluded_voltage_points": int(pd.to_numeric(group["exclude_voltage_fit"], errors="coerce").fillna(False).astype(bool).sum()),
            }
        )
    return rows
```

7.31 L1099 関数 identification_selection_score

- 行範囲: L1099-L1132
- 所属: module 直下
- 定義: `identification_selection_score(metrics:Dict[str,float])->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `float`, `get`
- 戻り値の要点: `power_term + energy_term + independent_pv_term + vehicle_voltage_term + battery_voltage_term + battery_terminal_term + vehicle_terminal_term + end_to_end_terminal_term + fit_window_term`
- 主なローカル代入名: `battery_terminal_term`, `battery_voltage_term`, `end_to_end_terminal_term`, `energy_rmse_25km`, `energy_term`, `fit_window_term`, `independent_pv_term`, `power_rmse`, `power_term`, `robust_power_rmse`, `vehicle_terminal_term`, `vehicle_voltage_term`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. power_rmse に `float(metrics.get('power_rmse_clean_w', float('inf')))` の結果を代入する。
2. robust_power_rmse に `float(metrics.get('power_residual_mean_120s_rmse_w', power_rmse))` の結果を代入する。
3. energy_rmse_25km に `float(metrics.get('energy_error_25km_rmse_wh', float('inf')))` の結果を代入する。
4. power_term に `0.6 * power_rmse + 0.4 * robust_power_rmse` の結果を代入する。
5. energy_term に `0.5 * energy_rmse_25km` の結果を代入する。
6. independent_pv_term に `0.25 * float(metrics.get('end_to_end_moving_pv_rmse_w', float('inf')))` の結果を代入する。
7. vehicle_voltage_term に `8.0 * float(metrics.get('voltage_rmse_clean_v', float('inf')))` の結果を代入する。
8. battery_voltage_term に `4.0 * float(metrics.get('battery_conditional_voltage_rmse_clean_v', float('inf')))` の結果を代入する。
9. battery_terminal_term に `80.0 * abs(float(metrics.get('battery_conditional_retire_anchor_soc_error', float('inf'))))` の結果を代入する。
10. vehicle_terminal_term に `1000.0 * abs(float(metrics.get('retire_anchor_soc_error', float('inf'))))` の結果を代入する。
11. end_to_end_terminal_term に `300.0 * abs(float(metrics.get('end_to_end_retire_anchor_soc_error', float('inf'))))` の結果を代入する。
12. fit_window_term に `0.35 * float(metrics.get('power_rmse_fit_window_w', power_term))` の結果を代入する。
13. `power_term + energy_term + independent_pv_term + vehicle_voltage_term + battery_voltage_term + battery_terminal_term + vehicle_terminal_term + end_to_end_terminal_term + fit_window_term` を返す。

代表コード断片

```
def identification_selection_score(metrics: Dict[str, float]) -> float:
    power_rmse = float(metrics.get("power_rmse_clean_w", float("inf")))
    robust_power_rmse = float(metrics.get("power_residual_mean_120s_rmse_w", power_rmse))
    energy_rmse_25km = float(metrics.get("energy_error_25km_rmse_wh", float("inf")))
    power_term = 0.60 * power_rmse + 0.40 * robust_power_rmse
    energy_term = 0.50 * energy_rmse_25km
    independent_pv_term = 0.25 * float(
        metrics.get("end_to_end_moving_pv_rmse_w", float("inf"))
    )
    vehicle_voltage_term = 8.0 * float(metrics.get("voltage_rmse_clean_v", float("inf")))
    battery_voltage_term = 4.0 * float(
        metrics.get("battery_conditional_voltage_rmse_clean_v", float("inf"))
    )
    battery_terminal_term = 80.0 * abs(
        float(metrics.get("battery_conditional_retire_anchor_soc_error", float("inf")))
    )
    vehicle_terminal_term = 1000.0 * abs(
        float(metrics.get("retire_anchor_soc_error", float("inf")))
    )
    end_to_end_terminal_term = 300.0 * abs(
        float(metrics.get("end_to_end_retire_anchor_soc_error", float("inf")))
    )
    fit_window_term = 0.35 * float(metrics.get("power_rmse_fit_window_w", power_term))
    return (
        power_term
        + energy_term
        + independent_pv_term
```

```

+ vehicle_voltage_term
+ battery_voltage_term
+ battery_terminal_term
+ vehicle_terminal_term
+ end_to_end_terminal_term
+ fit_window_term
)

```

7.32 L1135 関数 `condition_vehicle_fit_on_measured_pv`

- 行範囲: L1135–L1152
- 所属: module 直下
- 定義: `condition_vehicle_fit_on_measured_pv(frame:pd.DataFrame)->pd.DataFrame`
- docstring: Use measured array power when identifying vehicle-side coefficients.
Forecast/PV-map error is validated separately. Feeding predicted PV into the vehicle fit otherwise lets a cloudy-day irradiance error masquerade as CdA, rolling resistance, or drivetrain-efficiency error.
- このブロックが直接呼ぶ主な関数・メソッド: `clip`, `copy`, `get`, `notna`, `to_numeric`, `where`
- 戻り値の要点: `out / out`
- 主なローカル代入名: `measured`, `out`, `predicted`, `usable`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `out` に `frame.copy()` の結果を代入する。
2. `predicted` に `pd.to_numeric(out.get('solar_power_w_model'), errors='coerce')` の結果を代入する。
3. `measured` に `pd.to_numeric(out.get('solar_power_w_obs'), errors='coerce')` の結果を代入する。
4. 条件 `predicted is None or measured is None` を判定し、真なら内部処理を行う。
5. `out` を返す。
6. `measured` に `measured.clip(lower=0.0)` の結果を代入する。
7. `usable` に `measured.notna()` の結果を代入する。
8. `out['solar_power_w_forecast_model']` に `predicted` の結果を代入する。
9. `out['solar_power_w_model']` に `predicted.where(~usable, measured)` の結果を代入する。
10. `out['vehicle_fit_solar_source']` に `np.where(usable, 'measured', 'forecast_fallback')` の結果を代入する。
11. `out` を返す。

代表コード断片

```
def condition_vehicle_fit_on_measured_pv(frame: pd.DataFrame) -> pd.DataFrame:
    """Use measured array power when identifying vehicle-side coefficients.

    Forecast/PV-map error is validated separately. Feeding predicted PV into the
    vehicle fit otherwise lets a cloudy-day irradiance error masquerade as CdA,
    rolling resistance, or drivetrain-efficiency error.
    """
    out = frame.copy()
    predicted = pd.to_numeric(out.get("solar_power_w_model"), errors="coerce")
    measured = pd.to_numeric(out.get("solar_power_w_obs"), errors="coerce")
    if predicted is None or measured is None:
        return out
    measured = measured.clip(lower=0.0)
    usable = measured.notna()
    out["solar_power_w_forecast_model"] = predicted
    out["solar_power_w_model"] = predicted.where(~usable, measured)
    out["vehicle_fit_solar_source"] = np.where(usable, "measured", "forecast_fallback")
    return out
```

7.33 L1155 関数 `calibrate_solar_measurement_to_pack`

- 行範囲: L1155–L1268
- 所属: module 直下
- 定義: `calibrate_solar_measurement_to_pack(frame:pd.DataFrame,*,known_aux_power_w:float,stopped_speed_kmh:float=1.0,minimum_solar_power_w:float=50.0,minimum_samples:int=500,gain_bounds:tuple[float,float]=(0.7,1.05))->tuple[pd.DataFrame,Dict[str,Any]]`
- docstring: Calibrate the ZP solar channel from stationary DC-bus power balance.
At zero wheel speed, the independently measured channels satisfy “ $P_{\text{batt}} = P_{\text{aux}} - \text{gain} * P_{\text{solar_raw}}$ “. The known 21 W auxiliary load anchors the intercept, so the fitted gain cannot absorb vehicle drag or forecast irradiance error.
- このブロックが直接呼ぶ主な関数・メソッド: `Series`, `abs`, `array`, `asarray`, `astype`, `bool`, `copy`, `fillna`, `float`, `get`, `groupby`, `int`
- 戻り値の要点: `(out, result)`
- 主なローカル代入名: `accepted`, `battery`, `candidate_gain`, `daily_gains`, `daily_std`, `daily_values`, `day_fit`, `day_value`, `diagnostic`, `excluded`, `fixed`, `free_intercept`, `gain`, `group`, `gx`, `gy`, `mask`, `out`, `raw`, `raw_column`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 5、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。

上から順の処理

1. out に `frame.copy()` の結果を代入する。
2. `raw_column` に `'solar_power_w_obs_raw'` if `'solar_power_w_obs_raw'` in `out.columns` else `'solar_power_w_obs'` の結果を代入する。
3. `raw` に `pd.to_numeric(out.get(raw_column), errors='coerce')` の結果を代入する。
4. `battery` に `pd.to_numeric(out.get('battery_power_w_obs'), errors='coerce')` の結果を代入する。
5. `speed` に `pd.to_numeric(out.get('speed_kmh'), errors='coerce')` の結果を代入する。
6. `excluded` に `pd.Series(False, index=out.index)` の結果を代入する。
7. 条件 `'exclude_power_fit'` in `out.columns` を判定し、真なら内部処理を行う。
8. `excluded` に `out['exclude_power_fit'].fillna(False).astype(bool)` の結果を代入する。
9. `mask` に `~excluded & raw.notna() & battery.notna() & speed.notna() & (speed <= float(stopped_speed_kmh)) & (raw >= float(minimum_solar_power_w))` の結果を代入する。
10. `x` に `raw.loc[mask].to_numpy(dtype=float)` の結果を代入する。
11. `y` に `battery.loc[mask].to_numpy(dtype=float)` の結果を代入する。
12. `result` に `{'method': 'stationary DC-bus balance P_batt=P_aux-gain*P_solar_raw with robust Huber loss', 'known_aux_power_w': float(known_aux_power_w), 'sample_count': int(len(x)), 'minimum_samples': int(minimum_samples), 'gain_bounds': [float(gain_bounds[0]), float(gain_bounds[1])], 'accepted': False, 'gain_to_pack': 1.0, 'raw_column': raw_column, 'corrected_column': 'solar_power_w_obs'}` を代入する。
13. `gain` に 1.0 の結果を代入する。
14. 条件 `len(x) >= int(minimum_samples)` を判定し、真なら内部処理を行う。
15. `fixed` に `least_squares(lambda theta: y - (float(known_aux_power_w) - float(theta[0]) * x), x0=np.array([0.93], dtype=float), bounds=(np.array([gain_bounds[0]]), np.array([gain_bounds[1]])), loss='huber', f_scale=20.0)` の結果を代入する。
16. `diagnostic` に `least_squares(lambda theta: y - (float(theta[0]) - float(theta[1]) * x), x0=np.array([float(known_aux_power_w) - float(fixed.x[0])], dtype=float), bounds=(np.array([0.0, gain_bounds[0]], dtype=float), np.array([100.0, gain_bounds[1]], dtype=float)), loss='huber', f_scale=20.0)` の結果を代入する。
17. `candidate_gain` に `float(fixed.x[0])` の結果を代入する。
18. `residual` に `y - (float(known_aux_power_w) - candidate_gain * x)` の結果を代入する。
19. `daily_gains` に `{}` を代入する。
20. 条件 `'day'` in `out.columns` を判定し、真なら内部処理を行う。
21. `out.loc[mask].groupby('day', dropna=False)` を順に走査し、各要素を `(day_value, group)` に入れて処理する。
22. `gx` に `pd.to_numeric(group[raw_column], errors='coerce').to_numpy(dtype=float)` の結果を代入する。
23. `gy` に `pd.to_numeric(group['battery_power_w_obs'], errors='coerce').to_numpy(dtype=float)` の結果を代入する。
24. 条件 `len(gx) < 50` を判定し、真なら内部処理を行う。
25. `day_fit` に `least_squares(lambda theta: gy - (float(known_aux_power_w) - float(theta[0]) * gx), x0=np.array([candidate_gain], dtype=float), bounds=(np.array([gain_bounds[0]]), np.array([gain_bounds[1]])), loss='huber', f_scale=20.0)` の結果を代入する。

26. `daily_gains[str(day_value)]` に `float(day_fit.x[0])` の結果を代入する。
27. `daily_values` に `np.asarray(list(daily_gains.values()), dtype=float)` の結果を代入する。
28. `daily_std` に `float(np.std(daily_values, ddof=1)) if len(daily_values) >= 2 else float('nan')` の結果を代入する。
29. `free_intercept` に `float(diagnostic.x[0])` の結果を代入する。
30. `accepted` に `bool(np.isfinite(candidate_gain) and float(gain_bounds[0]) < candidate_gain < float(gain_bounds[1]) and (abs(free_intercept - float(known_aux_power_w)) <= 5.0) and (not np.isfinite(daily_std) or daily_std <= 0.02))` の結果を代入する。
31. `result.update(...)` を実行する。
32. 条件 `accepted` を判定し、真なら内部処理を行う。
33. `gain` に `candidate_gain` の結果を代入する。
34. `out['solar_power_w_obs_raw']` に `raw` の結果を代入する。
35. `out['solar_measurement_gain_to_pack']` に `float(gain)` の結果を代入する。
36. `out['solar_power_w_obs']` に `raw * float(gain)` の結果を代入する。
37. `(out, result)` を返す。

代表コード断片

```
def calibrate_solar_measurement_to_pack(
    frame: pd.DataFrame,
    *,
    known_aux_power_w: float,
    stopped_speed_kmh: float = 1.0,
    minimum_solar_power_w: float = 50.0,
    minimum_samples: int = 500,
    gain_bounds: tuple[float, float] = (0.70, 1.05),
) -> tuple[pd.DataFrame, Dict[str, Any]]:
    """Calibrate the ZP solar channel from stationary DC-bus power balance.

    At zero wheel speed, the independently measured channels satisfy
    ``P_batt = P_aux - gain * P_solar_raw``. The known 21 W auxiliary load
    anchors the intercept, so the fitted gain cannot absorb vehicle drag or
    forecast irradiance error.
    """
    out = frame.copy()
    raw_column = (
        "solar_power_w_obs_raw"
        if "solar_power_w_obs_raw" in out.columns
        else "solar_power_w_obs"
    )
    raw = pd.to_numeric(out.get(raw_column), errors="coerce")
    battery = pd.to_numeric(out.get("battery_power_w_obs"), errors="coerce")
    speed = pd.to_numeric(out.get("speed_kmh"), errors="coerce")
    excluded = pd.Series(False, index=out.index)
    if "exclude_power_fit" in out.columns:
        excluded = out["exclude_power_fit"].fillna(False).astype(bool)
    mask = (
        (~excluded)
        & raw.notna()
        & battery.notna()
        & speed.notna()
        & (speed <= float(stopped_speed_kmh))
        & (raw >= float(minimum_solar_power_w))
    )
    ...
```


7.34 L1271 関数 `_shift_acceleration_within_segments`

- 行範囲: L1271–L1280
- 所属: module 直下
- 定義: `_shift_acceleration_within_segments(frame:pd.DataFrame,sample_shift:int,acceleration:pd.Series|None=None)->pd.Series`
- docstring: Shift a derived acceleration trace without leaking across race days or log gaps.
- このブロックが直接呼ぶ主な関数・メソッド: `Series, cumsum, groupby, int, replay_segment_start_mask, shift, to_numeric`
- 戻り値の要点: `acceleration.groupby(segment_id).shift(int(sample_shift))`
- 主なローカル代入名: `acceleration, segment_id`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `acceleration is None` を判定し、真なら内部処理を行う。
2. `acceleration` に `pd.to_numeric(frame['accel_ms2'], errors='coerce')` の結果を代入する。
3. `segment_id` に `pd.Series(replay_segment_start_mask(frame), index=frame.index).cumsum()` の結果を代入する。
4. `acceleration.groupby(segment_id).shift(int(sample_shift))` を返す。

代表コード断片

```
def _shift_acceleration_within_segments(
    frame: pd.DataFrame,
    sample_shift: int,
    acceleration: pd.Series | None = None,
) -> pd.Series:
    """Shift a derived acceleration trace without leaking across race days or log gaps."""
    if acceleration is None:
        acceleration = pd.to_numeric(frame["accel_ms2"], errors="coerce")
    segment_id = pd.Series(replay_segment_start_mask(frame), index=frame.index).cumsum()
    return acceleration.groupby(segment_id).shift(int(sample_shift))
```

7.35 L1283 関数 `_acceleration_trace_from_speed`

- 行範囲: L1283–L1318
- 所属: module 直下
- 定義: `_acceleration_trace_from_speed(frame:pd.DataFrame,*,method:str>window_samples:int)->pd.Series`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Series`, `ValueError`, `clip`, `cumsum`, `diff`, `fillna`, `get`, `groupby`, `int`, `len`, `lower`, `max`
- 戻り値の要点: `smoothed.fillna(0.0).clip(lower=-1.5, upper=1.5)`
- 主なローカル代入名: `_`, `dt_sec`, `group`, `raw`, `segment_id`, `segment_start`, `smoothed`, `speed_ms`, `usable_window`, `window`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError(f'unsupported acceleration filter method: {method}')`
- 制御構造の規模: 条件分岐 4、ループ 1、try 0

この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `speed_ms` に `pd.to_numeric(frame['speed_kmh'], errors='coerce').fillna(0.0) / 3.6` の結果を代入する。
2. `dt_sec` に `pd.to_numeric(frame.get('dt_sec'), errors='coerce').replace(0.0, np.nan)` の結果を代入する。
3. `raw` に `speed_ms.diff() / dt_sec` の結果を代入する。
4. `segment_start` に `pd.Series(replay_segment_start_mask(frame), index=frame.index)` の結果を代入する。
5. `raw.loc[segment_start]` に `0.0` の結果を代入する。
6. `raw` に `raw.replace([np.inf, -np.inf], np.nan).fillna(0.0)` の結果を代入する。
7. `segment_id` に `segment_start.cumsum()` の結果を代入する。
8. `window` に `max(1, int(window_samples))` の結果を代入する。
9. 条件 `window % 2 == 0` を判定し、真なら内部処理を行う。
10. `window` を `Add` で更新する。
11. `smoothed` に `pd.Series(index=frame.index, dtype=float)` の結果を代入する。
12. `raw.groupby(segment_id)` を順に走査し、各要素を `(_, group)` に入れて処理する。
13. 条件 `str(method).lower() == 'savgol'` を判定し、真なら内部処理を行う。
14. `usable_window` に `min(window, len(group) if len(group) % 2 else len(group) - 1)` の結果を代入する。
15. 条件 `usable_window >= 5` を判定し、真なら内部処理を行う。

16. `smoothed.loc[group.index]` に `savgol_filter(group.to_numpy(dtype=float), usable_window, min(2, usable_window - 1), mode='interp')` の結果を代入する。
17. 上の条件が偽の場合:
18. `smoothed.loc[group.index]` に `group` の結果を代入する。
19. 上の条件が偽の場合:
20. 条件 `str(method).lower() == 'median'` を判定し、真なら内部処理を行う。
21. `smoothed.loc[group.index]` に `group.rolling(window, center=True, min_periods=1).median()` の結果を代入する。
22. 上の条件が偽の場合:
23. `ValueError(f'unsupported acceleration filter method: {method}')` を送出する。
24. `smoothed.fillna(0.0).clip(lower=-1.5, upper=1.5)` を返す。

代表コード断片

```
def _acceleration_trace_from_speed(
    frame: pd.DataFrame,
    *,
    method: str,
    window_samples: int,
) -> pd.Series:
    speed_ms = pd.to_numeric(frame["speed_kmh"], errors="coerce").fillna(0.0) / 3.6
    dt_sec = pd.to_numeric(frame.get("dt_sec"), errors="coerce").replace(0.0, np.nan)
    raw = speed_ms.diff() / dt_sec
    segment_start = pd.Series(replay_segment_start_mask(frame), index=frame.index)
    raw.loc[segment_start] = 0.0
    raw = raw.replace([np.inf, -np.inf], np.nan).fillna(0.0)
    segment_id = segment_start.cumsum()
    window = max(1, int(window_samples))
    if window % 2 == 0:
        window += 1
    smoothed = pd.Series(index=frame.index, dtype=float)
    for _, group in raw.groupby(segment_id):
        if str(method).lower() == "savgol":
            usable_window = min(window, len(group) if len(group) % 2 else len(group) - 1)
            if usable_window >= 5:
                smoothed.loc[group.index] = savgol_filter(
                    group.to_numpy(dtype=float),
                    usable_window,
                    min(2, usable_window - 1),
                    mode="interp",
                )
            else:
                smoothed.loc[group.index] = group
        elif str(method).lower() == "median":
            smoothed.loc[group.index] = group.rolling(
                window, center=True, min_periods=1
            ).median()
        else:
            raise ValueError(f"unsupported acceleration filter method: {method}")
    ...
```

7.36 L1321 関数 `_bilinear_interp_array`

- 行範囲: L1321–L1338

- 所属: module 直下
- 定義: `_bilinear_interp_array(x_grid, y_grid, values, x, y) -> np.ndarray`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `asarray`, `clip`, `divide`, `len`, `searchsorted`, `zeros_like`
- 戻り値の要点: $(1.0 - wx) * (1.0 - wy) * \text{values}[\text{ix}, \text{iy}] + wx * (1.0 - wy) * \text{values}[\text{ix} + 1, \text{iy}] + (1.0 - wx) * wy * \text{values}[\text{ix}, \text{iy} + 1] + wx * wy * \text{values}[\text{ix} + 1, \text{iy} + 1]$
- 主なローカル代入名: `ix`, `iy`, `values`, `wx`, `wy`, `x`, `x0`, `x1`, `x_grid`, `y`, `y0`, `y1`, `y_grid`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `x_grid` に `np.asarray(x_grid, dtype=float)` の結果を代入する。
2. `y_grid` に `np.asarray(y_grid, dtype=float)` の結果を代入する。
3. `values` に `np.asarray(values, dtype=float)` の結果を代入する。
4. `x` に `np.clip(np.asarray(x, dtype=float), x_grid[0], x_grid[-1])` の結果を代入する。
5. `y` に `np.clip(np.asarray(y, dtype=float), y_grid[0], y_grid[-1])` の結果を代入する。
6. `ix` に `np.clip(np.searchsorted(x_grid, x) - 1, 0, len(x_grid) - 2)` の結果を代入する。
7. `iy` に `np.clip(np.searchsorted(y_grid, y) - 1, 0, len(y_grid) - 2)` の結果を代入する。
8. `(x0, x1)` に `(x_grid[ix], x_grid[ix + 1])` の結果を代入する。
9. `(y0, y1)` に `(y_grid[iy], y_grid[iy + 1])` の結果を代入する。
10. `wx` に `np.divide(x - x0, x1 - x0, out=np.zeros_like(x), where=x1 != x0)` の結果を代入する。
11. `wy` に `np.divide(y - y0, y1 - y0, out=np.zeros_like(y), where=y1 != y0)` の結果を代入する。
12. $(1.0 - wx) * (1.0 - wy) * \text{values}[\text{ix}, \text{iy}] + wx * (1.0 - wy) * \text{values}[\text{ix} + 1, \text{iy}] + (1.0 - wx) * wy * \text{values}[\text{ix}, \text{iy} + 1] + wx * wy * \text{values}[\text{ix} + 1, \text{iy} + 1]$ を返す。

代表コード断片

```
def _bilinear_interp_array(x_grid, y_grid, values, x, y) -> np.ndarray:
    x_grid = np.asarray(x_grid, dtype=float)
    y_grid = np.asarray(y_grid, dtype=float)
    values = np.asarray(values, dtype=float)
    x = np.clip(np.asarray(x, dtype=float), x_grid[0], x_grid[-1])
    y = np.clip(np.asarray(y, dtype=float), y_grid[0], y_grid[-1])
    ix = np.clip(np.searchsorted(x_grid, x) - 1, 0, len(x_grid) - 2)
    iy = np.clip(np.searchsorted(y_grid, y) - 1, 0, len(y_grid) - 2)
    x0, x1 = x_grid[ix], x_grid[ix + 1]
    y0, y1 = y_grid[iy], y_grid[iy + 1]
    wx = np.divide(x - x0, x1 - x0, out=np.zeros_like(x), where=x1 != x0)
    wy = np.divide(y - y0, y1 - y0, out=np.zeros_like(y), where=y1 != y0)
    return (
        (1.0 - wx) * (1.0 - wy) * values[ix, iy]
        + wx * (1.0 - wy) * values[ix + 1, iy]
        + (1.0 - wx) * wy * values[ix, iy + 1]
        + wx * wy * values[ix + 1, iy + 1]
    )
```

7.37 L1341 関数 `_map_efficiency_array`

- 行範囲: L1341–L1361
- 所属: module 直下
- 定義: `_map_efficiency_array(base_model, maps, speed_ms, torque_nm, *, regen:bool) -> np.ndarray`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_bilinear_interp_array`, `clip`, `empty`, `float`, `full`, `get`, `len`, `lower`, `str`
- 戻り値の要点: `np.clip(result, 0.4 if regen else 0.55, 0.95 if regen else 0.99)`
- 主なローカル代入名: `eco_max`, `grid`, `key`, `mode`, `result`, `selected`, `selected_power`, `use_power`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 1、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `mode` に `str(base_model.drive_mode or 'default').lower()` の結果を代入する。
2. 条件 `mode in {'eco', 'power'}` を判定し、真なら内部処理を行う。
3. `selected_power` に `np.full(len(speed_ms), mode == 'power', dtype=bool)` の結果を代入する。
4. 上の条件が偽の場合:
5. `eco_max` に `base_model.tau_max.get('eco', base_model.tau_max.get('default', 0.0))` の結果を代入する。

6. selected_power に torque_nm > float(eco_max) + float(base_model.drive_mode_tau_margin) の結果を代入する。
7. result に np.empty(len(speed_ms), dtype=float) の結果を代入する。
8. (False, True) を順に走査し、各要素を use_power に入れて処理する。
9. selected に selected_power == use_power の結果を代入する。
10. key に 'power' if use_power else 'eco' の結果を代入する。
11. grid に maps.get(key, maps['default']) の結果を代入する。
12. result[selected] に _bilinear_interp_array(*grid, speed_ms[selected], torque_nm[selected]) の結果を代入する。
13. np.clip(result, 0.4 if regen else 0.55, 0.95 if regen else 0.99) を返す。

代表コード断片

```
def _map_efficiency_array(base_model, maps, speed_ms, torque_nm, *, regen: bool) -> np.ndarray:
    mode = str(base_model.drive_mode or "default").lower()
    if mode in {"eco", "power"}:
        selected_power = np.full(len(speed_ms), mode == "power", dtype=bool)
    else:
        eco_max = base_model.tau_max.get("eco", base_model.tau_max.get("default", 0.0))
        selected_power = torque_nm > (
            float(eco_max) + float(base_model.drive_mode_tau_margin)
        )
    result = np.empty(len(speed_ms), dtype=float)
    for use_power in (False, True):
        selected = selected_power == use_power
        key = "power" if use_power else "eco"
        grid = maps.get(key, maps["default"])
        result[selected] = _bilinear_interp_array(
            *grid, speed_ms[selected], torque_nm[selected]
        )
    # The candidate scale is applied by the caller. Inheriting the scale from
    # the input profile here would apply old_scale * candidate_scale during
    # fitting, while the generated profile contains candidate_scale only.
    return np.clip(result, 0.40 if regen else 0.55, 0.95 if regen else 0.99)
```

7.38 L1364 関数 _motion_predictions_for_acceleration

- 行範囲: L1364–L1428
- 所属: module 直下
- 定義: `_motion_predictions_for_acceleration(frame:pd.DataFrame,acceleration:pd.Series,base_model,mot:MotionFitResult)->np.ndarray`
- docstring: Vectorized equivalent of motion_power_prediction for filter/lag search.
- このブロックが直接呼ぶ主な関数・メソッド: Series, _map_efficiency_array, abs, arctan, clip, cos, empty, fillna, float, full, len, lower
- 戻り値の要点: electrical + float(mot.p_aux_w) - solar
- 主なローカル代入名: air_density, altitude, ambient, ambient_source, drive_eff, electrical, elevation, elevation_source, force, headwind, normal_force, omega_wheel, p_mech, positive, pressure, pressure_ratio, regen_eff, relative_speed, slope, solar

- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `speed_ms` に `pd.to_numeric(frame['speed_kmh'], errors='coerce').fillna(0.0).to_numpy() / 3.6` の結果を代入する。
2. `slope` に `pd.to_numeric(frame['slope_pct'], errors='coerce').fillna(0.0).to_numpy()` の結果を代入する。
3. `headwind` に `pd.to_numeric(frame['headwind_archive_ms'], errors='coerce').fillna(0.0).to_numpy() * float(mot.headwind_gain)` の結果を代入する。
4. `solar` に `pd.to_numeric(frame['solar_power_w_obs'], errors='coerce').fillna(0.0).to_numpy()` の結果を代入する。
5. `theta` に `np.arctan(slope * float(mot.grade_scale) / 100.0)` の結果を代入する。
6. `relative_speed` に `np.maximum(0.0, speed_ms + headwind)` の結果を代入する。
7. `ambient_source` に `frame['Tamb_archive_C'] if 'Tamb_archive_C' in frame else pd.Series(25.0, index=frame.index)` の結果を代入する。
8. `elevation_source` に `frame['alt_m'] if 'alt_m' in frame else pd.Series(0.0, index=frame.index)` の結果を代入する。
9. `ambient` に `pd.to_numeric(ambient_source, errors='coerce').fillna(25.0).to_numpy()` の結果を代入する。
10. `elevation` に `pd.to_numeric(elevation_source, errors='coerce').fillna(0.0).to_numpy()` の結果を代入する。
11. 条件 `str(base_model.p.air_density_mode or 'constant').lower() == 'ideal_gas_altitude'` を判定し、真なら内部処理を行う。
12. `altitude` に `np.clip(elevation, -500.0, 11000.0)` の結果を代入する。
13. `pressure_ratio` に `np.maximum(0.05, 1.0 - 0.0065 * altitude / 288.15) ** 5.255877` の結果を代入する。
14. `pressure` に `float(base_model.p.air_density_reference_pressure_pa) * pressure_ratio` の結果を代入する。
15. `air_density` に `pressure / (287.05 * np.maximum(180.0, ambient + 273.15))` の結果を代入する。
16. 上の条件が偽の場合:
17. `air_density` に `np.full(len(frame), float(base_model.p.rho), dtype=float)` の結果を代入する。
18. `normal_force` に `base_model.p.m * base_model.p.g * np.cos(theta)` の結果を代入する。
19. `force` に `0.5 * air_density * float(mot.cda) * relative_speed ** 2 + float(mot.crr) * normal_force + base_model.p.m * base_model.p.g * np.sin(theta)` の結果を代入する。
20. `p_mech` に `force * speed_ms + base_model.p.m * acceleration.fillna(0.0).to_numpy(dtype=float) * speed_ms` の結果を代入する。

21. `omega_wheel` に `speed_ms / float(base_model.p.wheel_radius)` の結果を代入する。
22. `torque` に `np.abs(p_mech) / (omega_wheel + 0.001) / float(base_model.p.gear_ratio)` の結果を代入する。
23. `drive_eff` に `_map_efficiency_array(base_model, base_model.maps_drive, speed_ms, torque, regen=False)` の結果を代入する。
24. `drive_eff` に `np.clip(drive_eff * float(mot.drive_eff_scale), 0.55, 0.99)` の結果を代入する。
25. `regen_eff` に `_map_efficiency_array(base_model, base_model.maps_regen, speed_ms, torque, regen=True)` の結果を代入する。
26. `regen_eff` に `np.clip(regen_eff * float(mot.drive_eff_scale), 0.4, 0.95)` の結果を代入する。
27. `positive` に `p_mech >= 0.0` の結果を代入する。
28. `electrical` に `np.empty(len(frame), dtype=float)` の結果を代入する。
29. `electrical[positive]` に `p_mech[positive] / np.maximum(drive_eff[positive] * base_model.p.gear_eta * base_model.p.inverter_eta, 1e-06)` の結果を代入する。
30. `electrical[~positive]` に `-float(np.clip(mot.regen_utilization, 0.0, 1.0)) * regen_eff[~positive] * base_model.p.gear_eta * base_model.p.inverter_eta * np.abs(p_mech[~positive])` の結果を代入する。
31. `electrical + float(mot.p_aux_w) - solar` を返す。

代表コード断片

```
def _motion_predictions_for_acceleration(
    frame: pd.DataFrame,
    acceleration: pd.Series,
    base_model,
    mot: MotionFitResult,
) -> np.ndarray:
    """Vectorized equivalent of motion_power_prediction for filter/lag search."""
    speed_ms = (
        pd.to_numeric(frame["speed_kmh"], errors="coerce").fillna(0.0).to_numpy()
        / 3.6
    )
    slope = pd.to_numeric(frame["slope_pct"], errors="coerce").fillna(0.0).to_numpy()
    headwind = (
        pd.to_numeric(frame["headwind_archive_ms"], errors="coerce").fillna(0.0).to_numpy()
        * float(mot.headwind_gain)
    )
    solar = (
        pd.to_numeric(frame["solar_power_w_obs"], errors="coerce").fillna(0.0).to_numpy()
    )
    theta = np.arctan((slope * float(mot.grade_scale)) / 100.0)
    relative_speed = np.maximum(0.0, speed_ms + headwind)
    ambient_source = frame["Tamb_archive_C"] if "Tamb_archive_C" in frame else pd.Series(
        25.0, index=frame.index
    )
    elevation_source = frame["alt_m"] if "alt_m" in frame else pd.Series(0.0, index=frame.index)
    ambient = pd.to_numeric(ambient_source, errors="coerce").fillna(25.0).to_numpy()
    elevation = pd.to_numeric(elevation_source, errors="coerce").fillna(0.0).to_numpy()
    if str(base_model.p.air_density_mode or "constant").lower() == "ideal_gas_altitude":
        altitude = np.clip(elevation, -500.0, 11000.0)
        pressure_ratio = np.maximum(0.05, 1.0 - 0.0065 * altitude / 288.15) ** 5.255877
        pressure = float(base_model.p.air_density_reference_pressure_pa) * pressure_ratio
        air_density = pressure / (287.05 * np.maximum(180.0, ambient + 273.15))
    else:
        air_density = np.full(len(frame), float(base_model.p.rho), dtype=float)
    normal_force = base_model.p.m * base_model.p.g * np.cos(theta)
    force = (
```



```
0.5 * air_density * float(mot.cda) * relative_speed**2
```

```
...
```

7.39 L1431 関数 `fit_acceleration_timestamp_alignment`

- 行範囲: L1431–L1628
- 所属: module 直下
- 定義: `fit_acceleration_timestamp_alignment(frame:pd.DataFrame,base_model,mot:MotionFitResult,options:dict|None=None)->tuple[pd.DataFrame,Dict[str,Any]]`
- docstring: Identify the filter and timestamp offset of GPS-derived acceleration.
Vehicle mass remains fixed. Candidate observation filters and offsets are fitted on all but the last race day and adopted only when the held-out last day does not regress. This models quantized/asynchronous GNSS observations; it is not a tunable vehicle force or a live-command filter.
- このブロックが直接呼ぶ主な関数・メソッド: `_acceleration_trace_from_speed`, `_motion_predictions_for_acceleration`, `_shift_acceleration_within_segments`, `abs`, `any`, `append`, `astype`, `bool`, `clip`, `copy`, `fillna`, `float`
- 戻り値の要点: (out, {'enabled': True, 'adopted': adopted, 'method': 'fixed-mass GNSS acceleration filter/lag selection with last-race-day holdout', 'sample_period_sec': sample_period_sec, 'selected_filter_method': selected_method, 'selected_filter_window_samples': selected_window, 'selected_filter_window_sec': selected_window * sample_period_sec, 'selected_lag_sec': selected_lag, 'lag_search_min_sec': lag_search_min, 'lag_search_max_sec': lag_search_max, 'lag_search_boundary_hit': lag_boundary_hit, 'holdout_day': holdout_day, 'training_sample_count': int(train_mask.sum()), 'validation_sample_count': int(validation_mask.sum()), 'baseline_training_rmse_w': float(baseline['training_rmse_w']), 'selected_training_rmse_w': float(selected['training_rmse_w']), 'baseline_validation_rmse_w': float(baseline['validation_rmse_w']), 'selected_validation_rmse_w': float(selected['validation_rmse_w']), 'training_improvement_w': float(float(baseline['training_rmse_w']) - float(selected['training_rmse_w'])), 'validation_rmse_ratio': validation_ratio if adopted else 1.0, 'candidates': records}) / (frame.copy(), {'enabled': enabled, 'adopted': False, 'reason': 'disabled_or_missing_columns'}) / (out, {'enabled': True, 'adopted': False, 'reason': 'insufficient_training_samples', 'training_sample_count': int(train_mask.sum()), 'validation_sample_count': int(validation_mask.sum())}) / float(np.sqrt(np.mean(np.square(observed[use] - predicted[use])))) if use.any() else float('nan')
- 主なローカル代入名: `actual_lag_sec`, `adopted`, `aligned`, `base_valid`, `baseline`, `best`, `candidate_filters`, `candidate_lag_sec`, `cfg`, `configured_holdout_day`, `day_values`, `dt_sec`, `enabled`, `enough_validation`, `filter_method`, `filter_window`, `filtered`, `finite_days`, `finite_records`, `holdout_day`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 5、ループ 3、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `cfg` に `options if isinstance(options, dict) else {}` の結果を代入する。
2. `enabled` に `bool(cfg.get('enabled', True))` の結果を代入する。
3. `required` に `{'time_utc', 'day', 'speed_kmh', 'slope_pct', 'headwind_archive_ms', 'solar_power_w_obs', 'battery_power_w_obs', 'exclude_power_fit', 'accel_ms2'}` の結果を代入する。
4. 条件 `not enabled or not required.issubset(frame.columns)` を判定し、真なら内部処理を行う。
5. `(frame.copy(), {'enabled': enabled, 'adopted': False, 'reason': 'disabled_or_missing_columns'})` を返す。
6. `out` に `frame.copy()` の結果を代入する。
7. `out['accel_ms2_previous']` に `pd.to_numeric(out['accel_ms2'], errors='coerce')` の結果を代入する。
8. `out['accel_ms2_raw']` に `_acceleration_trace_from_speed(out, method='median', window_samples=1)` の結果を代入する。
9. `dt_sec` に `pd.to_numeric(out.get('dt_sec'), errors='coerce')` の結果を代入する。
10. `usable_dt` に `dt_sec[np.isfinite(dt_sec) & (dt_sec > 0.0) & (dt_sec <= 60.0)]` の結果を代入する。
11. `sample_period_sec` に `float(np.median(usable_dt)) if len(usable_dt) else 5.0` の結果を代入する。
12. `raw_candidates` に `cfg.get('candidate_lag_sec', [-30.0, -20.0, -15.0, -10.0, -5.0, 0.0, 5.0, 10.0, 15.0, 20.0, 30.0])` の結果を代入する。
13. `candidate_lag_sec` に `sorted(set((float(value) for value in raw_candidates if np.isfinite(float(value)))) | {0.0})` の結果を代入する。
14. `raw_filters` に `cfg.get('candidate_filters', [{ 'method': 'median', 'window_samples': 5}, { 'method': 'median', 'window_samples': 9}, { 'method': 'median', 'window_samples': 15}, { 'method': 'savgol', 'window_samples': 11}, { 'method': 'savgol', 'window_samples': 15}, { 'method': 'savgol', 'window_samples': 21}])` の結果を代入する。
15. `candidate_filters` に `[]` を代入する。
16. `raw_filters` を順に走査し、各要素を `item` に入れて処理する。
17. 条件 `isinstance(item, dict)` を判定し、真なら内部処理を行う。
18. `method` に `str(item.get('method', 'median')).strip().lower()` の結果を代入する。
19. `window` に `int(item.get('window_samples', 5))` の結果を代入する。
20. 上の条件が偽の場合:
21. 条件 `isinstance(item, (list, tuple)) and len(item) >= 2` を判定し、真なら内部処理を行う。
22. `(method, window)` に `(str(item[0]).strip().lower(), int(item[1]))` の結果を代入する。
23. 上の条件が偽の場合:
24. `Continue` 文を実行する。
25. 条件 `method in {'median', 'savgol'}` を判定し、真なら内部処理を行う。
26. `candidate_filters.append(...)` を実行する。
27. `candidate_filters` に `list(dict.fromkeys(candidate_filters + [('median', 5)]))` の結果を代入する。
28. `day_values` に `pd.to_numeric(out['day'], errors='coerce')` の結果を代入する。

29. `finite_days` に `sorted(set((int(value) for value in day_values[np.isfinite(day_values)])))` の結果を代入する。
30. `configured_holdout_day` に `int(cfg.get('holdout_day', 0) or 0)` の結果を代入する。
31. `holdout_day` に `configured_holdout_day if configured_holdout_day > 0 else int(finite_days[-1] if finite_days else 0)` の結果を代入する。
32. `base_valid` に `~out['exclude_power_fit'].fillna(True).astype(bool) & (pd.to_numeric(out['speed_kmh'], errors='coerce') >= float(cfg.get('min_speed_kmh', 12.0))) & np.isfinite(pd.to_numeric(out['battery_power_w_obs'], errors='coerce')) & np.isfinite(pd.to_numeric(out['solar_power_w_obs'], errors='coerce'))` の結果を代入する。
33. `train_mask` に `base_valid & (day_values != holdout_day)` の結果を代入する。
34. `validation_mask` に `base_valid & (day_values == holdout_day)` の結果を代入する。
35. 条件 `int(train_mask.sum()) < int(cfg.get('minimum_train_samples', 500))` を判定し、真なら内部処理を行う。
36. `(out, {'enabled': True, 'adopted': False, 'reason': 'insufficient_training_samples', 'training_sample_count': int(train_mask.sum()), 'validation_sample_count': int(validation_mask.sum())})` を返す。
37. `observed` に `pd.to_numeric(out['battery_power_w_obs'], errors='coerce').to_numpy(dtype=float)` の結果を代入する。
38. `records` に `[]` を代入する。
39. `traces` に `{}` を代入する。
40. `candidate_filters` を順に走査し、各要素を `(filter_method, filter_window)` に入れて処理する。
41. `filtered` に `_acceleration_trace_from_speed(out, method=filter_method, window_samples=filter_window)` の結果を代入する。
42. `candidate_lag_sec` を順に走査し、各要素を `lag_sec` に入れて処理する。
43. `sample_shift` に `int(round(float(lag_sec) / sample_period_sec))` の結果を代入する。
44. `actual_lag_sec` に `float(sample_shift * sample_period_sec)` の結果を代入する。
45. `aligned` に `_shift_acceleration_within_segments(out, sample_shift, acceleration=filtered)` の結果を代入する。
46. `predicted` に `_motion_predictions_for_acceleration(out, aligned, base_model, mot)` の結果を代入する。
47. 関数 `rmse` を定義する。
48. `records.append(...)` を実行する。
49. `traces[filter_method, int(filter_window), actual_lag_sec]` に `aligned` の結果を代入する。
50. `baseline` に `next((row for row in records if row['filter_method'] == 'median' and int(row['filter_window_samples']) == 5 and (abs(float(row['lag_sec'])) < 0.5 * sample_period_sec)), min(records, key=lambda row: abs(float(row['lag_sec']))))` の結果を代入する。
51. `finite_records` に `[row for row in records if np.isfinite(float(row['training_rmse_w']))]` の結果を代入する。
52. `best` に `min(finite_records, key=lambda row: float(row['training_rmse_w']))` の結果を代入する。
53. `train_improvement` に `float(baseline['training_rmse_w']) - float(best['training_rmse_w'])` の結果を代入する。
54. `validation_ratio` に `float(best['validation_rmse_w']) / max(float(baseline['validation_rmse_w']), 1e-09) if np.isfinite(float(best['validation_rmse_w'])) and np.isfinite(float(baseline['validation_rmse_w'])) else float('nan')` の結果を代入する。

55. `enough_validation` に `int(validation_mask.sum()) >= int(cfg.get('minimum_validation_samples', 100))` の結果を代入する。
56. `adopted` に `bool(train_improvement >= float(cfg.get('minimum_training_improvement_w', 2.0)) and (not enough_validation or (np.isfinite(validation_ratio) and validation_ratio <= float(cfg.get('maximum_validation_rmse_ratio', 1.02)))))` の結果を代入する。
57. `selected` に `best if adopted else baseline` の結果を代入する。
58. `selected_lag` に `float(selected['lag_sec'])` の結果を代入する。
59. `selected_method` に `str(selected['filter_method'])` の結果を代入する。
60. `selected_window` に `int(selected['filter_window_samples'])` の結果を代入する。
61. `lag_search_min` に `float(min(candidate_lag_sec))` の結果を代入する。
62. `lag_search_max` に `float(max(candidate_lag_sec))` の結果を代入する。
63. `lag_boundary_hit` に `bool(math.isclose(selected_lag, lag_search_min, abs_tol=0.5 * sample_period_sec) or math.isclose(selected_lag, lag_search_max, abs_tol=0.5 * sample_period_sec))` の結果を代入する。
64. `out['accel_ms2']` に `traces[selected_method, selected_window, selected_lag].fillna(0.0).clip(lower=-1.5, upper=1.5)` の結果を代入する。
65. `out['acceleration_timestamp_lag_sec']` に `selected_lag` の結果を代入する。
66. `out['acceleration_filter_method']` に `selected_method` の結果を代入する。
67. `out['acceleration_filter_window_samples']` に `selected_window` の結果を代入する。
68. `(out, {'enabled': True, 'adopted': adopted, 'method': 'fixed-mass GNSS acceleration filter/lag selection with last-race-day holdout', 'sample_period_sec': sample_period_sec, 'selected_filter_method': selected_method, 'selected_filter_window_samples': selected_window, 'selected_filter_window_sec': selected_window * sample_period_sec, 'selected_lag_sec': selected_lag, 'lag_search_min_sec': lag_search_min, 'lag_search_max_sec': lag_search_max, 'lag_search_boundary_hit': lag_boundary_hit, 'holdout_day': holdout_day, 'training_sample_count': int(train_mask.sum()), 'validation_sample_count': int(validation_mask.sum()), 'baseline_training_rmse_w': float(baseline['training_rmse_w']), 'selected_training_rmse_w': float(selected['training_rmse_w']), 'baseline_validation_rmse_w': float(baseline['validation_rmse_w']), 'selected_validation_rmse_w': float(selected['validation_rmse_w']), 'training_improvement_w': float(float(baseline['training_rmse_w']) - float(selected['training_rmse_w'])), 'validation_rmse_ratio': validation_ratio if adopted else 1.0, 'candidates': records})` を返す。

代表コード断片

```
def fit_acceleration_timestamp_alignment(
    frame: pd.DataFrame,
    base_model,
    mot: MotionFitResult,
    options: dict | None = None,
) -> tuple[pd.DataFrame, Dict[str, Any]]:
    """Identify the filter and timestamp offset of GPS-derived acceleration.

    Vehicle mass remains fixed. Candidate observation filters and offsets are
    fitted on all but the last race day and adopted only when the held-out last
    day does not regress. This models quantized/asynchronous GNSS observations;
    it is not a tunable vehicle force or a live-command filter.
    """
    cfg = options if isinstance(options, dict) else {}
    enabled = bool(cfg.get("enabled", True))
    required = {
        "time_utc",
        "day",
    }
```

```

        "speed_kmh",
        "slope_pct",
        "headwind_archive_ms",
        "solar_power_w_obs",
        "battery_power_w_obs",
        "exclude_power_fit",
        "accel_ms2",
    }
    if not enabled or not required.issubset(frame.columns):
        return frame.copy(), {
            "enabled": enabled,
            "adopted": False,
            "reason": "disabled_or_missing_columns",
        }

    out = frame.copy()
    out["accel_ms2_previous"] = pd.to_numeric(out["accel_ms2"], errors="coerce")
    ...

```

7.40 L1631 関数 `_grade_from_smoothed_elevation`

- 行範囲: L1631–L1650
- 所属: module 直下
- 定義: `_grade_from_smoothed_elevation(distance_km:np.ndarray,elevation_m:np.ndarray,smoothing_window_km:float)->np.ndarray`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `diff`, `float`, `gradient`, `int`, `len`, `max`, `median`, `min`, `round`, `savgol_filter`
- 戻り値の要点: `np.gradient(smoothed, distance_km) * 0.1 / np.gradient(elevation_m, distance_km) * 0.1`
- 主なローカル代入名: `samples`, `smoothed`, `spacing`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `spacing` に `float(np.median(np.diff(distance_km)))` の結果を代入する。
2. `samples` に `max(3, int(round(float(smoothing_window_km) / max(spacing, 1e-09))))` の結果を代入する。
3. 条件 `samples % 2 == 0` を判定し、真なら内部処理を行う。
4. `samples` を `Add` で更新する。
5. `samples` に `min(samples, len(elevation_m) if len(elevation_m) % 2 else len(elevation_m) - 1)` の結果を代入する。

6. 条件 `samples < 3` を判定し、真なら内部処理を行う。
7. `np.gradient(elevation_m, distance_km) * 0.1` を返す。
8. `smoothed` に `savgol_filter(elevation_m, samples, min(2, samples - 1), mode='interp')` の結果を代入する。
9. `np.gradient(smoothed, distance_km) * 0.1` を返す。

代表コード断片

```
def _grade_from_smoothed_elevation(
    distance_km: np.ndarray,
    elevation_m: np.ndarray,
    smoothing_window_km: float,
) -> np.ndarray:
    spacing = float(np.median(np.diff(distance_km)))
    samples = max(3, int(round(float(smoothing_window_km) / max(spacing, 1.0e-9))))
    if samples % 2 == 0:
        samples += 1
    samples = min(samples, len(elevation_m) if len(elevation_m) % 2 else len(elevation_m) - 1)
    if samples < 3:
        return np.gradient(elevation_m, distance_km) * 0.1
    smoothed = savgol_filter(
        elevation_m,
        samples,
        min(2, samples - 1),
        mode="interp",
    )
    # elevation [m] / distance [km] * 0.1 converts to percent grade.
    return np.gradient(smoothed, distance_km) * 0.1
```

7.41 L1653 関数 `fit_grade_observation_alignment`

- 行範囲: L1653–L1883
- 所属: module 直下
- 定義: `fit_grade_observation_alignment(frame:pd.DataFrame,route_profile_csv:Path,output_csv:Path,base_model,mot:MotionFitResult,options:dict|None=None)->tuple[pd.DataFrame,Dict[str,Any],Path|None]`
- docstring: Cross-validate a DEM smoothing length and distance alignment.
This stage calibrates the route observation, not vehicle mass or resistance. The selected route stores the unscaled DEM grade. “grade_scale” remains a separately fitted model coefficient in the following motion fit.
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `MotionFitResult`, `_grade_from_smoothed_elevation`, `_motion_predictions_for_acceleration`, `all`, `append`, `arange`, `astype`, `bool`, `copy`, `diff`, `difference`
- 戻り値の要点: (out, {'enabled': True, 'adopted': True, 'reason': 'training_improvement_and_last_day_holdout_passed', 'method': 'Savitzky-Golay DEM elevation differentiation with last-day holdout', 'source_route_profile_csv': str(route_profile_csv), 'adopted_route_profile_csv': str(output_csv), 'holdout_day': holdout_day, 'training_sample_count': int(train_mask.sum()), 'validation_sample_count': int(validation_mask.sum()), 'selected_smoothing_window_km': selected_window, 'smoothing_search_max_km': smoothing_search_max, 'smoothing_search_boundary_hit': smoothing_boundary_hit, 'selected_distance_offset_km': selected_offset, 'selected_provisional_grade_scale': float(best['provisional_grade_scale']), 'baseline_training_rmse_w': baseline_training_rmse, 'selected_training_rmse_w': float(best['training_rmse_w']), 'training_improvement_w': training_improvement, 'baseline_validation_rmse_w': baseline_validation_rmse, 'selected_validation_rmse_w': float(best['validation_rmse_w']), 'validation_rmse_ratio': validation_ratio, 'candidate_count': len(records), 'top_candidates': top_candidates}, output_csv) / (frame.copy(), {'enabled': bool(cfg.get('enabled', True))},

```
'adopted': False, 'reason': 'disabled_or_route_profile_missing'}, None) / (frame.copy(), {'enabled': True, 'adopted': False, 'reason': 'route_profile_requires_distance_and_elevation', 'route_profile_csv': str(route_profile_csv)}, None) / (frame.copy(), {'enabled': True, 'adopted': False, 'reason': 'insufficient_monotonic_route_elevation_samples', 'sample_count': int(len(source))}, None)
```

- 主なローカル代入名: acceleration, adopted, adopted_route, aligned_route_slope, baseline_predicted, baseline_training_rmse, baseline_validation_rmse, best, candidate_frame, candidate_mot, cfg, common, distance, distance_column, elevation, elevation_column, grade_scale, grade_scales, holdout_day, key
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 6、ループ 3、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- lambda は名前を付けずに短い関数オブジェクトを作る構文である。
- 内包表記は、反復・条件・値生成を一つの式で記述して collection を作る。

上から順の処理

1. cfg に options if isinstance(options, dict) else {} の結果を代入する。
2. 条件 not bool(cfg.get('enabled', True)) or not route_profile_csv.is_file() を判定し、真なら内部処理を行う。
3. (frame.copy(), {'enabled': bool(cfg.get('enabled', True)), 'adopted': False, 'reason': 'disabled_or_route_profile_missing', None) を返す。
4. route に pd.read_csv(route_profile_csv) の結果を代入する。
5. distance_column に next((key for key in ('dist_km', 's_km', 'distance_km') if key in route.columns), None) の結果を代入する。
6. elevation_column に next((key for key in ('elev_m', 'elev_dem_m', 'alt_m', 'altitude_m') if key in route.columns), None) の結果を代入する。
7. 条件 distance_column is None or elevation_column is None を判定し、真なら内部処理を行う。
8. (frame.copy(), {'enabled': True, 'adopted': False, 'reason': 'route_profile_requires_distance_and_elevation', 'route_profile_csv': str(route_profile_csv)}, None) を返す。
9. source に pd.DataFrame({'dist_km': pd.to_numeric(route[distance_column], errors='coerce'), 'elev_m': pd.to_numeric(route[elevation_column], errors='coerce')}).dropna() の結果を代入する。
10. source に source.groupby('dist_km', as_index=False).mean().sort_values('dist_km') の結果を代入する。
11. 条件 len(source) < 21 or not np.all(np.diff(source['dist_km'].to_numpy()) > 0.0) を判定し、真なら内部処理を行う。
12. (frame.copy(), {'enabled': True, 'adopted': False, 'reason': 'insufficient_monotonic_route_elevation_samples', 'sample_count': int(len(source))}, None) を返す。
13. required に {'day', 's_km', 'speed_kmh', 'accel_ms2', 'headwind_archive_ms', 'solar_power_w_obs', 'battery_power_w_obs', 'exclude_power_fit', 'slope_pct'} の結果を代入する。

14. 条件 `not required.issubset(frame.columns)` を判定し、真なら内部処理を行う。
15. `(frame.copy(), {'enabled': True, 'adopted': False, 'reason': 'replay_missing_grade_validation_columns', 'missing_columns': sorted(required.difference(frame.columns))}, None)` を返す。
16. `out` に `frame.copy()` の結果を代入する。
17. `observed` に `pd.to_numeric(out['battery_power_w_obs'], errors='coerce').to_numpy(dtype=float)` の結果を代入する。
18. `common` に `~out['exclude_power_fit'].fillna(False).astype(bool) & pd.to_numeric(out['speed_kmh'], errors='coerce').ge(12.0) & np.isfinite(observed) & np.isfinite(pd.to_numeric(out['s_kmh'], errors='coerce'))` の結果を代入する。
19. `holdout_day` に `int(cfg.get('holdout_day', pd.to_numeric(out['day'], errors='coerce').max()))` の結果を代入する。
20. `train_mask` に `(common & pd.to_numeric(out['day'], errors='coerce').ne(holdout_day)).to_numpy(dtype=bool)` の結果を代入する。
21. `validation_mask` に `(common & pd.to_numeric(out['day'], errors='coerce').eq(holdout_day)).to_numpy(dtype=bool)` の結果を代入する。
22. `min_train` に `int(cfg.get('minimum_train_samples', 500))` の結果を代入する。
23. `min_validation` に `int(cfg.get('minimum_validation_samples', 100))` の結果を代入する。
24. 条件 `int(train_mask.sum()) < min_train or int(validation_mask.sum()) < min_validation` を判定し、真なら内部処理を行う。
25. `(out, {'enabled': True, 'adopted': False, 'reason': 'insufficient_train_or_validation_samples', 'training_sample_count': int(train_mask.sum()), 'validation_sample_count': int(validation_mask.sum())}, None)` を返す。
26. `acceleration` に `pd.to_numeric(out['accel_ms2'], errors='coerce').fillna(0.0)` の結果を代入する。
27. 関数 `rmse` を定義する。
28. `baseline_predicted` に `_motion_predictions_for_acceleration(out, acceleration, base_model, mot)` の結果を代入する。
29. `baseline_training_rmse` に `rmse(baseline_predicted, train_mask)` の結果を代入する。
30. `baseline_validation_rmse` に `rmse(baseline_predicted, validation_mask)` の結果を代入する。
31. `windows` に `[float(value) for value in cfg.get('smoothing_windows_kmh', [0.5, 1.1, 2.1, 3.1, 5.1])]` の結果を代入する。
32. `offsets` に `[float(value) for value in cfg.get('distance_offsets_kmh', np.round(np.arange(-0.5, 0.5001, 0.1), 6).tolist())]` の結果を代入する。
33. `grade_scales` に `[float(value) for value in cfg.get('grade_scales', np.round(np.arange(0.5, 1.0001, 0.05), 6).tolist())]` の結果を代入する。
34. `distance` に `source['dist_kmh'].to_numpy(dtype=float)` の結果を代入する。
35. `elevation` に `source['elev_m'].to_numpy(dtype=float)` の結果を代入する。
36. `replay_distance` に `pd.to_numeric(out['s_kmh'], errors='coerce').to_numpy(dtype=float)` の結果を代入する。
37. `records` に `[]` を代入する。
38. `route_slopes` に `{}` を代入する。

39. windows を順に走査し、各要素を window_km に入れて処理する。
40. source_slope に _grade_from_smoothed_elevation(distance, elevation, window_km) の結果を代入する。
41. offsets を順に走査し、各要素を offset_km に入れて処理する。
42. aligned_route_slope に np.interp(distance + offset_km, distance, source_slope, left=source_slope[0], right=source_slope[-1]) の結果を代入する。
43. route_slopes[window_km, offset_km] に aligned_route_slope の結果を代入する。
44. candidate_frame に out.copy() の結果を代入する。
45. candidate_frame['slope_pct'] に np.interp(replay_distance, distance, aligned_route_slope, left=aligned_route_slope[0], right=aligned_route_slope[-1]) の結果を代入する。
46. grade_scales を順に走査し、各要素を grade_scale に入れて処理する。
47. candidate_mot に MotionFitResult(**{**mot.__dict__, 'grade_scale': float(grade_scale)}) の結果を代入する。
48. predicted に _motion_predictions_for_acceleration(candidate_frame, acceleration, base_model, candidate_mot) の結果を代入する。
49. records.append(...) を実行する。
50. best に min(records, key=lambda row: float(row['training_rmse_w'])) の結果を代入する。
51. training_improvement に baseline_training_rmse - float(best['training_rmse_w']) の結果を代入する。
52. validation_ratio に float(best['validation_rmse_w']) / max(baseline_validation_rmse, 1e-09) の結果を代入する。
53. adopted に bool(training_improvement >= float(cfg.get('minimum_training_improvement_w', 5.0)) and validation_ratio <= float(cfg.get('maximum_validation_rmse_ratio', 1.02))) の結果を代入する。
54. 条件 not adopted を判定し、真なら内部処理を行う。
55. (out, {'enabled': True, 'adopted': False, 'reason': 'training_gain_or_holdout_gate_failed', 'method': 'Savitzky-Golay DEM elevation differentiation with last-day holdout', 'baseline_training_rmse_w': baseline_training_rmse, 'selected_training_rmse_w': float(best['training_rmse_w']), 'baseline_validation_rmse_w': baseline_validation_rmse, 'selected_validation_rmse_w': float(best['validation_rmse_w']), 'validation_rmse_ratio': validation_ratio, 'candidate_count': len(records)}, None) を返す。
56. selected_window に float(best['smoothing_window_km']) の結果を代入する。
57. selected_offset に float(best['distance_offset_km']) の結果を代入する。
58. smoothing_search_max に float(max(windows)) の結果を代入する。
59. smoothing_boundary_hit に math.isclose(selected_window, smoothing_search_max, rel_tol=0.0, abs_tol=1e-09) の結果を代入する。
60. selected_route_slope に route_slopes[selected_window, selected_offset] の結果を代入する。
61. out['slope_pct_previous'] に pd.to_numeric(out['slope_pct'], errors='coerce') の結果を代入する。
62. out['slope_pct'] に np.interp(replay_distance, distance, selected_route_slope, left=selected_route_slope[0], right=selected_route_slope[-1]) の結果を代入する。
63. adopted_route に pd.DataFrame({'dist_km': distance, 'elev_m': elevation, 'slope_pct': selected_route_slope, 'headwind_ms': np.zeros(len(distance), dtype=float)}) の結果を代入する。
64. ensure_dir(...) を実行する。

65. `adopted_route.to_csv(...)` を実行する。
66. `top_candidates` に `sorted(records, key=lambda row: float(row['training_rmse_w']))[:25]` の結果を代入する。
67. `(out, {'enabled': True, 'adopted': True, 'reason': 'training_improvement_and_last_day_holdout_passed', 'method': 'Savitzky-Golay DEM elevation differentiation with last-day holdout', 'source_route_profile_csv': str(route_profile_csv), 'adopted_route_profile_csv': str(output_csv), 'holdout_day': holdout_day, 'training_sample_count': int(train_mask.sum()), 'validation_sample_count': int(validation_mask.sum()), 'selected_smoothing_window_km': selected_window, 'smoothing_search_max_km': smoothing_search_max, 'smoothing_search_boundary_hit': smoothing_boundary_hit, 'selected_distance_offset_km': selected_offset, 'selected_provisional_grade_scale': float(best['provisional_grade_scale']), 'baseline_training_rmse_w': baseline_training_rmse, 'selected_training_rmse_w': float(best['training_rmse_w']), 'training_improvement_w': training_improvement, 'baseline_validation_rmse_w': baseline_validation_rmse, 'selected_validation_rmse_w': float(best['validation_rmse_w']), 'validation_rmse_ratio': validation_ratio, 'candidate_count': len(records), 'top_candidates': top_candidates}, output_csv)` を返す。

代表コード断片

```
def fit_grade_observation_alignment(
    frame: pd.DataFrame,
    route_profile_csv: Path,
    output_csv: Path,
    base_model,
    mot: MotionFitResult,
    options: dict | None = None,
) -> tuple[pd.DataFrame, Dict[str, Any], Path | None]:
    """Cross-validate a DEM smoothing length and distance alignment.

    This stage calibrates the route observation, not vehicle mass or resistance.
    The selected route stores the unscaled DEM grade. ``grade_scale`` remains a
    separately fitted model coefficient in the following motion fit.
    """
    cfg = options if isinstance(options, dict) else {}
    if not bool(cfg.get("enabled", True)) or not route_profile_csv.is_file():
        return frame.copy(), {
            "enabled": bool(cfg.get("enabled", True)),
            "adopted": False,
            "reason": "disabled_or_route_profile_missing",
        }, None

    route = pd.read_csv(route_profile_csv)
    distance_column = next(
        (key for key in ("dist_km", "s_km", "distance_km") if key in route.columns),
        None,
    )
    elevation_column = next(
        (key for key in ("elev_m", "elev_dem_m", "alt_m", "altitude_m") if key in route.columns),
        None,
    )
    if distance_column is None or elevation_column is None:
        return frame.copy(), {
            "enabled": True,
            "adopted": False,
        }, None
    ...
```

7.42 L1750 関数 `fit_grade_observation_alignment.rmse`

- 行範囲: L1750–L1752
- 所属: `fit_grade_observation_alignment`

- 定義: `rmse(predicted:np.ndarray,mask:np.ndarray)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `isfinite`, `mean`, `sqrt`, `square`
- 戻り値の要点: `float(np.sqrt(np.mean(np.square(observed[usable] - predicted[usable])))`
- 主なローカル代入名: `usable`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `usable` に `mask & np.isfinite(predicted) & np.isfinite(observed)` の結果を代入する。
2. `float(np.sqrt(np.mean(np.square(observed[usable] - predicted[usable])))` を返す。

代表コード断片

```
def rmse(predicted: np.ndarray, mask: np.ndarray) -> float:
    usable = mask & np.isfinite(predicted) & np.isfinite(observed)
    return float(np.sqrt(np.mean(np.square(observed[usable] - predicted[usable]))))
```

7.43 L1886 関数 `pv_leave_one_day_out_validation`

- 行範囲: L1886–L1961
- 所属: module 直下
- 定義: `pv_leave_one_day_out_validation(frame:pd.DataFrame,model,*,panel_deployment_options:Dict[str,float]|None=None)->Dict[str,float]`
- docstring: Validate the PV chain on days excluded from all PV-scalar fitting.
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `asarray`, `astype`, `attach_archive_pv_model`, `copy`, `dict`, `difference`, `drop`, `drop_duplicates`, `dropna`, `extend`, `fillna`
- 戻り値の要点: `{'pv_lodo_fold_count': int(fold_count), 'pv_lodo_moving_rmse_w': rmse(moving_residuals), 'pv_lodo_moving_sample_count': int(len(moving_residuals)), 'pv_lodo_deployed_stop_rmse_w': rmse(deployed_residuals), 'pv_lodo_deployed_stop_sample_count': int(len(deployed_residuals))} / float(np.sqrt(np.mean(np.square(array)))) if array.size else float('nan')`
- 主なローカル代入名: `array`, `day_value`, `deployed`, `deployed_residuals`, `deployment`, `fold_count`, `fold_fit`, `hold-out`, `keep`, `local_time`, `missing`, `modeled`, `moving`, `moving_residuals`, `observed`, `predicted`, `required`, `train`, `valid`, `work`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError(f'PV leave-one-day-out validation is missing columns: {missing}')`
- 制御構造の規模: 条件分岐 4、ループ 1、try 1

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. required に {'time_utc', 'speed_kmh', 'GHI_archive', 'DNI_archive', 'DHI_archive', 'Tamb_archive_C', 'solar_power_w_obs', 'exclude_weather_fit'} の結果を代入する。
2. missing に sorted(required.difference(frame.columns)) の結果を代入する。
3. 条件 missing を判定し、真なら内部処理を行う。
4. ValueError(f'PV leave-one-day-out validation is missing columns: {missing}') を送出する。
5. keep に sorted(required.union({'day', 's_kmh'}).intersection(frame.columns)) の結果を代入する。
6. work に frame.loc[:, keep].copy() の結果を代入する。
7. 条件 'day' in work.columns を判定し、真なら内部処理を行う。
8. work['_pv_cv_day'] に pd.to_numeric(work['day'], errors='coerce') の結果を代入する。
9. 上の条件が偽の場合:
10. local_time に pd.to_datetime(work['time_utc'], format='mixed', utc=True).dt.tz_convert(TIMEZONE_LOCAL) の結果を代入する。
11. work['_pv_cv_day'] に local_time.dt.date.astype(str) の結果を代入する。
12. deployment に dict(panel_deployment_options or {}) の結果を代入する。
13. moving_residuals に [] の結果を代入する。
14. deployed_residuals に [] の結果を代入する。
15. fold_count に 0 の結果を代入する。
16. work['_pv_cv_day'].dropna().drop_duplicates().tolist() を順に走査し、各要素を day_value に入れて処理する。
17. holdout に work.loc[work['_pv_cv_day'] == day_value].drop(columns='_pv_cv_day') の結果を代入する。
18. train に work.loc[work['_pv_cv_day'] != day_value].drop(columns='_pv_cv_day') の結果を代入する。
19. 条件 len(train) < 100 or len(holdout) < 20 を判定し、真なら内部処理を行う。
20. Continue 文を実行する。
21. 例外処理を伴う try ブロックを実行する。
22. fold_fit に fit_pv_parameters(train, model, irradiance_source='GHI_archive', operating_state='moving') の結果を代入する。
23. fold_fit に fit_stop_tilt_fraction(train, model, fold_fit, **deployment) の結果を代入する。
24. predicted に attach_archive_pv_model(holdout, model, fold_fit, irradiance_source='GHI_archive', **deployment) の結果を代入する。
25. (KeyError, ValueError, RuntimeError) を捕捉した場合:
26. Continue 文を実行する。

27. `observed` に `pd.to_numeric(predicted['solar_power_w_obs'], errors='coerce')` の結果を代入する。
28. `modeled` に `pd.to_numeric(predicted['solar_power_w_model'], errors='coerce')` の結果を代入する。
29. `valid` に `observed.notna() & modeled.notna()` の結果を代入する。
30. 条件 `'exclude_weather_fit'` in `predicted.columns` を判定し、真なら内部処理を行う。
31. `valid` を `BitAnd` で更新する。
32. `moving` に `valid & pd.to_numeric(predicted['speed_kmh'], errors='coerce').ge(12.0)` の結果を代入する。
33. `deployed` に `valid & predicted['panel_deployed_model'].fillna(False).astype(bool)` の結果を代入する。
34. `moving_residuals.extend(...)` を実行する。
35. `deployed_residuals.extend(...)` を実行する。
36. `fold_count` を `Add` で更新する。
37. 関数 `rmse` を定義する。
38. `{'pv_lodo_fold_count': int(fold_count), 'pv_lodo_moving_rmse_w': rmse(moving_residuals), 'pv_lodo_moving_sample': int(len(moving_residuals)), 'pv_lodo_deployed_stop_rmse_w': rmse(deployed_residuals), 'pv_lodo_deployed_stop_sample': int(len(deployed_residuals))}` を返す。

代表コード断片

```
def pv_leave_one_day_out_validation(
    frame: pd.DataFrame,
    model,
    *,
    panel_deployment_options: Dict[str, float] | None = None,
) -> Dict[str, float]:
    """Validate the PV chain on days excluded from all PV-scalar fitting."""
    required = {
        "time_utc",
        "speed_kmh",
        "GHI_archive",
        "DNI_archive",
        "DHI_archive",
        "Tamb_archive_C",
        "solar_power_w_obs",
        "exclude_weather_fit",
    }
    missing = sorted(required.difference(frame.columns))
    if missing:
        raise ValueError(f"Pv leave-one-day-out validation is missing columns: {missing}")
    keep = sorted(required.union({"day", "s_kmh"}).intersection(frame.columns))
    work = frame.loc[:, keep].copy()
    if "day" in work.columns:
        work["_pv_cv_day"] = pd.to_numeric(work["day"], errors="coerce")
    else:
        local_time = pd.to_datetime(work["time_utc"], format="mixed", utc=True).dt.
        tz_convert(TIMEZONE_LOCAL)
        work["_pv_cv_day"] = local_time.dt.date.astype(str)
    deployment = dict(panel_deployment_options or {})
    moving_residuals = []
    deployed_residuals = []
    fold_count = 0
    for day_value in work["_pv_cv_day"].dropna().drop_duplicates().tolist():
        holdout = work.loc[work["_pv_cv_day"] == day_value].drop(columns="_pv_cv_day")
        train = work.loc[work["_pv_cv_day"] != day_value].drop(columns="_pv_cv_day")
        if len(train) < 100 or len(holdout) < 20:
            ...
```

7.44 L1950 関数 `pv_leave_one_day_out_validation.rmse`

- 行範囲: L1950–L1953
- 所属: `pv_leave_one_day_out_validation`
- 定義: `rmse(values)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `asarray`, `float`, `isfinite`, `mean`, `sqrt`, `square`
- 戻り値の要点: `float(np.sqrt(np.mean(np.square(array)))) if array.size else float('nan')`
- 主なローカル代入名: `array`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `array` に `np.asarray(values, dtype=float)` の結果を代入する。
2. `array` に `array[np.isfinite(array)]` の結果を代入する。
3. `float(np.sqrt(np.mean(np.square(array)))) if array.size else float('nan')` を返す。

代表コード断片

```
def rmse(values) -> float:
    array = np.asarray(values, dtype=float)
    array = array[np.isfinite(array)]
    return float(np.sqrt(np.mean(np.square(array)))) if array.size else float("nan")
```

7.45 L1964 関数 `add_end_to_end_metrics`

- 行範囲: L1964–L1970
- 所属: `module` 直下
- 定義: `add_end_to_end_metrics(primary:Dict[str,float],end_to_end:Dict[str,float])->Dict[str,float]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `dict`, `items`
- 戻り値の要点: `out`
- 主なローカル代入名: `key`, `out`, `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. out に dict(primary) の結果を代入する。
2. end_to_end.items() を順に走査し、各要素を (key, value) に入れて処理する。
3. out[f'end_to_end_{key}'] に value の結果を代入する。
4. out['vehicle_fit_solar_source'] に 'measured_when_available' の結果を代入する。
5. out['end_to_end_solar_source'] に 'weather_and_pv_model' の結果を代入する。
6. out を返す。

代表コード断片

```
def add_end_to_end_metrics(primary: Dict[str, float], end_to_end: Dict[str, float]) -> Dict[str, float]:
    out = dict(primary)
    for key, value in end_to_end.items():
        out[f"end_to_end_{key}"] = value
    out["vehicle_fit_solar_source"] = "measured_when_available"
    out["end_to_end_solar_source"] = "weather_and_pv_model"
    return out
```

7.46 L1973 関数 add_battery_conditional_metrics

- 行範囲: L1973–L1978
- 所属: module 直下
- 定義: add_battery_conditional_metrics(primary:Dict[str,float],conditional:Dict[str,float])->Dict[str,float]
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: dict, items
- 戻り値の要点: out
- 主なローカル代入名: key, out, value
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 1、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. out に dict(primary) の結果を代入する。
2. conditional.items() を順に走査し、各要素を (key, value) に入れて処理する。
3. out[f'battery_conditional_{key}'] に value の結果を代入する。
4. out['battery_conditional_source'] に 'observed_pack_power_and_current' の結果を代入する。
5. out を返す。

代表コード断片

```
def add_battery_conditional_metrics(primary: Dict[str, float], conditional: Dict[str, float]) -> Dict[str, float]:
    out = dict(primary)
    for key, value in conditional.items():
        out[f"battery_conditional_{key}"] = value
    out["battery_conditional_source"] = "observed_pack_power_and_current"
    return out
```

7.47 L1981 関数 write_replay_csv

- 行範囲: L1981–L1995
- 所属: module 直下
- 定義: write_replay_csv(frame:pd.DataFrame,output_path:Path,*,chunk_rows:int=5000) ->None
- docstring: Write large replay tables without materializing a full string copy.
- このブロックが直接呼ぶ主な関数・メソッド: copy, ensure_dir, int, len, max, range, strftime, to_csv, to_datetime
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: chunk, rows, start
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 1、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. ensure_dir(...) を実行する。
2. rows に max(1, int(chunk_rows)) の結果を代入する。
3. range(0, len(frame), rows) を順に走査し、各要素を start に入れて処理する。
4. chunk に frame.iloc[start:start + rows].copy() の結果を代入する。
5. chunk['time_utc'] に pd.to_datetime(chunk['time_utc'], format='mixed', utc=True).dt.strftime('%Y-%m-%dT%H:%M:%SZ') の結果を代入する。
6. chunk.to_csv(...) を実行する。

代表コード断片

```
def write_replay_csv(frame: pd.DataFrame, output_path: Path, *, chunk_rows: int = 5000) ->
    None:
    """Write large replay tables without materializing a full string copy."""
    ensure_dir(output_path.parent)
    rows = max(1, int(chunk_rows))
    for start in range(0, len(frame), rows):
        chunk = frame.iloc[start : start + rows].copy()
        chunk["time_utc"] = pd.to_datetime(
            chunk["time_utc"], format="mixed", utc=True
        ).dt.strftime("%Y-%m-%dT%H:%M:%SZ")
        chunk.to_csv(
            output_path,
            index=False,
            mode="w" if start == 0 else "a",
            header=start == 0,
        )
```

7.48 L1998 関数 apply_fit_to_cfg

- 行範囲: L1998–L2110
- 所属: module 直下
- 定義: `apply_fit_to_cfg(cfg:dict,*,package_dir:Path,map_assets:Dict[str,Path],pv:PvFitResult,batt:BatteryFitResult,mot:MotionFitResult,observed_log_csv:Path,battery_dynamic_fit:Dict[str,Any]|None=None,solar_measurement_calibration:Dict[str,Any]|None=None,sync_sim_soc0:bool=False)->dict`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `ValueError`, `clip`, `dropna`, `float`, `get`, `items`, `max`, `min`, `read_csv`, `relpath`, `replace`
- 戻り値の要点: `cfg`
- 主なローカル代入名: `autocal`, `dynamic`, `identification`, `key`, `live`, `model`, `mpc`, `ocv_asset`, `ocv_frame`, `ocv_max_v`, `ocv_min_v`, `ocv_path`, `ocv_values`, `path`, `sim`, `soc_max`, `solar_calibration`, `validation_gate`, `weather`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError(f'adopted OCV map has no finite ocv_v values: {ocv_path}')`, `ValueError(f'adopted OCV map is missing ocv_v: {ocv_path}')`, `ValueError(f'adopted OCV maximum {ocv_max_v:.6f} V exceeds the grounded 25S pack limit {BATTERY_PACK_MAX_CHARGE_V:.6f} V')`
- 制御構造の規模: 条件分岐 5、ループ 1、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `cfg.setdefault(...)` を実行する。
2. `map_assets.items()` を順に走査し、各要素を `(key, path)` に入れて処理する。
3. `cfg['paths'][key]` に `os.path.relpath(path, package_dir).replace('\', '/')` の結果を代入する。

4. `cfg['paths']['progress_reference_csv']` に `os.path.relpath(observed_log_csv, package_dir).replace('\\', '/')` の結果を代入する。
5. `model` に `cfg.setdefault('model', {})` の結果を代入する。
6. `model['CdA']` に `round(float(mot.cda), 6)` の結果を代入する。
7. `model['Crr']` に `round(float(mot.crr), 6)` の結果を代入する。
8. `model['P_aux']` に `round(float(mot.p_aux_w), 3)` の結果を代入する。
9. `model['P_aux_stopped']` に `round(float(mot.p_aux_w), 3)` の結果を代入する。
10. `model['P_aux_night']` に `0.0` の結果を代入する。
11. `model.setdefault(...)` を実行する。
12. `model['air_density_mode']` に `'ideal_gas_altitude'` の結果を代入する。
13. `model.setdefault(...)` を実行する。
14. `solar_calibration` に `solar_measurement_calibration` or `{}` の結果を代入する。
15. `model['solar_measurement_gain_to_pack']` に `round(float(solar_calibration.get('gain_to_pack', 1.0)), 8)` の結果を代入する。
16. `model['panel_gain']` に `round(float(pv.panel_gain), 6)` の結果を代入する。
17. `model['grade_scale']` に `round(float(mot.grade_scale), 6)` の結果を代入する。
18. `model['drive_eff_scale']` に `round(float(mot.drive_eff_scale), 6)` の結果を代入する。
19. `model['regen_eff_scale']` に `round(float(mot.drive_eff_scale), 6)` の結果を代入する。
20. `model['regen_utilization']` に `round(float(mot.regen_utilization), 6)` の結果を代入する。
21. `model['rint_scale']` に `round(float(batt.rint_scale), 6)` の結果を代入する。
22. `model['r_line_ohm']` に `round(float(batt.r_line_ohm), 6)` の結果を代入する。
23. `model['eta_charge']` に `round(float(batt.eta_charge), 6)` の結果を代入する。
24. `model['E_nom_Wh']` に `round(float(batt.e_nom_wh), 3)` の結果を代入する。
25. `model['Q_nom_Ah']` に `round(float(batt.e_nom_wh) / BATTERY_NOMINAL_VOLTAGE_V, 6)` の結果を代入する。
26. `ocv_asset` に `map_assets.get('ocv_soc_map')` の結果を代入する。
27. `ocv_max_v` に `float('-inf')` の結果を代入する。
28. 条件 `ocv_asset is not None` を判定し、真なら内部処理を行う。
29. `ocv_path` に `Path(ocv_asset)` の結果を代入する。
30. `ocv_frame` に `pd.read_csv(ocv_path)` の結果を代入する。
31. 条件 `'ocv_v' not in ocv_frame.columns` を判定し、真なら内部処理を行う。
32. `ValueError(f'adopted OCV map is missing ocv_v: {ocv_path}')` を送出する。
33. `ocv_values` に `pd.to_numeric(ocv_frame['ocv_v'], errors='coerce').dropna()` の結果を代入する。
34. 条件 `ocv_values.empty` を判定し、真なら内部処理を行う。
35. `ValueError(f'adopted OCV map has no finite ocv_v values: {ocv_path}')` を送出する。

36. `ocv_max_v` に `float(ocv_values.max())` の結果を代入する。
37. `ocv_min_v` に `float(ocv_values.min())` の結果を代入する。
38. 条件 `ocv_max_v > BATTERY_PACK_MAX_CHARGE_V + 1e-06` を判定し、真なら内部処理を行う。
39. `ValueError(f'adopted OCV maximum {ocv_max_v:.6f} V exceeds the grounded 25S pack limit {BATTERY_PACK_MAX_CHARGE_V:.6f} V')` を送出する。
40. `model['V_min']` に `round(min(float(model.get('V_min', ocv_min_v)), ocv_min_v), 3)` の結果を代入する。
41. `model['V_max']` に `round(BATTERY_PACK_MAX_CHARGE_V, 3)` の結果を代入する。
42. `dynamic` に `battery_dynamic_fit or {}` の結果を代入する。
43. `model['r_polarization_ohm']` に `round(float(dynamic.get('r_polarization_ohm', 0.0)), 6)` の結果を代入する。
44. `model['polarization_tau_sec']` に `round(float(dynamic.get('tau_sec', 60.0)), 6)` の結果を代入する。
45. `model['headwind_gain']` に `round(float(mot.headwind_gain), 6)` の結果を代入する。
46. `identification` に `cfg.setdefault('identification', {})` の結果を代入する。
47. `identification['fitted_replay_soc0']` に `round(float(batt.soc0), 6)` の結果を代入する。
48. `validation_gate` に `identification.setdefault('validation_gate', {})` の結果を代入する。
49. `validation_gate.setdefault(...)` を実行する。
50. `validation_gate.setdefault(...)` を実行する。
51. `validation_gate.setdefault(...)` を実行する。
52. `validation_gate.setdefault(...)` を実行する。
53. `validation_gate.setdefault(...)` を実行する。
54. `validation_gate.setdefault(...)` を実行する。
55. `validation_gate.setdefault(...)` を実行する。
56. `validation_gate.setdefault(...)` を実行する。
57. `validation_gate.setdefault(...)` を実行する。
58. `validation_gate.setdefault(...)` を実行する。
59. `validation_gate.setdefault(...)` を実行する。
60. `validation_gate.setdefault(...)` を実行する。
61. `validation_gate.setdefault(...)` を実行する。
62. `validation_gate.setdefault(...)` を実行する。
63. `validation_gate.setdefault(...)` を実行する。
64. `validation_gate.setdefault(...)` を実行する。
65. `validation_gate.setdefault(...)` を実行する。
66. `validation_gate.setdefault(...)` を実行する。
67. `validation_gate.setdefault(...)` を実行する。
68. `validation_gate.setdefault(...)` を実行する。

69. sim に `cfg.setdefault('simulation', {})` の結果を代入する。
70. soc_max に `float(model.get('soc_max', 0.98))` の結果を代入する。
71. 条件 `sync_sim_soc0` を判定し、真なら内部処理を行う。
72. `sim['soc0']` に `round(float(np.clip(batt.soc0, 0.8, soc_max)), 4)` の結果を代入する。
73. live に `cfg.setdefault('live', {})` の結果を代入する。
74. autocal に `live.setdefault('autocal', {})` の結果を代入する。
75. `autocal['aux_power_w_init']` に `round(float(mot.p_aux_w), 3)` の結果を代入する。
76. weather に `live.setdefault('weather', {})` の結果を代入する。
77. `weather['tcell_gain']` に `round(float(pv.tcell_gain_c_per_wm2), 6)` の結果を代入する。
78. mpc に `cfg.setdefault('mpc', {})` の結果を代入する。
79. `mpc['stop_tilt_fraction']` に `round(float(np.clip(pv.stop_tilt_fraction, 0.0, 1.0)), 6)` の結果を代入する。
80. `mpc.setdefault(...)` を実行する。

代表コード断片

```
def apply_fit_to_cfg(
    cfg: dict,
    *,
    package_dir: Path,
    map_assets: Dict[str, Path],
    pv: PvFitResult,
    batt: BatteryFitResult,
    mot: MotionFitResult,
    observed_log_csv: Path,
    battery_dynamic_fit: Dict[str, Any] | None = None,
    solar_measurement_calibration: Dict[str, Any] | None = None,
    sync_sim_soc0: bool = False,
) -> dict:
    cfg.setdefault("paths", {})
    for key, path in map_assets.items():
        cfg["paths"][key] = os.path.relpath(path, package_dir).replace("\\", "/")
    cfg["paths"]["progress_reference_csv"] = os.path.relpath(observed_log_csv, package_dir).
    replace("\\", "/")

    model = cfg.setdefault("model", {})
    model["CdA"] = round(float(mot.cda), 6)
    model["Crr"] = round(float(mot.crr), 6)
    model["P_aux"] = round(float(mot.p_aux_w), 3)
    model["P_aux_stopped"] = round(float(mot.p_aux_w), 3)
    model["P_aux_night"] = 0.0
    model.setdefault("aux_night_ghi_threshold_wm2", 20.0)
    model["air_density_mode"] = "ideal_gas_altitude"
    model.setdefault("air_density_reference_pressure_pa", 101325.0)
    solar_calibration = solar_measurement_calibration or {}
    model["solar_measurement_gain_to_pack"] = round(
        float(solar_calibration.get("gain_to_pack", 1.0)), 8
    )
    model["panel_gain"] = round(float(pv.panel_gain), 6)
    model["grade_scale"] = round(float(mot.grade_scale), 6)
    model["drive_eff_scale"] = round(float(mot.drive_eff_scale), 6)
    model["regen_eff_scale"] = round(float(mot.drive_eff_scale), 6)
    ...
```

7.49 L2113 関数 update_profile

- 行範囲: L2113–L2154
- 所属: module 直下
- 定義: `update_profile(profile_yaml:Path, cfg:dict, map_assets:Dict[str,Path], pv:PvFitResult, batt:BatteryFitResult, mot:MotionFitResult, observed_log_csv:Path, battery_dynamic_fit:Dict[str,Any]|None=None, solar_measurement_calibration:Dict[str,Any]|None=None, route_profile_asset:Path|None=None, package_dir:Path|None=None)->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path, apply_fit_to_cfg, exists, get, is_absolute, items, list, open, relpath_from, resolve, safe_dump, setdefault`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `cfg, f, key, package_dir, package_path, path, raw`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 5、ループ 1、`try` 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `package_dir` に `profile_yaml.parent` if `package_dir` is `None` else `Path(package_dir)` の結果を代入する。
2. `cfg` に `apply_fit_to_cfg(cfg, package_dir=package_dir, map_assets=map_assets, pv=pv, batt=batt, mot=mot, battery_dynamic_fit=battery_dynamic_fit, solar_measurement_calibration=solar_measurement_calibration, observed_log_csv=observed_log_csv, sync_sim_soc0=False)` の結果を代入する。
3. 条件 `route_profile_asset is not None` を判定し、真なら内部処理を行う。
4. `cfg.setdefault('paths', {})[ 'route_profile_csv' ]` に `relpath_from(package_dir, route_profile_asset)` の結果を代入する。
5. 条件 `profile_yaml.parent.resolve() != package_dir.resolve()` を判定し、真なら内部処理を行う。
6. `list((cfg.get('paths', {}) or {}).items())` を順に走査し、各要素を `(key, raw)` に入れて処理する。
7. 条件 `raw in (None, '')` を判定し、真なら内部処理を行う。
8. `Continue` 文を実行する。
9. `path` に `Path(str(raw))` の結果を代入する。
10. 条件 `path.is_absolute()` を判定し、真なら内部処理を行う。
11. `Continue` 文を実行する。

12. `package_path` に `(package_dir / path).resolve()` の結果を代入する。
13. 条件 `package_path.exists()` を判定し、真なら内部処理を行う。
14. `cfg['paths'][key]` に `relpath_from(profile_yaml.parent, package_path)` の結果を代入する。
15. `with` 文で `profile_yaml.open('w', encoding='utf-8', newline='\n')` を管理しながら処理する。
16. `yaml.safe_dump(...)` を実行する。

代表コード断片

```
def update_profile(
    profile_yaml: Path,
    cfg: dict,
    map_assets: Dict[str, Path],
    pv: PvFitResult,
    batt: BatteryFitResult,
    mot: MotionFitResult,
    observed_log_csv: Path,
    battery_dynamic_fit: Dict[str, Any] | None = None,
    solar_measurement_calibration: Dict[str, Any] | None = None,
    route_profile_asset: Path | None = None,
    package_dir: Path | None = None,
) -> None:
    package_dir = profile_yaml.parent if package_dir is None else Path(package_dir)
    cfg = apply_fit_to_cfg(
        cfg,
        package_dir=package_dir,
        map_assets=map_assets,
        pv=pv,
        batt=batt,
        mot=mot,
        battery_dynamic_fit=battery_dynamic_fit,
        solar_measurement_calibration=solar_measurement_calibration,
        observed_log_csv=observed_log_csv,
        sync_sim_soc0=False,
    )
    if route_profile_asset is not None:
        cfg.setdefault("paths", {})[ "route_profile_csv" ] = relpath_from(
            package_dir, route_profile_asset
        )
    if profile_yaml.parent.resolve() != package_dir.resolve():
        for key, raw in list((cfg.get("paths", {}) or {}).items()):
            if raw in (None, ""):
                continue
            path = Path(str(raw))
...

```

7.50 L2157 関数 `update_profile_artifact_references`

- 行範囲: L2157–L2177
- 所属: module 直下
- 定義: `update_profile_artifact_references(profile_yaml:Path,package_dir:Path,*,fit_summary_yaml:Path,terminal_consistency_yaml:Path|None=None)->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `is_file`, `pop`, `read_text`, `relpath_from`, `safe_dump`, `safe_load`, `setdefault`, `write_text`

- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `cfg`, `identification`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `cfg` に `yaml.safe_load(profile_yaml.read_text(encoding='utf-8'))` or `{}` の結果を代入する。
2. `identification` に `cfg.setdefault('identification', {})` の結果を代入する。
3. `identification['fit_summary_yaml']` に `relpath_from(profile_yaml.parent, fit_summary_yaml)` の結果を代入する。
4. 条件 `terminal_consistency_yaml is not None and terminal_consistency_yaml.is_file()` を判定し、真なら内部処理を行う。
5. `identification['terminal_consistency_yaml']` に `relpath_from(profile_yaml.parent, terminal_consistency_yaml)` の結果を代入する。
6. 上の条件が偽の場合:
7. `identification.pop(...)` を実行する。
8. `profile_yaml.write_text(...)` を実行する。

代表コード断片

```
def update_profile_artifact_references(
    profile_yaml: Path,
    package_dir: Path,
    *,
    fit_summary_yaml: Path,
    terminal_consistency_yaml: Path | None = None,
) -> None:
    cfg = yaml.safe_load(profile_yaml.read_text(encoding="utf-8")) or {}
    identification = cfg.setdefault("identification", {})
    identification["fit_summary_yaml"] = relpath_from(profile_yaml.parent, fit_summary_yaml)
    if terminal_consistency_yaml is not None and terminal_consistency_yaml.is_file():
        identification["terminal_consistency_yaml"] = relpath_from(
            profile_yaml.parent, terminal_consistency_yaml
        )
    else:
        identification.pop("terminal_consistency_yaml", None)
    profile_yaml.write_text(
        yaml.safe_dump(cfg, sort_keys=False, allow_unicode=True),
        encoding="utf-8",
        newline="\n",
    )
```

7.51 L2180 関数 `write_terminal_consistency_from_anchor`

- 行範囲: L2180–L2315
- 所属: module 直下
- 定義: `write_terminal_consistency_from_anchor(output_path:Path,terminal_anchor:Dict[str,Any],*,max_spread:float,validation_metrics:Dict[str,Any]|None=None, replay_soc_error_max:float=0.02,replay_voltage_error_max_v:float=0.5,vehicle_soc_error_max:float=0.02,vehicle_voltage_error_max_v:float=0.5,end_to_end_soc_error_max:float=0.03,end_to_end_voltage_error_max_v:float=1.0)->Path`
- docstring: Always materialize the independent terminal-evidence gate.
Detailed channel reconstruction may later enrich this file, but profile adoption must never silently drop the gate merely because a separate reporting command was not run.
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `bool`, `float`, `get`, `isfinite`, `list`, `max`, `min`, `mkdir`, `safe_dump`, `write_text`
- 戻り値の要点: `output_path`
- 主なローカル代入名: `cross_channel_gate`, `end_to_end_gate`, `end_to_end_soc_error`, `end_to_end_voltage_error`, `evidence_spread_gate`, `hi`, `high_precision_gate`, `lo`, `local_anchor_gate`, `metrics`, `payload`, `replay_gate`, `replay_soc_error`, `replay_voltage_error`, `replay_voltage_observed`, `replay_voltage_predicted`, `sigma`, `spread`, `target`, `vehicle_gate`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `lo` に `float(terminal_anchor.get('soc_evidence_min', float('nan')))` の結果を代入する。
2. `hi` に `float(terminal_anchor.get('soc_evidence_max', float('nan')))` の結果を代入する。
3. `target` に `float(terminal_anchor.get('soc_target', float('nan')))` の結果を代入する。
4. `sigma` に `float(terminal_anchor.get('soc_sigma', float('nan')))` の結果を代入する。
5. `spread` に `hi - lo if np.isfinite(lo) and np.isfinite(hi) else float('nan')` の結果を代入する。
6. `metrics` に `validation_metrics` or `{}` の結果を代入する。
7. `replay_soc_error` に `abs(float(metrics.get('battery_conditional_retire_anchor_soc_error', float('nan'))))` の結果を代入する。
8. `replay_voltage_observed` に `float(metrics.get('battery_conditional_retire_anchor_voltage_obs_v', float('nan')))` の結果を代入する。
9. `replay_voltage_predicted` に `float(metrics.get('battery_conditional_retire_anchor_voltage_pred_v', float('nan')))` の結果を代入する。

10. `replay_voltage_error` に `abs(replay_voltage_predicted - replay_voltage_observed)` の結果を代入する。
11. `vehicle_soc_error` に `abs(float(metrics.get('retire_anchor_soc_error', float('nan'))))` の結果を代入する。
12. `vehicle_voltage_error` に `abs(float(metrics.get('retire_anchor_voltage_pred_v', float('nan')))) - float(metrics.get('retire_anchor_voltage_obs_v', float('nan'))))` の結果を代入する。
13. `end_to_end_soc_error` に `abs(float(metrics.get('end_to_end_retire_anchor_soc_error', float('nan'))))` の結果を代入する。
14. `end_to_end_voltage_error` に `abs(float(metrics.get('end_to_end_retire_anchor_voltage_pred_v', float('nan')))) - float(metrics.get('end_to_end_retire_anchor_voltage_obs_v', float('nan'))))` の結果を代入する。
15. `evidence_spread_gate` に `bool(np.isfinite(spread) and spread <= float(max_spread))` の結果を代入する。
16. `local_anchor_gate` に `bool(terminal_anchor.get('quality_gate_pass', False))` の結果を代入する。
17. `cross_channel_gate` に `bool(terminal_anchor.get('weak_channel_cross_consistency_gate_pass', False))` の結果を代入する。
18. `replay_gate` に `bool(np.isfinite(replay_soc_error) and replay_soc_error <= float(replay_soc_error_max) and np.isfinite(replay_voltage_error) and (replay_voltage_error <= float(replay_voltage_error_max_v)))` の結果を代入する。
19. `vehicle_gate` に `bool(np.isfinite(vehicle_soc_error) and vehicle_soc_error <= float(vehicle_soc_error_max) and np.isfinite(vehicle_voltage_error) and (vehicle_voltage_error <= float(vehicle_voltage_error_max_v)))` の結果を代入する。
20. `end_to_end_gate` に `bool(np.isfinite(end_to_end_soc_error) and end_to_end_soc_error <= float(end_to_end_soc_error_max) and np.isfinite(end_to_end_voltage_error) and (end_to_end_voltage_error <= float(end_to_end_voltage_error_max_v)))` の結果を代入する。
21. `high_precision_gate` に `bool(evidence_spread_gate and local_anchor_gate and cross_channel_gate and replay_gate and vehicle_gate and end_to_end_gate)` の結果を代入する。
22. `payload` に `{'source': 'terminal_anchor_evidence_envelope', 'terminal_distance_km': float(terminal_anchor.get('s_km', float('nan'))), 'evidence_interval_min': lo, 'evidence_interval_max': hi, 'unweighted_central_estimate': target, 'spread_percentage_points': 100.0 * spread if np.isfinite(spread) else float('nan'), 'high_precision_gate_pass': high_precision_gate, 'high_precision_checks': {'terminal_evidence_spread': evidence_spread_gate, 'local_terminal_anchor_quality': local_anchor_gate, 'independent_cross_channel_consistency': cross_channel_gate, 'conditional_replay_terminal_soc': bool(np.isfinite(replay_soc_error) and replay_soc_error <= float(replay_soc_error_max)), 'conditional_replay_terminal_voltage': bool(np.isfinite(replay_voltage_error) and replay_voltage_error <= float(replay_voltage_error_max_v)), 'vehicle_replay_terminal': vehicle_gate, 'end_to_end_replay_terminal': end_to_end_gate}, 'conditional_replay_terminal_soc_error': replay_soc_error, 'conditional_replay_terminal_soc_error_max': float(replay_soc_error_max), 'conditional_replay_terminal_voltage_error_v': replay_voltage_error, 'conditional_replay_terminal_voltage_error_max_v': float(replay_voltage_error_max_v), 'vehicle_replay_terminal_soc_error': vehicle_soc_error, 'vehicle_replay_terminal_soc_error_max': float(vehicle_soc_error_max), 'vehicle_replay_terminal_voltage_error': vehicle_voltage_error, 'vehicle_replay_terminal_voltage_error_max_v': float(vehicle_voltage_error_max_v), 'end_to_end_replay_terminal_soc_error': end_to_end_soc_error, 'end_to_end_replay_terminal_soc_error_max': float(end_to_end_soc_error_max), 'end_to_end_replay_terminal_voltage_error_v': end_to_end_voltage_error, 'end_to_end_replay_terminal_voltage_error_max_v': float(end_to_end_voltage_error_max_v), 'random_effects_soc': target, 'random_effects_standard_error': sigma, 'random_effects_ci95_min': max(0.0, target - 1.96 * sigma) if np.isfinite(target) and np.isfinite(sigma) else float('nan'), 'random_effects_ci95_max': min(1.0, target + 1.96 * sigma) if np.isfinite(target) and np.isfinite(sigma) else float('nan'), 'method': terminal_anchor.get('method', ''), 'source_documents': list(terminal_anchor.get('source_documents', []) or []), 'fusion_caution': 'The center summarizes the declared evidence envelope. A narrow conditional pulse interval cannot override cross-channel disagreement or replay mismatch.', 'interpretation': 'Independent evidence and the battery-only, vehicle, and end-to-end replays satisfy every configured terminal limit.' if high_precision_gate else 'One or more independent-evidence or replay checks failed; do not claim high-precision terminal SoC.'}` の結果を代入する。

23. `output_path.parent.mkdir(...)` を実行する。
24. `output_path.write_text(...)` を実行する。
25. `output_path` を返す。

代表コード断片

```
def write_terminal_consistency_from_anchor(
    output_path: Path,
    terminal_anchor: Dict[str, Any],
    *,
    max_spread: float,
    validation_metrics: Dict[str, Any] | None = None,
    replay_soc_error_max: float = 0.02,
    replay_voltage_error_max_v: float = 0.5,
    vehicle_soc_error_max: float = 0.02,
    vehicle_voltage_error_max_v: float = 0.5,
    end_to_end_soc_error_max: float = 0.03,
    end_to_end_voltage_error_max_v: float = 1.0,
) -> Path:
    """Always materialize the independent terminal-evidence gate.

    Detailed channel reconstruction may later enrich this file, but profile
    adoption must never silently drop the gate merely because a separate
    reporting command was not run.
    """
    lo = float(terminal_anchor.get("soc_evidence_min", float("nan")))
    hi = float(terminal_anchor.get("soc_evidence_max", float("nan")))
    target = float(terminal_anchor.get("soc_target", float("nan")))
    sigma = float(terminal_anchor.get("soc_sigma", float("nan")))
    spread = hi - lo if np.isfinite(lo) and np.isfinite(hi) else float("nan")
    metrics = validation_metrics or {}
    replay_soc_error = abs(
        float(metrics.get("battery_conditional_retire_anchor_soc_error", float("nan")))
    )
    replay_voltage_observed = float(
        metrics.get("battery_conditional_retire_anchor_voltage_obs_v", float("nan"))
    )
    replay_voltage_predicted = float(
        metrics.get("battery_conditional_retire_anchor_voltage_pred_v", float("nan"))
    )
    replay_voltage_error = abs(replay_voltage_predicted - replay_voltage_observed)
    ...
```

7.52 L2318 関数 `sync_canonical_fullsim_profile`

- 行範囲: L2318–L2350
- 所属: module 直下
- 定義: `sync_canonical_fullsim_profile(package_dir:Path,*,map_assets:Dict[str, Path],pv:PvFitResult,batt:BatteryFitResult,mot:MotionFitResult,observed_log_csv: Path,battery_dynamic_fit:Dict[str,Any]|None=None,solar_measurement_calibration: Dict[str,Any]|None=None)->Path|None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `apply_fit_to_cfg`, `exists`, `isinstance`, `open`, `safe_dump`, `safe_load`
- 戻り値の要点: `fullsim_yaml` / `None`

- 主なローカル代入名: `cfg`, `f`, `fullsim_yaml`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError(f'fullsim profile must be a mapping: {fullsim_yaml}')`
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `fullsim_yaml` に `package_dir / 'profile_fullsim_selflearned.yaml'` の結果を代入する。
2. 条件 `not fullsim_yaml.exists()` を判定し、真なら内部処理を行う。
3. `None` を返す。
4. `with` 文で `fullsim_yaml.open('r', encoding='utf-8')` を管理しながら処理する。
5. `cfg` に `yaml.safe_load(f)` or `{}` の結果を代入する。
6. 条件 `not isinstance(cfg, dict)` を判定し、真なら内部処理を行う。
7. `ValueError(f'fullsim profile must be a mapping: {fullsim_yaml}')` を送出する。
8. `cfg` に `apply_fit_to_cfg(cfg, package_dir=package_dir, map_assets=map_assets, pv=pv, batt=batt, mot=mot, battery_dynamic_fit=battery_dynamic_fit, solar_measurement_calibration=solar_measurement_calibration, observed_log_csv=observed_log_csv, sync_sim_soc0=False)` の結果を代入する。
9. `with` 文で `fullsim_yaml.open('w', encoding='utf-8', newline='\n')` を管理しながら処理する。
10. `yaml.safe_dump(...)` を実行する。
11. `fullsim_yaml` を返す。

代表コード断片

```
def sync_canonical_fullsim_profile(
    package_dir: Path,
    *,
    map_assets: Dict[str, Path],
    pv: PvFitResult,
    batt: BatteryFitResult,
    mot: MotionFitResult,
    observed_log_csv: Path,
    battery_dynamic_fit: Dict[str, Any] | None = None,
    solar_measurement_calibration: Dict[str, Any] | None = None,
) -> Path | None:
    fullsim_yaml = package_dir / "profile_fullsim_selflearned.yaml"
    if not fullsim_yaml.exists():
        return None
    with fullsim_yaml.open("r", encoding="utf-8") as f:
        cfg = yaml.safe_load(f) or {}
    if not isinstance(cfg, dict):
        raise ValueError(f"fullsim profile must be a mapping: {fullsim_yaml}")
    cfg = apply_fit_to_cfg(
```

```

    cfg,
    package_dir=package_dir,
    map_assets=map_assets,
    pv=pv,
    batt=batt,
    mot=mot,
    battery_dynamic_fit=battery_dynamic_fit,
    solar_measurement_calibration=solar_measurement_calibration,
    observed_log_csv=observed_log_csv,
    sync_sim_soc0=False,
)
with fullsim_yaml.open("w", encoding="utf-8", newline="\n") as f:
    yaml.safe_dump(cfg, f, sort_keys=False, allow_unicode=True)
return fullsim_yaml

```

7.53 L2353 関数 write_generic_summary

- 行範囲: L2353–L2419
- 所属: module 直下
- 定義: `write_generic_summary(package_dir:Path,manifest_path:Path,profile_yaml:Path, map_assets:Dict[str,Path],pv:PvFitResult,batt:BatteryFitResult,mot:MotionFitResult, metrics:Dict[str,float],terminal_anchor:Dict[str,float],stage_anchors:list[dict] ,map_shape_fit:Dict[str,object],post_refine:PostRefineResult,day_metrics:list[dict] ,battery_dynamic_fit:Dict[str,Any]|None=None,fit_plan:dict|None=None,manifest_ context:dict|None=None,output_dir:Path|None=None,replay_csv:Path|None=None, battery_conditioned_replay_csv:Path|None=None,end_to_end_replay_csv:Path|None= None)->Path`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `dict, ensure_dir, get, items, list, open, relpath, relpath_from, replace, safe_dump`
- 戻り値の要点: `out_path`
- 主なローカル代入名: `f, key, out_path, output_dir, path, payload`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `output_dir` に `output_dir` or `package_dir / 'outputs' / 'identification'` の結果を代入する。
2. `out_path` に `output_dir / f'{package_dir.name}_generic_fit_summary.yaml'` の結果を代入する。
3. `ensure_dir(...)` を実行する。
4. `payload` に `{'builder': 'generic_replay_mle', 'manifest_yaml': os.path.relpath(manifest_path, package_dir).replace('\', '/'), 'profile_yaml': os.path.relpath(profile_yaml, package_dir).replace('\', '/'), 'active_maps': {key: os.path.relpath(path, package_dir).replace('\', '/') for key, path in map_assets.items()}, 'pv_fit': pv.__dict__, 'battery_fit': batt.__dict__, 'battery_dynamic_fit': dict(battery_dynamic_fit or {}), 'motion_fit': mot.__dict__, 'validation_metrics': metrics, 'validation_protocol': {'vehicle_conditional_replay_csv': relpath_from(package_dir, replay_csv), 'battery_conditioned_replay_csv': relpath_from(package_dir, battery_conditioned_replay_csv), 'end_to_end_replay_csv': relpath_from(package_dir, end_to_end_replay_csv), 'vehicle_fit_solar_source': 'measured_when_available', 'battery_conditioned_source': 'observed_pack_power_and_current', 'end_to_end_solar_source': 'independent_GHI_archive_and_moving_PV_model', 'restart_soc_anchor': 'median_of_valid_stationary_window'}, 'terminal_anchor': terminal_anchor, 'stage_anchors': stage_anchors, 'day_metrics': day_metrics, 'map_shape_fit': map_shape_fit, 'post_refine': post_refine.__dict__, 'fit_plan': dict(fit_plan or {}), 'evidence_bundle': {'actual_event_yaml': relpath_from(package_dir, (manifest_context or {}).get('actual_event_path')), 'counterfactual_event_yaml': relpath_from(package_dir, (manifest_context or {}).get('counterfactual_event_path')), 'terminal_anchor_yaml': relpath_from(package_dir, (manifest_context or {}).get('terminal_anchor_path')), 'grounded_map_summary_yaml': relpath_from(package_dir, (manifest_context or {}).get('grounded_summary_path')), 'source_inventory_json': relpath_from(package_dir, (manifest_context or {}).get('source_inventory_path')), 'notes_markdown': relpath_from(package_dir, (manifest_context or {}).get('notes_markdown_path')), 'explicit_grounded_assets': {key: relpath_from(package_dir, path) for key, path in ((manifest_context or {}).get('explicit_grounded_assets') or {}).items()}, 'external_documents': list((manifest_context or {}).get('external_documents', []))}}` の結果を代入する。
5. `with` 文で `out_path.open('w', encoding='utf-8', newline='\n')` を管理しながら処理する。
6. `yaml.safe_dump(...)` を実行する。
7. `out_path` を返す。

代表コード断片

```
def write_generic_summary(
    package_dir: Path,
    manifest_path: Path,
    profile_yaml: Path,
    map_assets: Dict[str, Path],
    pv: PvFitResult,
    batt: BatteryFitResult,
    mot: MotionFitResult,
    metrics: Dict[str, float],
    terminal_anchor: Dict[str, float],
    stage_anchors: list[dict],
    map_shape_fit: Dict[str, object],
    post_refine: PostRefineResult,
    day_metrics: list[dict],
    battery_dynamic_fit: Dict[str, Any] | None = None,
    fit_plan: dict | None = None,
    manifest_context: dict | None = None,
    output_dir: Path | None = None,
    replay_csv: Path | None = None,
    battery_conditioned_replay_csv: Path | None = None,
    end_to_end_replay_csv: Path | None = None,
) -> Path:
    output_dir = output_dir or package_dir / "outputs" / "identification"
    out_path = output_dir / f'{package_dir.name}_generic_fit_summary.yaml'
    ensure_dir(out_path.parent)
```

```

payload = {
    "builder": "generic_replay_mle",
    "manifest_yaml": os.path.relpath(manifest_path, package_dir).replace("\\", "/"),
    "profile_yaml": os.path.relpath(profile_yaml, package_dir).replace("\\", "/"),
    "active_maps": {key: os.path.relpath(path, package_dir).replace("\\", "/") for key,
path in map_assets.items()}},
    "pv_fit": pv.__dict__,
    "battery_fit": batt.__dict__,
    "battery_dynamic_fit": dict(battery_dynamic_fit or {}),
    "motion_fit": mot.__dict__,
    "validation_metrics": metrics,
...

```

7.54 L2422 関数 write_generic_report

- 行範囲: L2422–L3072
- 所属: module 直下
- 定義: `write_generic_report(package_dir:Path,profile_yaml:Path,manifest_path:Path,summary_yaml:Path,observed_log_csv:Path,pv:PvFitResult,batt:BatteryFitResult,mot:MotionFitResult,metrics:Dict[str,float],post_refine:PostRefineResult,map_assets:Dict[str,Path],*,terminal_anchor:Dict[str,float]|None=None,stage_anchors:list[dict]|None=None,day_metrics:list[dict]|None=None,battery_dynamic_fit:Dict[str,Any]|None=None,fit_plan:dict|None=None,grounded_map_summary:Dict[str,object]|None=None,manifest_context:dict|None=None,terminal_consistency:Dict[str,Any]|None=None,report_dir:Path|None=None,current_maps_path:Path|None=None)->Tuple[Path,Path]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `bool, compile_tex, dedent, ensure_dir, enumerate, float, get, int, items, join, len, read_text`
- 戻り値の要点: `(md_path, pdf_path)`
- 主なローカル代入名: `anchor, battery_dynamic_fit, current_maps_path, current_maps_rel_package, current_maps_rel_report, day_metrics, evidence_rows, explicit_grounded_assets, external_documents, fit_plan, grade_observation_fit, grounded_yaml, idx, item, key, manifest_context, md, md_path, pack_voltage_limit_v, path`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `report_dir` に `report_dir` or `package_dir / 'outputs' / 'reports'` の結果を代入する。
2. `ensure_dir(...)` を実行する。
3. `md_path` に `report_dir / f'{package_dir.name}_generic_identification_report.md'` の結果を代入する。
4. `tex_path` に `report_dir / f'{package_dir.name}_generic_identification_report.tex'` の結果を代入する。
5. `pdf_path` に `tex_path.with_suffix('.pdf')` の結果を代入する。
6. `rel_maps` に `{key: os.path.relpath(path, package_dir).replace('\\', '/') for key, path in map_assets.items()}` の結果を代入する。
7. `terminal_anchor` に `terminal_anchor` or `{}` の結果を代入する。
8. `stage_anchors` に `stage_anchors` or `[]` の結果を代入する。
9. `day_metrics` に `day_metrics` or `[]` の結果を代入する。
10. `battery_dynamic_fit` に `battery_dynamic_fit` or `{}` の結果を代入する。
11. `fit_plan` に `fit_plan` or `{}` の結果を代入する。
12. `solar_calibration` に `fit_plan.get('solar_measurement_calibration', {})` or `{}` の結果を代入する。
13. `grade_observation_fit` に `fit_plan.get('grade_observation_fit', {})` or `{}` の結果を代入する。
14. `solar_gain_to_pack` に `float(solar_calibration.get('gain_to_pack', 1.0))` の結果を代入する。
15. `solar_calibration_samples` に `int(solar_calibration.get('sample_count', 0) or 0)` の結果を代入する。
16. `solar_calibration_accepted` に `bool(solar_calibration.get('accepted', False))` の結果を代入する。
17. `solar_calibration_intercept_w` に `float(solar_calibration.get('free_intercept_w', float('nan')))` の結果を代入する。
18. `solar_calibration_intercept_error_w` に `float(solar_calibration.get('free_intercept_error_w', float('nan')))` の結果を代入する。
19. `solar_calibration_daily_std` に `float(solar_calibration.get('daily_gain_std', float('nan')))` の結果を代入する。
20. `profile_cfg` に `yaml.safe_load(profile_yaml.read_text(encoding='utf-8-sig'))` or `{}` の結果を代入する。
21. `profile_model` に `profile_cfg.get('model', {})` or `{}` の結果を代入する。
22. `pack_voltage_limit_v` に `float(profile_model.get('V_max', BATTERY_PACK_MAX_CHARGE_V))` の結果を代入する。
23. `grounded_yaml` に `str((grounded_map_summary or {}).get('summary_yaml', '') or '')` の結果を代入する。
24. `manifest_context` に `manifest_context` or `{}` の結果を代入する。
25. `terminal_consistency` に `terminal_consistency` or `{}` の結果を代入する。
26. `terminal_checks` に `terminal_consistency.get('high_precision_checks', {})` or `{}` の結果を代入する。
27. `terminal_gate_pass` に `bool(terminal_consistency.get('high_precision_gate_pass', False))` の結果を代入する。
28. `terminal_evidence_min` に `float(terminal_consistency.get('evidence_interval_min', float('nan')))` の結果を代入する。
29. `terminal_evidence_max` に `float(terminal_consistency.get('evidence_interval_max', float('nan')))` の結果を代入する。

30. `terminal_evidence_spread_pp` に `float(terminal_consistency.get('spread_percentage_points', float('nan')))` の結果を代入する。
31. `terminal_interpretation` に `str(terminal_consistency.get('interpretation', '') or '')` の結果を代入する。
32. `evidence_rows` に `[('actual_event_yaml', relpath_from(package_dir, manifest_context.get('actual_event_path'))), ('counterfactual_event_yaml', relpath_from(package_dir, manifest_context.get('counterfactual_event_path'))), ('terminal_anchor_yaml', relpath_from(package_dir, manifest_context.get('terminal_anchor_path'))), ('grounded_map_summary_yaml', relpath_from(package_dir, manifest_context.get('grounded_summary_path')) or grounded_yaml), ('source_inventory_json', relpath_from(package_dir, manifest_context.get('source_inventory_path'))), ('notes_markdown', relpath_from(package_dir, manifest_context.get('notes_markdown_path')))]` の結果を代入する。
33. `explicit_grounded_assets` に `{key: relpath_from(package_dir, path) for key, path in (manifest_context.get('explicit_grounded_assets', {})).items()}` の結果を代入する。
34. `external_documents` に `[str(item) for item in manifest_context.get('external_documents', []) if str(item).strip()]` の結果を代入する。
35. `current_maps_path` に `current_maps_path or report_dir / 'current_maps_and_coefficients.md'` の結果を代入する。
36. `profile_rel` に `os.path.relpath(profile_yaml, package_dir).replace('\\', '/')` の結果を代入する。
37. `current_maps_rel_package` に `os.path.relpath(current_maps_path, package_dir).replace('\\', '/')` の結果を代入する。
38. `current_maps_rel_report` に `os.path.relpath(current_maps_path, report_dir).replace('\\', '/')` の結果を代入する。
39. `md` に `f'# {package_dir.name} generic identification report\n\n## Inputs\n\n profile: {profile_rel}\n\nmanifest: {os.path.relpath(manifest_path, package_dir).replace('\\', '/')}\n\n normalized replay log: {os.path.relpath(observed_log_csv, package_dir).replace('\\', '/')}\n\n## Adopted coefficients\n\npanel_gain: {pv.panel_gain:.6f}\n\n tccl_gain_c_per_wm2: {pv.tccl_gain_c_per_wm2:.6f}\n\n pv_irradiance_source: {pv.irradiance_source}\n\n pv_operating_state: {pv.operating_state}\n\n pv_sample_count: {pv.sample_count}\n\n stop_tilt_fraction: {pv.stop_tilt_fraction:.6f}\n\n stop_solar_rmse_w: {pv.stop_solar_rmse_w:.3f}\n\n solar_measurement_gain_to_pack: {solar_gain_to_pack:.8f}\n\n solar_measurement_calibration_accepted: {solar_calibration_accepted}\n\n solar_measurement_calibration_samples: {solar_calibration_samples}\n\n solar_measurement_free_intercept_w: {solar_calibration_intercept_w:.6f}\n\n solar_measurement_intercept_error_w: {solar_calibration_intercept_error_w:.6f}\n\n solar_measurement_daily_gain_std: {solar_calibration_daily_std:.8f}\n\n soc0: {batt.soc0:.6f}\n\n E_nom_Wh: {batt.e_nom_wh:.3f}\n\n Q_nom_Ah: {batt.e_nom_wh / BATTERY_NOMINAL_VOLTAGE_V:.6f}\n\n rint_scale: {batt.rint_scale:.6f}\n\n r_line_ohm: {batt.r_line_ohm:.6f}\n\n r_polarization_ohm: {float(battery_dynamic_fit.get('r_polarization_ohm', 0.0)):.6f}\n\n polarization_tau_sec: {float(battery_dynamic_fit.get('tau_sec', 60.0)):.3f}\n\n eta_charge: {batt.eta_charge:.6f}\n\n CdA: {mot.cda:.6f}\n\n Crr: {mot.crr:.6f}\n\n P_aux_w: {mot.p_aux_w:.3f}\n\n grade_scale: {mot.grade_scale:.6f}\n\n drive_eff_scale: {mot.drive_eff_scale:.6f}\n\n regen_utilization: {mot.regen_utilization:.6f}\n\n regen_sample_count: {mot.regen_sample_count}\n\n regen_fit_rmse_w: {mot.regen_fit_rmse_w:.3f}\n\n headwind_gain: {mot.headwind_gain:.6f}\n\n V_max_v: {pack_voltage_limit_v:.3f}\n\n\n## Solar measurement calibration\n\nThe ZP solar channel is calibrated against the stationary DC-bus balance before\nvehicle or PV fitting. For samples with zero traction power, the fitted model is\n $P_{batt,k} = P_{aux} - g_{solar} * P_{solar,raw,k} + \epsilon_k$. The adopted gain is\nthe bounded Huber M-estimator\n $\argmin_{g_{solar}} (0.70 \leq g_{solar} \leq 1.05) \sum_k \rho_H(P_{batt,k} - P_{aux} + g_{solar} * P_{solar,raw,k})$. The known 21 W auxiliary load fixes the intercept; a separate free-intercept fit\nis retained only as a consistency diagnostic. Daily gain spread checks whether\na single gain is defensible across the race.\n\nThe telemetry contract is deliberately one-way: the sender transmits the raw ZP\nsolar_power_w value, and the receiving WiFi bridge multiplies it by the active\nprofile's solar_measurement_gain_to_pack exactly once. Corrected values must not\nbe sent through that raw field, which prevents double calibration.\n\nThe battery terminal-voltage ceiling is a hardware constraint, not a value fitted\nfrom a loaded discharge trace. For YATA the adopted {pack_voltage_limit_v:.3f} V\nmust equal the`

product limit '25 series cells * 4.35 V/cell = 108.75 V', and the inactive OCV map must not exceed it. The package audit checks both directions.

Validation

Vehicle coefficients are identified conditionally on measured array power so forecast/PV error cannot be absorbed into CdA, Crr, or drivetrain efficiency. The unprefixed metrics below are that conditional vehicle replay. Keys beginning with 'end_to_end_' use independent archive GHI plus the moving-PV model and therefore include forecast/PV error. 'GHI_effective' is target-derived diagnostic data and is prohibited from both fitting and end-to-end validation. Both datasets are retained; neither is substituted for the other.

Keys beginning with 'battery_conditional_' reuse observed pack power/current. They validate the battery submodel only and cannot certify vehicle energy use or the 2831 km vehicle-model terminal SoC. Promotion therefore requires separate terminal gates for the unprefixed vehicle replay and the end-to-end replay.

The field-analysis source states that the ZP logger start time was not recorded and was reconstructed backward from control-stop times. Therefore 5 s samples do not provide a traceable pointwise synchronization certificate among power, speed, and DEM grade. Parameter fitting uses the configured resampling window; the 10 km and 25 km energy residuals are the primary route-energy checks. A low pointwise RMSE must not be manufactured by fitting a time shift to this unknown.

The 5 s RMSE, 120 s mean-residual RMSE, and distance-window energy RMSE answer different questions. The 5 s value includes unresolved channel synchronization and transient noise. The 120 s value tests local mean power after aggregation. The 10/25 km values test the accumulated energy that drives long-horizon SoC. None may be relabelled as another metric, and operational promotion requires all configured gates rather than choosing only the smallest number.

DEM grade is treated as an observation model rather than a vehicle constant. Candidate Savitzky-Golay elevation smoothing lengths and route-distance offsets are selected on race days other than the held-out final day. The adopted route stores the unscaled differentiated elevation; 'grade_scale' is fitted only after that route is locked. This prevents DEM vertical noise from being hidden inside CdA, Crr, or drivetrain efficiency.

Regeneration is separated into conversion efficiency and use: $P_{\text{reg,dc}} = u_{\text{regen}} * \eta_{\text{reg}} * \eta_{\text{gear}} * \eta_{\text{inv}} * \max(-P_{\text{mech}}, 0)$. The bounded scalar 'u_regen' is fitted only where negative mechanical power is unobservable and is never folded into the motor efficiency map.

power_rmse_clean_w: '{metrics.get('power_rmse_clean_w', float('nan')):.3f}'
power_rmse_fit_window_w: '{metrics.get('power_rmse_fit_window_w', float('nan')):.3f}'
voltage_rmse_clean_v: '{metrics.get('voltage_rmse_clean_v', float('nan')):.3f}'
final_soc_pred: '{metrics.get('final_soc_pred', float('nan')):.6f}'
retire_anchor_soc_obs: '{metrics.get('retire_anchor_soc_obs', float('nan')):.6f}'
retire_anchor_soc_pred: '{metrics.get('retire_anchor_soc_pred', float('nan')):.6f}'
retire_anchor_soc_error: '{metrics.get('retire_anchor_soc_error', float('nan')):.6f}'
battery_conditional_power_rmse_clean_w: '{metrics.get('battery_conditional_power_rmse_clean_w', float('nan')):.3f}'
battery_conditional_voltage_rmse_clean_v: '{metrics.get('battery_conditional_voltage_rmse_clean_v', float('nan')):.3f}'
battery_conditional_retire_anchor_soc_error: '{metrics.get('battery_conditional_retire_anchor_soc_error', float('nan')):.6f}'
end_to_end_power_rmse_clean_w: '{metrics.get('end_to_end_power_rmse_clean_w', float('nan')):.3f}'
end_to_end_voltage_rmse_clean_v: '{metrics.get('end_to_end_voltage_rmse_clean_v', float('nan')):.3f}'
end_to_end_final_soc_pred: '{metrics.get('end_to_end_final_soc_pred', float('nan')):.6f}'
end_to_end_retire_anchor_soc_error: '{metrics.get('end_to_end_retire_anchor_soc_error', float('nan')):.6f}'
end_to_end_moving_pv_rmse_w: '{metrics.get('end_to_end_moving_pv_rmse_w', float('nan')):.3f}'
end_to_end_deployed_stop_pv_rmse_w: '{metrics.get('end_to_end_deployed_stop_pv_rmse_w', float('nan')):.3f}'
pv_lodo_moving_rmse_w: '{metrics.get('pv_lodo_moving_rmse_w', float('nan')):.3f}'
pv_lodo_deployed_stop_rmse_w: '{metrics.get('pv_lodo_deployed_stop_rmse_w', float('nan')):.3f}'
pv_lodo_fold_count: '{metrics.get('pv_lodo_fold_count', 0)}'
power_residual_mean_120s_rmse_w: '{metrics.get('power_residual_mean_120s_rmse_w', float('nan')):.3f}'
energy_error_10km_rmse_wh: '{metrics.get('energy_error_10km_rmse_wh', float('nan')):.3f}'
energy_error_25km_rmse_wh: '{metrics.get('energy_error_25km_rmse_wh', float('nan')):.3f}'

Terminal-SoC evidence and certification

evidence_interval: '[{terminal_evidence_min:.6f}, {terminal_evidence_max:.6f}]'
evidence_spread_percentage_points: '{terminal_evidence_spread_pp:.3f}'
high_precision_gate_pass: '{terminal_gate_pass}' + ('n'.join((f'- {key}:' + '{bool(value)}') for key, value in terminal_checks.items())) if terminal_checks else '- high_precision_checks: (not evaluated)'
interpretation: '{terminal_interpretation}'

The central terminal-SoC estimate is not a direct coulomb-counter measurement. If the evidence spread or any replay check fails, the value is retained as an engineering anchor only; this report does not certify a high-precision actual 2831 km SoC and the candidate must not replace the operational profile.

Method references

Open-Meteo Historical Weather API: <https://open-meteo.com/en/docs/historical-weather-api>
Sandia PVPMP plane-of-array irradiance guidance: <https://pvpmc.sandia.gov/modeling-guide/1-weather->

design-inputs/plane-of-array-poa-irradiance/\n- pvlib total-irradiance transposition API: https://pvlib-python.readthedocs.io/en/latest/reference/generated/pvlib.irradiance.get_total_irradiance.html\n- Byrd et al. (1995), L-BFGS-B: <https://doi.org/10.1137/0916069>\n- Liu et al. (2014), GPS road-grade quantification: <https://doi.org/10.1016/j.atmosenv.2013.12.025>\n- Wang et al. (2018), synchronized link-level EV energy data: <https://escholarship.org/uc/item/2bp8x04q>\n\n## Terminal anchor\n\n- anchor_s_km: '{terminal_anchor.get('s_km', float('nan'))}'\n- anchor_time_utc: '{terminal_anchor.get('time_utc', ' ')}'\n- anchor_voltage_v: '{terminal_anchor.get('voltage_v', float('nan'))}'\n- anchor_current_a: '{terminal_anchor.get('current_a', float('nan'))}'\n- anchor_temp_c: '{terminal_anchor.get('temp_c', float('nan'))}'\n- anchor_soc_target: '{terminal_anchor.get('soc_target', float('nan'))}'\n\n## Fit plan\n\n- quality: '{fit_plan.get('quality', ' ')}'\n- battery_restart_count: '{fit_plan.get('battery_restart_count', ' ')}'\n- battery_maxiter: '{fit_plan.get('battery_maxiter', ' ')}'\n- motion_restart_count: '{fit_plan.get('motion_restart_count', ' ')}'\n- motion_maxiter: '{fit_plan.get('motion_maxiter', ' ')}'\n- joint_restart_count: '{fit_plan.get('joint_restart_count', ' ')}'\n- joint_random_start_count: '{fit_plan.get('joint_random_start_count', ' ')}'\n- joint_local_topk: '{fit_plan.get('joint_local_topk', ' ')}'\n- joint_maxiter: '{fit_plan.get('joint_maxiter', ' ')}'\n- fit_stride: '{fit_plan.get('fit_stride', ' ')}'\n- allow_map_shape_fit: '{fit_plan.get('allow_map_shape_fit', ' ')}'\n- post_refine_enabled: '{fit_plan.get('post_refine_enabled', ' ')}'\n- terminal_anchor_role: '{fit_plan.get('terminal_anchor_role', ' ')}'\n- panel_deployment_stopped_speed_kmh: '{fit_plan.get('panel_deployment_stopped_speed_kmh', ' ')}'\n- panel_deployment_min_dwell_sec: '{fit_plan.get('panel_deployment_min_dwell_sec', ' ')}'\n- panel_deployment_max_sample_gap_sec: '{fit_plan.get('panel_deployment_max_sample_gap_sec', ' ')}'\n- grade_observation_adopted: '{bool(grade_observation_fit.get('adopted', False))}'\n- grade_smoothing_window_km: '{grade_observation_fit.get('selected_smoothing_window_km', ' ')}'\n- grade_distance_offset_km: '{grade_observation_fit.get('distance_offset_km', ' ')}'\n- grade_training_rmse_w: '{grade_observation_fit.get('baseline_training_rmse_w', float('nan'))}'\n- grade_validation_rmse_w: '{grade_observation_fit.get('baseline_validation_rmse_w', float('nan'))}'\n- grade_holdout_rmse_w: '{grade_observation_fit.get('selected_validation_rmse_w', float('nan'))}'\n\n## Stage anchors\n\n- stage_anchor_count: '{len(stage_anchors)}'\n- stage_anchor_{idx:02d}: 'time={anchor.get('time_utc', ' ')}, s_km={anchor.get('s_km', float('nan'))}, V={anchor.get('voltage_v', float('nan'))}, I={anchor.get('current_a', float('nan'))}, SoC={anchor.get('soc_target', float('nan'))}, dwell_sec={anchor.get('dwell_sec', float('nan'))}'\n\nfor idx, anchor in enumerate(stage_anchors, start=1)) if stage_anchors else ' - (none)') + f'\n\n## Day metrics\n\n- day_{row.get('day', ' ')}: 'dist_end={row.get('distance_end_km', float('nan')):.1f} km, final_soc={row.get('final_soc_pred', float('nan')):.4f}, power_rmse={row.get('power_rmse_clean_w', float('nan')):.2f} W, voltage_rmse={row.get('voltage_rmse_clean_v', float('nan')):.3f} V, excluded_power={row.get('excluded_power_points', 0)}, excluded_voltage={row.get('excluded_voltage_v', float('nan')):.3f} V'\n\nfor row in day_metrics)) if day_metrics else ' - (none)') + f'\n\n## Evidence bundle\n\n- evidence_{key}: '{value}'\n\nfor key, value in evidence_rows if value)) + ('' + 'n'.join((f'- explicit_{key}: {value}' for key, value in explicit_grouped_assets.items())) if explicit_grouped_assets else '') + ('' + 'n'.join((f'- external_document: {value}' for value in external_documents)) if external_documents else '') + f'\n\n## Active maps\n\n- map_{key}: '{value}'\n\nfor key, value in rel_maps.items())) + f'\n\n## Grounded map provenance\n\n- grounded_map_summary_yaml: '{grounded_yaml}'\n\n## Post refinement\n\n- accepted: '{post_refine.accepted}'\n- panel_gain_factor: '{post_refine.panel_gain_factor:.5f}'\n- cda_factor: '{post_refine.cda_factor:.5f}'\n- crr_factor: '{post_refine.crr_factor:.5f}'\n- drive_eff_factor: '{post_refine.drive_eff_factor:.5f}'\n- headwind_gain_factor: '{post_refine.headwind_gain_factor:.5f}'\n- e_nom_factor: '{post_refine.e_nom_factor:.5f}'\n- rint_factor: '{post_refine.rint_factor:.5f}'\n\n## Outputs\n\n- fit_summary: '{os.path.relpath(summary_yaml, package_dir).replace("\\\\", '/')}'\n- current_maps_and_coefficients: '{current_maps_rel_package}'\n\nの結果を代入する。

40. md_path.write_text(...) を実行する。

41. tex に f'\n\ndocumentclass[a4paper,11pt]{article}\n\n\usepackage[top=18mm,bottom=22mm,left=18mm,right=18mm]{New Roman}\n\n\setCJKmainfont{Yu Gothic}\n\n\setCJKmonofont{Yu Gothic}\n\n\usepackage{booktabs}\n\n\usepackage{Vehicle Identification Report}\n\n\author{solar_ws0129-main}\n\n\date{\n\n\begin{document}\n\n\raggedbottom\n\n\item profile: \path{project_packages/{package_dir.name}/profile.yaml}\n\n\item manifest: \path{{os.path.relpath(manifest_dir, report_dir).replace("\\\\", '/')}}\n\n\item normalized replay log: \path{{os.path.relpath(observed_log_csv, report_dir).replace("\\\\", '/')}}\n\n\end{document}\n\n\section{Adopted coefficients}\n\n\begin{longtable}\n\n\caption{項目 & 値}\n\n\midrule\n\n\endhead\n\n\panel gain & {pv.panel_gain:.6f}\n\n\ntcell gain [C/(W m\$^{-2}\$)] & {pv.tcell_gain_c_per_wm2:.6f}\n\n\ntstop tilt fraction [-] & {pv.stop_tilt_fraction:.6f}\n\n\ntstop solar RMSE [W] & {pv.stop_solar_rmse_w:.3f}\n\n\ntsolar measurement gain to pack & {solar_gain_to_pack:.8f}\n\n\ntsolar calibration samples & {solar_calibration_samples}\n\n\ntsolar free-intercept diagnostic [W] &

float('nan'))):.3f} \\\nPV leave-one-day-out folds [-] & {int(metrics.get('pv_lodo_fold_count', 0))} \\\n120 s mean-residual RMSE [W] & {float(metrics.get('power_residual_mean_120s_rmse_w', float('nan'))):.3f} \\\n10 km energy-error RMSE [Wh] & {float(metrics.get('energy_error_10km_rmse_wh', float('nan'))):.3f} \\\n25 km energy-error RMSE [Wh] & {float(metrics.get('energy_error_25km_rmse_wh', float('nan'))):.3f} \\\n\n\\bottomrule\n\\end{{longtable}}\n\n\\section{{Terminal-SoC evidence and certification}}\n\nThe independent evidence interval is\n\$[{{terminal_evidence_min:.6f}},{{terminal_evidence_max:.6f}}]\$, with width\n{{terminal_evidence_spread_pp:.3f}} percentage points. The complete\nhigh-precision gate result is \\\ntextbf{{{{terminal_gate_pass}}}}.\n\n\\begin{{longtable}}{{p{{0.58\\linewidth}}p{{0.18\\linewidth}}}}\n\\toprule\nch & pass \\\n\n\\midrule\n\\endhead\n" + 'n'.join((f'{{tex_path_fragment(str(key))}} & {bool(value)} \\\n' for key, value in terminal_checks.items())) + f"\n\n\\bottomrule\n\\end{{longtable}}\n\n\\tex_text_fragment(terminal_interpretation)\n\nThe central estimate is not a\nndirect coulomb-counter observation. If this gate is false, it remains an\nengineering anchor only; neither a high-precision actual 2831 km SoC nor\noperational model certification is claimed.\n\n\\section{{Method references}}\n\n\\begin{{itemize}}\n\\item Open-Meteo Historical Weather API: \\\nurl{{https://open-meteo.com/en/docs/historical-weather-api}}\n\\item Sandia PVPMP plane-of-array irradiance guidance: \\\nurl{{https://pvpmc.sandia.gov/modeling-guide/1-weather-design-inputs/plane-of-array-poa-irradiance/}}\n\\item pvlib total-irradiance transposition API: \\\nurl{{https://pvlib-python.readthedocs.io/en/latest/reference/generated/pvlib.irradiance.get_total_irradiance.html}}\n\\item Byrd et al. (1995), L-BFGS-B: \\\nurl{{https://doi.org/10.1137/0916069}}\n\\item Liu et al. (2014), GPS road-grade quantification: \\\nurl{{https://doi.org/10.1016/j.atmosenv.2013.12.025}}\n\\item Wang et al. (2018), synchronized link-level EV energy data: \\\nurl{{https://escholarship.org/uc/item/2bp8x04q}}\n\\end{{itemize}}\n\n\\section{{Terminal anchor}}\n\n\\begin{{longtable}}{{p{{0.48\\linewidth}}p{{0.32\\linewidth}}}}\n\\toprule\n项目 & 值 \\\n\n\\midrule\n\\endhead\n\nanchor distance [km] & {float(terminal_anchor.get('s_km', float('nan'))):.3f} \\\n\nanchor time UTC & \\\npath{{str(terminal_anchor.get('time_utc', ''))}} \\\n\nanchor voltage [V] & {float(terminal_anchor.get('voltage_v', float('nan'))):.3f} \\\n\nanchor current [A] & {float(terminal_anchor.get('current_a', float('nan'))):.3f} \\\n\nanchor temperature [C] & {float(terminal_anchor.get('temp_c', float('nan'))):.3f} \\\n\nanchor SoC target [-] & {float(terminal_anchor.get('soc_target', float('nan'))):.6f} \\\n\n\\bottomrule\n\\end{{longtable}}\n\nplan}}\n\n\\begin{{itemize}}\n\\item quality: {str(fit_plan.get('quality', ''))}\n\\item battery restart count: {fit_plan.get('battery_restart_count', '')}\n\\item battery maxiter: {fit_plan.get('battery_maxiter', '')}\n\\item motion restart count: {fit_plan.get('motion_restart_count', '')}\n\\item motion maxiter: {fit_plan.get('motion_maxiter', '')}\n\\item joint restart count: {fit_plan.get('joint_restart_count', '')}\n\\item joint random start count: {fit_plan.get('joint_random_start_count', '')}\n\\item joint local topk: {fit_plan.get('joint_local_topk', '')}\n\\item joint maxiter: {fit_plan.get('joint_maxiter', '')}\n\\item fit stride: {fit_plan.get('fit_stride', '')}\n\\item allow map-shape fit: {fit_plan.get('allow_map_shape_fit', '')}\n\\item post-refine enabled: {fit_plan.get('post_refine_enabled', '')}\n\\item terminal anchor role: \\\npath{{str(fit_plan.get('terminal_anchor_role', ''))}}\n\\item panel deployment stopped speed: {fit_plan.get('panel_deploy_stopped_speed', '')} km/h\n\\item panel deployment minimum dwell: {fit_plan.get('panel_deployment_min_dwell_sec', '')} s\n\\item panel deployment maximum sample gap: {fit_plan.get('panel_deployment_max_sample_gap_sec', '')} s\n\\item acceleration observation filter/alignment adopted: {bool((fit_plan.get('acceleration_observation_fit', {}) or {}).get('adopted', False))}\n\\item acceleration selected filter: \\\npath{{str((fit_plan.get('acceleration_observation_fit', {}) or {}).get('selected_filter_method', 'legacy'))}}\n, window={{(fit_plan.get('acceleration_observation_fit', {}) or {}).get('selected_filter_window_samples', '')} samples / ((fit_plan.get('acceleration_observation_fit', {}) or {}).get('selected_filter_window_sec', ''))} s\n\\item acceleration timestamp selected lag: {float((fit_plan.get('acceleration_observation_fit', {}) or {}).get('selected_lag_sec', float('nan'))):.3f} s\n\\item acceleration alignment train RMSE: {float((fit_plan.get('acceleration_observation_fit', {}) or {}).get('baseline_training_rmse_w', float('nan'))):.3f} \$\rightrightarrows\$ {float((fit_plan.get('acceleration_observation_fit', {}) or {}).get('selected_training_rmse_w', float('nan'))):.3f} W\n\\item acceleration alignment held-out RMSE: {float((fit_plan.get('acceleration_observation_fit', {}) or {}).get('baseline_validation_rmse_w', float('nan'))):.3f} \$\rightrightarrows\$ {float((fit_plan.get('acceleration_observation_fit', {}) or {}).get('selected_validation_rmse_w', float('nan'))):.3f} W\n\\item acceleration held-out RMSE ratio: {float((fit_plan.get('acceleration_observation_fit', {}) or {}).get('validation_rmse_ratio', float('nan'))):.6f}\n\\item DEM grade observation adopted: {bool(grade_observation_fit.get('adopted', False))}\n\\item DEM grade smoothing window: {grade_observation_fit.get('selected_smoothing_window_km', '')} km\n\\item DEM grade distance offset: {grade_observation_fit.get('selected_distance_offset_km', '')} km\n\\item DEM grade train RMSE: {float(grade_observation_fit.get('baseline_training_rmse_w', float('nan'))):.3f} \$\rightrightarrows\$ {float(grade_observation_fit.get('selected_training_rmse_w', float('nan'))):.3f} W\n\\item DEM grade held-out RMSE: {float(grade_observation_fit.get('baseline_validation_rmse_w', float('nan'))):.3f}

```

 $\rightarrow$  {float(grade_observation_fit.get('selected_validation_rmse_w', float('nan'))):.3f} W\n\\end{{itemize}}\n\\
anchors}}\n\\begin{{itemize}}\n \item stage anchor count: {len(stage_anchors)}\n" + ('\n' + '\n'.join((f'
\item anchor {idx:02d}: time=\path{{{str(anchor.get('time_utc', ''))}}}, s={float(anchor.get('s_km',
float('nan'))):.3f} km, V={float(anchor.get('voltage_v', float('nan'))):.3f} V, I={float(anchor.get('current_a',
float('nan'))):.3f} A, SoC={float(anchor.get('soc_target', float('nan'))):.6f}, dwell={float(anchor.get('dwell_sec',
float('nan'))):.1f} s" for idx, anchor in enumerate(stage_anchors, start=1))) if stage_anchors else '\n \item
(none)') + f'\n\\end{{itemize}}\n\n\\section{{Day metrics}}\n\\begin{{longtable}}>{{{{\\raggedright\\arraybackslash}}
& dist end [km] & final SoC & power RMSE [W] & voltage RMSE [V] & excl. power \\midrule\n\\endhead\n'
+ '\n'.join((f'{int(row.get('day', -1))} & {float(row.get('distance_end_km', float('nan'))):.1f} & {float(row.get('final_soc',
float('nan'))):.4f} & {float(row.get('power_rmse_clean_w', float('nan'))):.2f} & {float(row.get('voltage_rmse_clean_v',
float('nan'))):.3f} & {int(row.get('excluded_power_points', 0))} \\midrule\n\\end{{longtable}}\n\\end{{itemize}}\n'
for row in day_metrics))) + f'\n\\bottomrule\n\\end{{longtable}}\n\\end{{itemize}}\n' + '\n'.join((f' \item {tex_path_fragment(key)}: {tex_path_fragment(value)}'
for key, value in evidence_rows if value)) + ('\n' + '\n'.join((f' \item explicit {tex_path_fragment(key)}:
{tex_path_fragment(value)}' for key, value in explicit_grounded_assets.items())) if explicit_grounded_assets
else '') + ('\n' + '\n'.join((f' \item external document: {tex_path_fragment(value)}' for value in external_documents)) if external_documents else '') + f'\n\\end{{itemize}}\n\n\\section{{Active maps}}\n\\begin{{itemize}}\n'
+ '\n'.join((f' \item {tex_path_fragment(key)}: {tex_path_fragment(value)}' for key, value in rel_maps.items()))
+ f'\n\\end{{itemize}}\n\n\\section{{Grounded map provenance}}\n\\begin{{itemize}}\n \item grounded map
summary yaml: \path{{{grounded_yaml}}}\n\\end{{itemize}}\n\n\\section{{Post refinement}}\n\\begin{{longtable}}>{{
項目 & 値 \\midrule\n\\endhead\naccepted & {post_refine.accepted} \\midrule\npanel gain factor &
{post_refine.panel_gain_factor:.5f} \\midrule\nCdA factor & {post_refine.cda_factor:.5f} \\midrule\nCrr factor &
{post_refine.crr_factor:.5f} \\midrule\ndrive efficiency factor & {post_refine.drive_eff_factor:.5f} \\midrule\nheadwind
factor & {post_refine.headwind_gain_factor:.5f} \\midrule\nSE_{{nom}}$ factor & {post_refine.e_nom_factor:.5f}
\\midrule\nSR_{{int}}$ factor & {post_refine.rint_factor:.5f} \\midrule\n\\bottomrule\n\\end{{longtable}}\n\n\\section{{Outputs}}\n\\end{{itemize}}\n'
\item summary YAML: \path{{{os.path.relpath(summary_yaml, report_dir).replace('\\', '/')}}}\n \item cur-
rent maps and coefficients: \path{{{current_maps_rel_report}}}\n\\end{{itemize}}\n\n\\end{{document}}\n"
の結果を代入する。

```

42. `tex_path.write_text(...)` を実行する。

43. `compile_tex(...)` を実行する。

44. `(md_path, pdf_path)` を返す。

代表コード断片

```

def write_generic_report(
    package_dir: Path,
    profile_yaml: Path,
    manifest_path: Path,
    summary_yaml: Path,
    observed_log_csv: Path,
    pv: PvFitResult,
    batt: BatteryFitResult,
    mot: MotionFitResult,
    metrics: Dict[str, float],
    post_refine: PostRefineResult,
    map_assets: Dict[str, Path],
    *,
    terminal_anchor: Dict[str, float] | None = None,
    stage_anchors: list[dict] | None = None,
    day_metrics: list[dict] | None = None,
    battery_dynamic_fit: Dict[str, Any] | None = None,
    fit_plan: dict | None = None,
    grounded_map_summary: Dict[str, object] | None = None,
    manifest_context: dict | None = None,
    terminal_consistency: Dict[str, Any] | None = None,
    report_dir: Path | None = None,
    current_maps_path: Path | None = None,
) -> Tuple[Path, Path]:

```

```

report_dir = report_dir or package_dir / "outputs" / "reports"
ensure_dir(report_dir)
md_path = report_dir / f"{package_dir.name}_generic_identification_report.md"
tex_path = report_dir / f"{package_dir.name}_generic_identification_report.tex"
pdf_path = tex_path.with_suffix(".pdf")
rel_maps = {key: os.path.relpath(path, package_dir).replace("\\", "/") for key, path in
map_assets.items()}
terminal_anchor = terminal_anchor or {}
stage_anchors = stage_anchors or []
day_metrics = day_metrics or []
battery_dynamic_fit = battery_dynamic_fit or {}
fit_plan = fit_plan or {}
...

```

7.55 L3075 関数 main

- 行範囲: L3075–L3841
- 所属: module 直下
- 定義: `main()->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ArgumentParser`, `Path`, `PostRefineResult`, `R_int`, `ValueError`, `add_argument`, `add_battery_conditional_metrics`, `add_end_to_end_metrics`, `apply_battery_polarization`, `attach_archive_pv_model`, `bool`, `build_grounded_map_assets`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `_`, `acceleration_observation_fit`, `adopted_map_assets`, `adopted_model`, `adopted_ocv_df`, `adopted_route_profile`, `adopted_score`, `adopted_stage_anchors`, `allow_map_shape_fit`, `ap`, `args`, `base_cycle_batt_fit`, `base_cycle_battery_conditioned_replay_df`, `base_cycle_end_to_end_replay_df`, `base_cycle_logs`, `base_cycle_metrics`, `base_cycle_mot_fit`, `base_cycle_ocv_df`, `base_cycle_pv_fit`, `base_cycle_replay_df`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError(f'identification options disagree between profile and manifest: {names}'. Keep duplicate YAML values identical or define each option in only one file.)`
- 制御構造の規模: 条件分岐 19、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `ap` に `argparse.ArgumentParser()` の結果を代入する。
2. `ap.add_argument(...)` を実行する。
3. `ap.add_argument(...)` を実行する。

4. `ap.add_argument(...)` を実行する。
5. `ap.add_argument(...)` を実行する。
6. `ap.add_argument(...)` を実行する。
7. `ap.add_argument(...)` を実行する。
8. `ap.add_argument(...)` を実行する。
9. `ap.add_argument(...)` を実行する。
10. `ap.add_argument(...)` を実行する。
11. `args` に `ap.parse_args()` の結果を代入する。
12. `profile_yaml` に `resolve_relative(ROOT, args.profile)` の結果を代入する。
13. `package_dir` に `profile_yaml.parent` の結果を代入する。
14. `stage(...)` を実行する。
15. `profile_cfg` に `load_profile_yaml(profile_yaml)` の結果を代入する。
16. `(manifest_path, manifest)` に `load_manifest(package_dir, args.manifest)` の結果を代入する。
17. `manifest_context` に `resolve_manifest_context(package_dir, manifest)` の結果を代入する。
18. `inputs` に `manifest_context['inputs']` の結果を代入する。
19. `options` に `dict(manifest_context['options'])` の結果を代入する。
20. `identification_cfg` に `profile_cfg.get('identification', {})` or `{}` の結果を代入する。
21. `option_override_keys` に `('fit_quality', 'rebuild_grounded_base_maps', 'use_grounded_base_maps', 'allow_map_shape_fit', 'post_refine_enabled', 'sensor_filter', 'acceleration_observation', 'grade_observation', 'panel_deployment_stopped_speed_kmh', 'panel_deployment_min_dwell_sec', 'panel_deployment_max_sample_gap_sec', 'panel_control_stop_tolerance_kmh', 'battery_restart_count', 'battery_maxiter', 'motion_restart_count', 'motion_maxiter', 'joint_restart_count', 'joint_random_start_count', 'joint_local_topk', 'joint_maxiter', 'fit_stride')` の結果を代入する。
22. `conflicting_options` に `[key for key in option_override_keys if key in identification_cfg and key in options and (identification_cfg[key] != options[key])]` の結果を代入する。
23. 条件 `conflicting_options` を判定し、真なら内部処理を行う。
24. `names` に `', '.join(conflicting_options)` の結果を代入する。
25. `ValueError(f'identification options disagree between profile and manifest: {names}. Keep duplicate YAML values identical or define each option in only one file.')` を送出する。
26. `option_override_keys` を順に走査し、各要素を `key` に入れて処理する。
27. 条件 `key in identification_cfg` を判定し、真なら内部処理を行う。
28. `options[key]` に `identification_cfg[key]` の結果を代入する。
29. `fit_plan` に `resolve_fit_plan(options, quality=args.quality)` の結果を代入する。
30. `panel_deployment_options` に `{'stopped_speed_kmh': float(fit_plan['panel_deployment_stopped_speed_kmh']), 'deployment_min_dwell_sec': float(fit_plan['panel_deployment_min_dwell_sec']), 'deployment_max_sample_gap_sec': float(fit_plan['panel_deployment_max_sample_gap_sec']), 'horizontal_control_stop_kmh': declared_control_stop_kmh(profile_yaml), 'control_stop_tolerance_kmh': float(fit_plan['panel_control_stop_tolerance_kmh'])}` の結果を代入する。

31. `output_layout` に `resolve_identification_output_layout(package_dir, profile_cfg, output_tag_override=args.output_tag)` の結果を代入する。
32. `run_output_dir` に `Path(output_layout['run_root'])` の結果を代入する。
33. `report_output_dir` に `Path(output_layout['report_root'])` の結果を代入する。
34. 条件 `args.manifest_only` を判定し、真なら内部処理を行う。
35. `payload` に `{'profile_yaml': relpath_from(ROOT, profile_yaml), 'manifest_yaml': relpath_from(ROOT, manifest_path), 'actual_event_yaml': relpath_from(package_dir, manifest_context.get('actual_event_path')), 'counterfactual_event_yaml': relpath_from(package_dir, manifest_context.get('counterfactual_event_path')), 'terminal_anchor_yaml': relpath_from(package_dir, manifest_context.get('terminal_anchor_path')), 'grounded_map_summary_yaml': relpath_from(package_dir, manifest_context.get('grounded_summary_path')), 'source_inventory_json': relpath_from(package_dir, manifest_context.get('source_inventory_path')), 'notes_markdown': relpath_from(package_dir, manifest_context.get('notes_markdown_path')), 'explicit_grounded_assets': {key: relpath_from(package_dir, path) for key, path in manifest_context.get('explicit_grounded_assets', []).items()}, 'external_documents': list(manifest_context.get('external_documents', []))}` の結果を代入する。
36. `print(...)` を実行する。
37. を返す。
38. `base_model` に `build_model_from_profile_cfg(profile_cfg, profile_yaml)` の結果を代入する。
39. `source_map_assets` に `build_source_map_assets(profile_cfg, profile_yaml)` の結果を代入する。
40. `grounded_map_summary` に `dict(manifest_context.get('grounded_summary_payload', {}) or {})` を代入する。
41. `explicit_grounded_assets` に `manifest_context.get('explicit_grounded_assets', {})` の結果を代入する。
42. `rebuild_grounded` に `bool(args.rebuild_grounded_base_maps or options.get('rebuild_grounded_base_maps', False))` の結果を代入する。
43. 条件 `rebuild_grounded` を判定し、真なら内部処理を行う。
44. `stage(...)` を実行する。
45. (`source_map_assets, grounded_map_summary`) に `build_grounded_map_assets(profile_cfg, Path(output_layout['grounded_map_summary_path']), pv_area_m2=float((profile_cfg.get('model', {}) or {}).get('pv_area', 6.0)), base_dir=package_dir)` の結果を代入する。
46. `grounded_summary_file` に `Path(str(grounded_map_summary.get('summary_yaml', '') or '')).resolve()` の結果を代入する。
47. `grounded_map_summary['summary_yaml']` に `relpath_from(package_dir, grounded_summary_file)` の結果を代入する。
48. `evidence_summary_path` に `manifest_context.get('grounded_summary_path')` の結果を代入する。
49. 条件 `evidence_summary_path is not None and (not str(output_layout['tag']))` を判定し、真なら内部処理を行う。
50. `payload` に `dict(grounded_map_summary)` の結果を代入する。
51. `payload.pop(...)` を実行する。
52. `with` 文で `Path(evidence_summary_path).open('w', encoding='utf-8', newline='\n')` を管理しながら処理する。
53. `yaml.safe_dump(...)` を実行する。
54. `manifest_context['grounded_summary_payload']` に `payload` の結果を代入する。

55. `base_model` に `build_model_from_map_assets(base_model, source_map_assets)` の結果を代入する。
56. `ocv_df` に `pd.read_csv(source_map_assets['ocv_soc_map'])` の結果を代入する。
57. 上の条件が偽の場合:
58. 条件 `explicit_grounded_assets` を判定し、真なら内部処理を行う。
59. `stage(...)` を実行する。
60. `source_map_assets` に `explicit_grounded_assets` の結果を代入する。
61. `base_model` に `build_model_from_map_assets(base_model, source_map_assets)` の結果を代入する。
62. `ocv_df` に `pd.read_csv(source_map_assets['ocv_soc_map'])` の結果を代入する。
63. 上の条件が偽の場合:
64. 条件 `bool(options.get('use_grounded_base_maps', False))` を判定し、真なら内部処理を行う。
65. `stage(...)` を実行する。
66. (`source_map_assets`, `grounded_map_summary`) に `build_grounded_map_assets(profile_cfg, Path(output_layout['grounded_map_summary'], 'grounded_map_summary.yaml'), pv_area_m2=float((profile_cfg.get('model', {}) or {}).get('pv_area', 6.0)), base_dir=package_dir)` の結果を代入する。
67. `grounded_summary_file` に `Path(str(grounded_map_summary.get('summary_yaml', '') or '')).resolve()` の結果を代入する。
68. `grounded_map_summary['summary_yaml']` に `relpath_from(package_dir, grounded_summary_file)` の結果を代入する。
69. `base_model` に `build_model_from_map_assets(base_model, source_map_assets)` の結果を代入する。
70. `ocv_df` に `pd.read_csv(source_map_assets['ocv_soc_map'])` の結果を代入する。
71. 上の条件が偽の場合:
72. `ocv_df` に `load_ocv_df(profile_cfg, profile_yaml)` の結果を代入する。
73. `base_model` に `neutralize_identification_scalars(base_model)` の結果を代入する。
74. `raw_log` に `str(inputs.get('normalized_replay_log_csv', 'data/identification/raw/observed_replay_log.csv') or '').strip()` の結果を代入する。
75. `observed_log_csv` に `resolve_relative(package_dir, raw_log)` の結果を代入する。
76. `stage(...)` を実行する。
77. `logs` に `normalize_generic_log(observed_log_csv, actual_event_payload=manifest_context.get('actual_event_payload', {}), base_model=base_model, options=fit_plan)` の結果を代入する。
78. `stage(...)` を実行する。
79. (`logs`, `solar_measurement_calibration`) に `calibrate_solar_measurement_to_pack(logs, known_aux_power_w=float((profile_cfg.get('P_aux', 21.0)) or {}).get('P_aux', 21.0)))` の結果を代入する。
80. `fit_plan['solar_measurement_calibration']` に `solar_measurement_calibration` の結果を代入する。

代表コード断片

```
def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--profile", required=True)
    ap.add_argument("--manifest")
    ap.add_argument(
        "--quality",
        choices=sorted(FIT_QUALITY_PRESETS.keys()),
        default=None,
        help="Override identification.fit_quality; omit to use the profile/manifest YAML value.",
    )
    ap.add_argument("--output-tag", help="Override identification.output_tag for versioned artifacts.")
    ap.add_argument(
        "--adopt-profile",
        action="store_true",
        help="Allow a tagged run to update the canonical profile.yaml; otherwise write a candidate profile in the run directory.",
    )
    ap.add_argument("--allow-map-shape-fit", action="store_true")
    ap.add_argument("--skip-map-shape-fit", action="store_true")
    ap.add_argument(
        "--rebuild-grounded-base-maps",
        action="store_true",
        help="Rebuild product/theory grounded maps before fitting instead of reusing manifest assets.",
    )
    ap.add_argument("--manifest-only", action="store_true")
    args = ap.parse_args()

    profile_yaml = resolve_relative(ROOT, args.profile)
    package_dir = profile_yaml.parent
    stage(f"loading profile: {profile_yaml}")
    profile_cfg = load_profile_yaml(profile_yaml)
    manifest_path, manifest = load_manifest(package_dir, args.manifest)
    manifest_context = resolve_manifest_context(package_dir, manifest)
    inputs = manifest_context["inputs"]
    options = dict(manifest_context["options"])
    identification_cfg = profile_cfg.get("identification", {}) or {}
    ...
```

7.56 CLI 引数

行	引数
3077	--profile
3078	--manifest
3079	--quality
3085	--output-tag
3086	--adopt-profile
3091	--allow-map-shape-fit
3092	--skip-map-shape-fit
3093	--rebuild-grounded-base-maps
3098	--manifest-only

8 処理の流れ

1. CLI 引数を解釈し、`main()` から処理を起動する。

9 このファイルの位置づけ

この資料群では、`scripts/run_vehicle_identification.py` を **identification** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。